

ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA SOFTWARE

DESARROLLO DE UNA APLICACIÓN WEB PARA LA GESTIÓN DE LOS SERVICIOS EN LA EMPRESA "HOSPITAL DEL COMPUTADOR" INTEGRANDO PRINCIPIOS UX/UI

Trabajo de Titulación

Modalidad: Proyecto técnico

Presentado para optar al grado académico de:

INGENIERO EN SOFTWARE

AUTOR:

KEVIN ALEXANDER URBANO LOMBEIDA

Riobamba – Ecuador

2026



ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA SOFTWARE

DESARROLLO DE UNA APLICACIÓN WEB PARA LA GESTIÓN DE LOS SERVICIOS EN LA EMPRESA "HOSPITAL DEL COMPUTADOR" INTEGRANDO PRINCIPIOS UX/UI

Trabajo de Titulación

Modalidad: Proyecto técnico

Presentado para optar al grado académico de:

INGENIERO EN SOFTWARE

AUTOR: KEVIN ALEXANDER URBANO LOMBEIDA

DIRECTOR: ING. JORGE ARIEL MENENDEZ VERDECIA

Riobamba – Ecuador

2026

DERECHO DE AUTOR (COPYRIGHT)

© 2026, Kevin Alexander Urbano Lombeida

Se autoriza la reproducción total o parcial, con fines académicos, por cualquier medio o procedimiento, incluyendo la cita bibliográfica del documento, siempre y cuando se reconozca el Derecho de Autor.

CARTA DE CESIÓN DE DERECHOS PATRIMONIALES DE AUTOR DE TRABAJO DE TITULACIÓN DE GRADO

Yo, Kevin Alexander Urbano Lombeida con cédula de identidad 2300439680, en calidad de estudiante, en uso de mis facultades mentales y legales para contratar, libre y voluntariamente convengo en ceder a favor de la Escuela Politécnica de Chimborazo de manera total y exclusiva, la titularidad sobre los derechos patrimoniales de autor del trabajo de titulación denominado: "DESARROLLO DE UNA APLICACIÓN WEB PARA LA GESTIÓN DE LOS SERVICIOS EN LA EMPRESA "HOSPITAL DEL COMPUTADOR" INTEGRANDO PRINCIPIOS UX/UI", esta cesión la realizo en virtud de que soy autor puesto que lo he desarrollado como parte del proceso de formación académica superior para optar por el título de Ingeniero de Software, en la carrera de Software, de la Facultad de Informática y Electrónica de la Escuela Superior Politécnica de Chimborazo , a través de la presente cesión la ESPOCH está facultada para ejercer plenamente los derechos cedidos.

En la aplicación a lo determinado en el artículo 118 del Código Orgánico de la Economía Social de los Conocimientos, Creatividad e Innovación, en condición de autores nos reservamos los derechos morales de la obra antes citada. En concordancia a lo expuesto, suscribimos el presente documento en el momento que se realiza la entrega final del trabajo de titulación.

Riobamba, 23 de junio de 2026



FIRMA:

NOMBRES Y APELLIDOS: Kevin Alexander Urbano Lombeida

C.I: 2300439680

Yo, **Kevin Alexander Urbano Lombeida**, declaro que el presente Trabajo de Titulación es de mi autoría y los resultados del mismo son auténticos. Los textos en el documento que provienen de otras fuentes están debidamente citados y referenciados.

Como autor asumo la responsabilidad legal y académica de los contenidos de este Trabajo de Titulación; el patrimonio intelectual pertenece a la Escuela Superior Politécnica de Chimborazo.

Riobamba, 23 de junio de 2026



Kevin Alexander Urbano Lombeida
2300439680

ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO
FACULTAD DE INFORMÁTICA Y ELECTRÓNICA
CARRERA SOFTWARE

El Tribunal del Trabajo de Titulación certifica que: El Trabajo de Titulación; **DESARROLLO DE UNA APLICACIÓN WEB PARA LA GESTIÓN DE LOS SERVICIOS EN LA EMPRESA "HOSPITAL DEL COMPUTADOR" INTEGRANDO PRINCIPIOS UX/UI,** Modalidad: Proyecto técnico, realizado por el señor: **KEVIN ALEXANDER URBANO LOMBEIDA**, ha sido minuciosamente revisado por los Miembros del Tribunal del Trabajo de Titulación, el mismo que cumple con los requisitos científicos, técnicos, legales, en tal virtud el Tribunal Autoriza su presentación.

	FIRMA	**FECHA**
Ing. / Dr. Nombres y Apellidos **PRESIDENTE DEL TRIBUNAL**	_____	aa-mm-dd
Ing. / Dr. Nombres y Apellidos **COORDINADOR DE LA CARRERA / DELEGADO**	_____	aa-mm-dd
Ing. Jorge Ariel Menendez Verdecia **DIRECTOR(A) DEL TRABAJO DE TITULACIÓN**	_____	aa-mm-dd

DEDICATORIA

Dedico este trabajo a mi familia, quienes han sido un pilar fundamental en mi vida. Su amor incondicional, su apoyo constante y su preocupación por mi bienestar me han dado la fuerza necesaria para seguir adelante en cada etapa de este camino. También extiendo mi agradecimiento a mis compañeros de carrera, con quienes no solamente compartí aulas y proyectos, sino también experiencias, aprendizajes y momentos que atesoro profundamente. Gracias a su amistad, su apoyo y compañerismo, esta etapa de mi vida universitaria ha sido inolvidable.

Con cariño,

Kevin Urbano

AGRADECIMIENTO

Expresando un profundo agradecimiento a la Escuela Superior Politécnica de Chimborazo al brindarme la oportunidad de proporcionarme de conocimientos y herramientas en la institución, las cuales fueron fundamentales para mi formación personal y académica. De igual manera agradecer al Ing. Jorge Ariel Menendez, por su guía durante todo el proceso de desarrollo del presente trabajo de titulación, con su experiencia y conocimiento permitiendo su culminación. También agradecer a los docentes de la carrera, los cuales han guiado y preparado para no rendirme y dar lo mejor en el mundo laboral.

Kevin Urbano

RESUMEN

El presente trabajo de titulación aborda el diseño y desarrollo de una aplicación web para la gestión de servicios y mantenimiento de equipos informáticos en la empresa "Hospital del Computador", en la ciudad de Riobamba. Tradicionalmente, la organización operaba mediante un sistema local monousuario y un flujo de comunicación informal a través de WhatsApp, lo que limitaba la trazabilidad y generaba retrasos en la atención. Con el fin de mitigar estos problemas, se propuso el desarrollo de un ecosistema digital multiusuario integrando principios de experiencia de usuario (UX/UI) y accesibilidad web. La investigación se sustentó en un enfoque analítico y evaluativo bajo la metodología ágil Scrum, estructurada en ocho sprints de desarrollo. Para la implementación del software se utilizó una arquitectura multicapa (N-capas) lógicamente desacoplada y físicamente distribuida, compuesta por una capa de presentación en Next.js (App Router), una capa de lógica de negocio en NestJS y una capa de datos en PostgreSQL mapeada con el ORM TypeORM. Los principales resultados incluyeron la automatización del registro de clientes mediante la validación tributaria asíncrona del SRI, un módulo de diagnóstico técnico estructurado a través de listas de verificación dinámicas en formato JSONB, y un portal web público que permite a los clientes consultar sus órdenes y autorizar presupuestos en tiempo real. Finalmente, la validación del sistema se realizó combinando auditorías automáticas mediante WAVE y Lighthouse. Los resultados evidenciaron el cumplimiento de los criterios de conformidad Nivel AA de las pautas de accesibilidad WCAG 2.1, demostrando la viabilidad de la plataforma para reducir la carga cognitiva de los operarios, agilizar los tiempos de respuesta y ofrecer un servicio inclusivo y eficiente.

Palabras clave: <Aplicación web>, <Gestión de servicios>, <Accesibilidad web>, <Diseño UX/UI>, <Metodología Scrum>

Ing. Jorge Ariel Menéndez Verdecia
DIRECTOR DEL TRABAJO DE TITULACIÓN

SUMMARY / ABSTRACT

Una vez que tenga la traducción de un docente de inglés debe incluir en este espacio de forma textual.

Keywords: < >, < >, < >, < >, < >, < >, < >, < >. [máximo cinco descriptores que definan el contenido de la investigación]

ÍNDICE DE CONTENIDO

RESUMEN vii
SUMMARY / ABSTRACT viii
ÍNDICE DE TABLAS iv
ÍNDICE DE ILUSTRACIONES v
ÍNDICE DE ANEXOS vi
INTRODUCCIÓN 1
CAPÍTULO I 2
1. DIAGNÓSTICO DEL PROBLEMA 2
1.1 Antecedentes 2
1.2 Formulación del problema 3
1.3 Sistematización del problema 3
1.4 Justificación teórica 3
1.5 Justificación aplicativa 5
1.6 Objetivos 6
1.6.1 Objetivo general 6
1.6.2 Objetivos específicos 7
CAPÍTULO II 8
2. MARCO TEÓRICO 8
2.1 Sistema de Gestión 8
2.1.1 Tipos de sistemas de gestión 8
2.1.2 Características y funcionalidades de los Sistema de gestión 9
2.1.3 Ventajas y Desafíos de los Sistemas de gestión 10
2.1.4 Sistemas de Gestión de servicios 10
2.1.5 Hospital del Computador 12
2.2 Diseño UX/UI en Aplicaciones Web 17
2.2.1 Principios Fundamentales de UX/UI 18

2.2.2 Elementos de Diseño de UX/UI 19
2.3 Calidad de Software 23
2.3.1 Accesibilidad en Aplicaciones Web 23
2.3.2 Estándar WCAG 23
2.3.3 Criterios de WCAG 24
2.3.4 Instrumentos y Herramientas de Evaluación de la Accesibilidad 25
2.3.5 Importancia de la Accesibilidad 25
2.4 Trabajos relacionados 26
CAPÍTULO III 28
3. MARCO METODOLÓGICO 28
3.1 Tipo de estudio 28
3.1.1 Métodos y Técnicas 28
3.2 Identificar el proceso de gestión de servicios 29
3.2.1 Planificación 30
3.2.2 Diseño de entrevista 31
3.2.3 Aplicación 31
3.2.4 Registro 32
3.2.5 Resultados obtenidos 32
3.2.6 Limitaciones encontradas 32
3.3 Identificar características UX/UI y accesibilidad 33
3.3.1 Análisis de principios UX/UI 33
3.3.2 Análisis de accesibilidad web 33
3.3.3 Técnicas de recopilación de información 34
3.3.4 Fuentes de información 35
3.4 Desarrollo de los módulos para la aplicación web utilizando características UX/UI 35
3.4.1 Fase de análisis y planificación 36
3.4.2 Fase de planeación y estimación 43
3.4.3 Fase de implementación 49
3.4.4 Fase de revisión y retrospectiva 54

3.4.5 Fase de lanzamiento 56
3.5 Evaluación de la accesibilidad de la aplicación web 57
3.5.1 Selección de interfaces para la evaluación 58
3.5.2 Evaluación automática de accesibilidad 58
3.5.3 Análisis de los reportes de evaluación de accesibilidad 59
3.5.4 Verificación manual de la navegación mediante teclado 60
3.5.5 Criterios de evaluación de accesibilidad 60
3.5.6 Criterios de conformidad WCAG 62
3.5.7 Fuentes utilizadas 62
CAPÍTULO IV 64
4. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS 64
4.1 Resultados del proceso de evaluación de accesibilidad web 64
4.1.1 Interfaces evaluadas 64
4.1.2 Resultados de la evaluación de accesibilidad web 65
4.1.3 Fase 1: Evaluación diagnóstica con WAVE 66
4.1.4 Fase 2: Evaluación final con Lighthouse 67
4.1.5 Análisis de navegación y accesibilidad manual 70
CONCLUSIONES 71
RECOMENDACIONES 72
BIBLIOGRAFÍA
ANEXOS

ÍNDICE DE TABLAS

Tabla 2-1: Características y funcionalidades de los sistemas de gestión 9

Tabla 2-2: Ventajas y desafíos de los sistemas de gestión 10

Tabla 2-3: Tipos y Características de plataformas de gestión de servicios 11

Tabla 2-4: Ventajas y Desafíos de Implementación de Plataformas de Gestión de Servicios 12

Tabla 2-5: Características Principales de los Aspectos de UX y UI 19

Tabla 2-6: Ventajas y Desventajas del Diseño UX/UI 20

Tabla 3-1: Métodos, técnicas y fuentes. 28

Tabla 3-2 Interfaces seleccionadas para la evaluación de accesibilidad 58

Tabla 3-3 Aspectos verificados durante la navegación mediante teclado 60

Tabla 3-4 Criterios de accesibilidad evaluados e indicadores de verificación 61

Tabla 3-5: Interpretación del puntaje de accesibilidad 61

Tabla 3-6: Niveles de conformidad WCAG 2.1 62

Tabla 4-1: Interfaces evaluadas 64

Tabla 4-2: Consolidado de Resultados de Accesibilidad y Mitigación 69

ÍNDICE DE ILUSTRACIONES

Ilustración 2-1: Ciclo tradicional de atención técnica (Proceso AS-IS)..... 13

Ilustración 2-2: Subprocesos fundamentales del ecosistema web en Hospital del Computador. 14

Ilustración 2-3: Subproceso de registro de clientes..... 14

Ilustración 2-4: Subproceso de ingreso de equipos y generación de ods 15

Ilustración 2-5: subproceso de peritaje técnico y presupuesto 15

Ilustración 4-1: Reporte WAVE Página de Inicio..... 67

Ilustración 4-2: Reporte Lighthouse Final - Inicio de Sesión ..... **¡Error! Marcador no definido.**

Ilustración 4-3: Reporte Lighthouse Final – Dashboard ..... **¡Error! Marcador no definido.**

Ilustración 4-4: Reporte Lighthouse Final – Registro ODS ..... **¡Error! Marcador no definido.**

Ilustración 4-5: Reporte Lighthouse Final - Portal del Cliente ... **¡Error! Marcador no definido.**

ÍNDICE DE ANEXOS

ANEXO A: ENTREVISTA PARA IDENTIFICAR EL PROCESO DE GESTIÓN DE PRÁCTICAS CLÍNICAS EN CONSULTING GROUP ECUADO

ANEXO B: ACTA DE ENTREGA AL CLIENTE

ANEXO C: REPORTE LIGHTHOUSE – ORDEN DE SERVICIO

INTRODUCCIÓN

En la actualidad, la transformación digital se ha convertido en un pilar fundamental para optimizar la competitividad y eficiencia de las pequeñas y medianas empresas. En el sector de los servicios técnicos informáticos, la incorporación de plataformas basadas en la nube y arquitecturas distribuidas de N-capas permite centralizar la información operativa, automatizar flujos de trabajo administrativos y mitigar los errores humanos inherentes al registro manual. Las aplicaciones web modernas no solo facilitan una gestión multiusuario en tiempo real, sino que también promueven interacciones más transparentes y eficientes entre los proveedores de servicios y los clientes finales.

La empresa “Hospital del Computador”, con sede en la ciudad de Riobamba, enfrentaba severas limitaciones en su control operativo debido al uso de un sistema de escritorio monousuario y a la dependencia de canales de mensajería informal como WhatsApp para notificar a los clientes sobre el avance de sus reparaciones. Esta falta de centralización impedía el acceso simultáneo del personal administrativo y técnico al historial de servicio, obstaculizaba la asignación organizada de casilleros físicos en el laboratorio, y limitaba la trazabilidad del stock de repuestos utilizados en las reparaciones, lo que repercutía en tiempos de espera prolongados y en una baja satisfacción del usuario.

Para solventar esta problemática, el presente trabajo de titulación detalla el diseño, desarrollo e implementación de una aplicación web multiusuario orientada a la gestión de servicios y control de inventarios. Para la construcción del software se adoptó una arquitectura robusta y desacoplada con Next.js (App Router) en el frontend y NestJS en el backend, soportada en PostgreSQL. Un rasgo diferenciador del proyecto consiste en la integración metodológica de principios de diseño centrado en el usuario (UX/UI) y criterios de accesibilidad web bajo el estándar internacional WCAG 2.1 (Nivel AA), asegurando que las interfaces sean intuitivas para el personal técnico del taller, disminuyendo la carga cognitiva y garantizando un acceso inclusivo desde múltiples dispositivos.

CAPÍTULO I

1. DIAGNÓSTICO DEL PROBLEMA

En el presente capítulo se presentará el problema existente en la empresa Hospital del Computador, en la cual se necesita una aplicación web para gestionar los servicios que ofrece la empresa, para brindar una solución accesible, confiable y tecnológica, de igual manera la descripción de los antecedentes, formulación y sistematización del problema, su justificación y los objetivos establecidos para este trabajo de titulación.

1.1 Antecedentes

Actualmente las tecnologías de la Información ofrecen una gran variedad de herramientas que brindan una gran mejora en la gestión de información y automatización de procesos dentro de cualquier área del conocimiento disminuyendo riesgos, costos y ampliando la confiabilidad de los resultados, reduciendo significativamente los tiempos de respuesta de cada uno de los procesos que involucren las Tecnologías de la Información (Salgado Reyes et al., 2024).

En la empresa “Hospital del computador”, dedicada a la reparación y mantenimiento de equipos informáticos, se implementó previamente una aplicación web de gestión de servicios. No obstante, esta aplicación no cumplió con las expectativas de los usuarios debido a deficiencias de accesibilidad, reflejadas en dificultades para la navegación, organización de la información y comprensión de las funcionalidades del sistema, así como en una interfaz poco clara que complicaba la interacción. Estas limitaciones generaron inconvenientes en el uso de la aplicación y desarrollo de las actividades, lo que llevó a que el sistema dejara de ser utilizado en las operaciones de la empresa.

Actualmente, la empresa utiliza un sistema local alojado en un solo computador con una licencia no oficial, el cual presenta ciertas limitaciones. Este sistema no permite el acceso simultáneo de múltiples usuarios desde diferentes dispositivos o estaciones de trabajo, lo que provoca una cola de espera para su utilización. Esto impacta en el registro de ingresos, las modificaciones de datos y la atención al cliente, incrementando los tiempos de espera y afectando tanto al soporte técnico como a la experiencia del cliente. Por otra parte, los clientes consultan con frecuencia el estado de sus equipos, pero en muchos casos no se puede dar una respuesta inmediata, debido a que esta información se gestiona actualmente a través de la aplicación de mensajería WhatsApp.

Para abordar la problemática identificada, se desarrolló una aplicación web para la gestión de servicios, basada en las tecnologías Next.js y Nest.js, orientada a mejorar la accesibilidad del sistema mediante una navegación estructurada, una adecuada disposición de la información y un acceso claro a sus funcionalidades, en concordancia con los principios de UX/UI. La aplicación fue diseñada bajo un enfoque centrado en el usuario, incorporando una interfaz gráfica que facilita la interacción y comprensión del sistema. Asimismo, se implementaron funcionalidades que permiten la interacción simultánea de múltiples usuarios desde distintos dispositivos, superando las limitaciones del sistema local y mejorando el desarrollo de las actividades dentro de la empresa.

1.2 Formulación del problema

¿Cómo se puede mejorar la accesibilidad de una aplicación web desarrollada para la gestión de servicios en la empresa "Hospital del computador", integrando principios de UX/UI?

1.3 Sistematización del problema

¿Cómo se lleva a cabo el proceso de gestión de servicios en la empresa Hospital del computador y cuál es su funcionamiento actual?

¿Qué principios de diseño UX/UI son adecuados para desarrollar una aplicación web que satisfaga las necesidades de la empresa?

¿Cómo se pueden construir los módulos de la aplicación web siguiendo las características definidas en el diseño UX/UI?

¿Qué tan efectiva es la accesibilidad de la aplicación web, evaluada según los criterios establecidos en el estándar WCAG?

1.4 Justificación teórica

Rodríguez García y Villa Arévalo (2022) desarrollaron una investigación enfocada en implementar una aplicación web para optimizar la gestión del control de servicio técnico en B&R Electronics en Iquitos. Su estudio empleó una metodología cuantitativa con diseño pre experimental, aplicada a una muestra de 30 trabajadores del área técnica. Utilizaron la metodología de desarrollo ágil SCRUM y recopilaron datos a través de cuestionarios pre y post implementación. Los resultados arrojaron una mejora significativa en los indicadores de gestión: el tiempo promedio de registro de servicios se redujo en un 62.5%, la generación de reportes ser

redujo de 20 a 5 minutos, y la satisfacción del cliente aumento en un 45%. Concluyeron que la implementación de la aplicación web optimizo los procesos de control de servicios, mejorando la satisfacción del cliente y la eficiencia operativa de sus trabajadores.

Santamaria Ballesteros y Rincón Agredo (2021) ejecutaron un estudio enfocado en desarrollar una aplicación móvil para la gestión de servicios técnicos de computadores, su investigación utilizo un enfoque donde se aplicaron entrevistas a 50 clientes y 15 técnicos. El aplicativo se desarrolló incorporando diferentes funcionalidades como seguimiento en tiempo real, sistema de notificaciones y gestión de inventarios; los resultados evidenciaron una reducción del 40% en tiempo de respuesta a solicitudes de servicio y un incremento del 55% en la eficiencia de asignación de técnicos. Finalmente concluyeron que la implementación de la aplicación móvil optimizo significativamente la gestión de servicios técnicos, mejorando la comunicación entre cliente-técnico y la eficiencia operativa.

En Trujillo - Perú realizaron una investigación para implementar una aplicación móvil orientada a mejorar el servicio de soporte técnico de computadoras. Utilizaron una metodología mixta con diseño cuasi-experimental, aplicando instrumentos a 25 técnicos y 120 usuarios. El desarrollo de la aplicación se realizó utilizando la metodología ágil XP (Extreme Programming) y ciertos principios de diseño enfocados en el usuario. Los resultados mostraron una mejora significativa del 70% en la trazabilidad de los servicios, un aumento del 65% en la satisfacción del usuario, y una reducción del 50% en tiempos de respuesta. Como resultado concluyeron que la aplicación móvil transformo positivamente la gestión de servicio técnico, mejorando la eficiencia operativa y la experiencia del usuario (Chumacero Troncos & Navarro Honores, 2024).

En el estudio “UX y UI: Análisis metodológico de la aplicación y sus principios”, llevaron a cabo un análisis sobre la aplicación de principios UX/UI en el desarrollo de software, utilizaron una metodología de revisión sistemática, analizando 50 casos de estudio y entrevistando 30 diseñadores UX/UI senior. Esta investigación se centró en identificar los principios fundamentales del diseño de experiencia de usuario y las mejores prácticas. Arrojando como resultados la identificación de cinco principios clave: retroalimentación inmediata, navegación intuitiva, diseño emocional, consistencia visual y accesibilidad universal. Y concluyeron que la efectiva implementación de principios UX/UI permiten aumentar significativamente la satisfacción del usuario en un 80% y reducir la tasa de abandono de aplicaciones web en un 60% (Vergara Zurita & Poma Velastegui, 2023).

Los principios establecidos por las Web Content Accessibility Guidelines proporcionan un marco sólido para el diseño de aplicaciones web, organizado en torno a cuatro ejes fundamentales: contenido perceptible, interfaz operable, información comprensible y código robusto (Caldwell et al., 2008). Diversos análisis a gran escala, como el informe anual WebAIM Million evidencian que los problemas de accesibilidad más comunes, como la ausencia de encabezados descriptivos, el contraste insuficiente o el uso de enlaces con texto genérico, constituyen barreras que dificultan la navegación, la comprensión de la información y la interacción dentro de los sistemas web (WebAIM web accessibility in mind, 2025). En este sentido, la aplicación de las WCAG contribuye a establecer una navegación clara, una interacción predecible y una adecuada organización de la información, beneficiando el acceso y uso del sistema por parte de los usuarios en general.

1.5 Justificación aplicativa

En la actualidad las aplicaciones de gestión de servicios han tomado relevancia en los últimos años a razón de que permiten optimizar los procesos y mejorar la capacidad de interacción. Estas aplicaciones brindan diversas herramientas que automatizan tareas repetitivas, facilitan el seguimiento de actividades y manteniendo un control sobre los recursos. Por otra parte, ayudan a centralizar información en una sola plataforma, lo cual ayuda a mejorar significativamente la comunicación entre cliente - técnico y además ofrecer un mejor servicio.

El desarrollo de la aplicación se realiza en base a las necesidades que tiene la empresa Hospital del computador para optimizar la gestión de servicios técnicos, automatizando las distintas funciones que realiza el personal técnico y administrativo. Todo esto tendrá lugar en una aplicación web desarrollada con una arquitectura N-capas, la cual permitirá el registro, seguimiento y control integral de los servicios técnicos de computadoras, así como la generación de informes y reportes para la gerencia.

El sistema contará con los siguientes componentes:

Módulo de registro de clientes y equipos: Permite registrar y gestionar información de clientes y los equipos que requieren servicio técnico.

Módulo de gestión de servicios técnicos: Administra las solicitudes de servicios, desde la recepción hasta su finalización, incluyendo diagnósticos y reparaciones.

Módulo de asignación de técnicos: Facilita la asignación de técnicos a las solicitudes de servicio según disponibilidad y especialidad.

Módulo de seguimiento del estado del servicio: Permite a los clientes y administradores verificar el estado y progreso de los servicios.

Módulo de generación de reportes e informes: Genera reportes detallados sobre el desempeño, estado de los servicios.

Módulo para inventarios: Gestiona el inventario de repuestos y herramientas utilizadas en los servicios técnicos.

Módulo de histórico de servicios realizados: Almacena y permite consultar los registros de servicios técnicos realizados, con detalles de fechas, técnicos y soluciones aplicadas.

La aplicación web integrará principios UX/UI para garantizar una experiencia de usuario intuitiva, accesible y eficiente, facilitando la interacción entre clientes, técnicos y personal administrativo de Hospital del computador.

El presente trabajo corresponde a la línea de investigación de Tecnologías de la Información y Comunicación bajo el programa de Ingeniería de Software de la Escuela Superior Politécnica de Chimborazo, y en los Objetivos de Desarrollo Sostenible corresponde al Objetivo 8 que señala “Trabajo decente y crecimiento económico” con la meta 8.2 que menciona “Lograr niveles más elevados de productividad económica mediante la diversificación, la modernización tecnológica y la innovación, entre otras cosas centrándose en los sectores con gran valor añadido y un uso intensivo de la mano de obra.” (Programa de las Naciones Unidas para el Desarrollo, 2024).

1.6 Objetivos

1.6.1 Objetivo general

Desarrollar una aplicación web para la gestión de servicios en la empresa “Hospital del computador", integrando principios de UX/UI.

1.6.2 Objetivos específicos

- Identificar el proceso de la gestión de servicios de la empresa Hospital del computador para conocer su funcionamiento.
- Identificar las características de UX/UI para mejorar la accesibilidad de la aplicación web.
- Desarrollar los módulos para la aplicación web utilizando las características presentadas en el diseño UX/UI.
- Evaluar la accesibilidad de la aplicación web mediante el estándar Web Content Access Guidelines.

CAPÍTULO II

2. MARCO TEÓRICO

Dentro de este capítulo se presentará de manera minuciosa la información respectiva al sustento técnico y académico del presente proyecto, como la descripción del proceso de desarrollo de la aplicación web de gestión, herramientas utilizadas para la creación del proyecto, su enfoque en la accesibilidad y trabajos relacionados con el tema del proyecto.

2.1 Sistema de Gestión

Los sistemas de gestión surgieron como una solución para mejorar el control y organización de los procesos administrativos dentro de las empresas. Inicialmente, la información se manejaba de forma manual mediante registros físicos, lo que generaba errores, pérdida de datos y lentitud en las operaciones. Con el avance de la informática durante las décadas de 1960 y 1970, comenzaron a desarrollarse sistemas computarizados capaces de automatizar tareas como facturación, nómina y control de inventarios (Laudon & Laudon, 2016).

Posteriormente, el crecimiento de las bases de datos y las tecnologías web permitió la evolución de sistemas más completos e integrados, capaces de gestionar múltiples áreas empresariales dentro de una sola plataforma. Actualmente, los sistemas de gestión web son herramientas fundamentales para optimizar procesos, mejorar la administración de recursos y facilitar el acceso a la información en tiempo real, incorporando además principios de accesibilidad y experiencia de usuario para ofrecer plataformas más eficientes e intuitivas (Pressman & Maxim, 2020).

2.1.1 Tipos de sistemas de gestión

Los sistemas de gestión pueden clasificarse según los procesos y funciones que administran dentro de una organización. Su finalidad principal es centralizar la información, automatizar tareas y facilitar el control de las operaciones empresariales (Laudon & Laudon, 2016).
Entre los principales tipos se encuentran:

- **Sistemas de Gestión Administrativa:** orientados al manejo de clientes, facturación, órdenes y procesos internos de la organización.
- **Sistemas de Gestión de Inventario:** permiten controlar entradas, salidas y disponibilidad de productos o repuestos, facilitando el seguimiento del stock.

- **Sistemas ERP (Enterprise Resource Planning):** integran múltiples áreas de la empresa dentro de una sola plataforma, permitiendo compartir información en tiempo real.
- **Sistemas CRM (Customer Relationship Management):** enfocados en la gestión de relaciones con clientes, seguimiento de solicitudes y mejora de la atención al usuario.
- **Sistemas de Gestión Documental:** destinados al almacenamiento, organización y consulta de archivos digitales y registros administrativos.

2.1.2 Características y funcionalidades de los Sistema de gestión

Los sistemas de gestión incorporan diversas características y funcionalidades orientadas a mejorar la administración, organización y control de los procesos dentro de una organización. Estas herramientas permiten centralizar información, automatizar tareas y optimizar el flujo de trabajo, contribuyendo a una gestión más eficiente de los recursos empresariales (Laudon & Laudon, 2016).

La **Tabla 2-1** resume las principales características y funcionalidades de los sistemas de gestión, considerando aspectos relacionados con la administración de información, automatización de procesos y control operativo.

Tabla 2-1: Características y funcionalidades de los sistemas de gestión

Aspecto	Descripción
Centralización de información	Permite almacenar y administrar la información en una única plataforma, facilitando su organización y acceso.
Acceso en tiempo real	Facilita la consulta y actualización inmediata de datos desde distintos dispositivos y ubicaciones.
Automatización de procesos	Reduce tareas manuales y minimiza errores operativos mediante procesos automatizados.
Seguridad de la información	Incorpora mecanismos de autenticación y control de acceso para proteger los datos del sistema.
Escalabilidad	Permite adaptarse al crecimiento y a nuevas necesidades de la organización.
Interfaz intuitiva	Ofrece entornos visuales fáciles de utilizar para mejorar la experiencia del usuario.
Registro y actualización de datos	Permite ingresar, modificar y eliminar información relacionada con usuarios, productos o servicios.
Control de inventario	Facilita el seguimiento de entradas, salidas y disponibilidad de productos o repuestos.
Generación de reportes	Permite obtener estadísticas e informes para apoyar la toma de decisiones.
Gestión de usuarios y roles	Administra permisos y accesos según el tipo de usuario dentro del sistema.
Seguimiento de actividades	Facilita el monitoreo de procesos y tareas realizadas dentro de la plataforma.
Consulta de información	Permite acceder rápidamente a datos organizados y actualizados.

Fuente: (Laudon & Laudon, 2016; Pressman & Maxim, 2020)
Elaborado por: Urbano K., 2026

2.1.3 Ventajas y Desafíos de los Sistemas de gestión

Los sistemas de gestión ofrecen múltiples beneficios para las organizaciones, debido a que permiten optimizar procesos, mejorar el control de información y aumentar la eficiencia operativa. Sin embargo, su implementación también puede presentar desafíos relacionados con costos, capacitación y adaptación tecnológica dentro de la empresa (Laudon & Laudon, 2016).

La **Tabla 2-2** resume las principales ventajas y desafíos de los sistemas de gestión, considerando aspectos operativos, tecnológicos y administrativos.

Tabla 2-2: Ventajas y desafíos de los sistemas de gestión

Aspecto	Ventajas	Desventajas / Desafíos
Procesos Operativos	- Automatización de tareas y reducción de errores operativos. - Mayor eficiencia y productividad en las actividades empresariales.	- La transición de procesos manuales a digitales puede generar resistencia al cambio.
Gestión de Información	- Centralización y mejor organización de los datos. - Acceso y actualización de información en tiempo real.	- Riesgos de seguridad si no se implementan mecanismos adecuados de protección de datos.
Administración y Control	- Generación de reportes para apoyar la toma de decisiones. - Mejor supervisión de recursos, usuarios y actividades.	- Dependencia tecnológica ante fallas del sistema o problemas de conexión.
Implementación Tecnológica	- Permite optimizar el flujo de trabajo y mejorar la gestión organizacional.	- Requiere inversión en desarrollo, infraestructura y mantenimiento. - Necesidad de capacitación constante para el personal.

Fuente: (Laudon & Laudon, 2016; Pressman & Maxim, 2020)
Elaborado por: Urbano K., 2026

2.1.4 Sistemas de Gestión de servicios

Según Widianto & Subriadi (2022), la gestión de servicios comprende un conjunto de prácticas, procesos y tecnologías diseñadas para planificar, entregar, operar y mejorar los servicios que una organización ofrece, con el fin de maximizar el valor para el cliente y la eficiencia operativa. En

el contexto de las Tecnologías de la Información (TI), se suele guiar por marcos de referencia como ITIL 4, los cuales enfatizan el ciclo de valor del servicio y la mejora continua.

2.1.4.1 Plataformas de Gestión de Servicios: Tipos y Características

Las organizaciones implementan diferentes tipos de sistemas de gestión de servicios mediante plataformas especializadas que optimizan procesos operativos y administrativos.
La **Tabla 2-3** presenta los principales tipos de plataformas de gestión de servicios, junto con su propósito principal y las funcionalidades más relevantes que aportan a la eficiencia de los procesos empresariales.

Tabla 2-3: Tipos y Características de plataformas de gestión de servicios

Plataforma	Propósito Principal	Funcionalidades Clave
ITSM & Service Desk	Gestión integral de servicios TI y punto único de contacto con usuarios	• Centralización y automatización del ciclo de tickets. • Gestión de incidentes, problemas y cambios. • Base de conocimiento integrada.
Field Service Management (FSM)	Coordinación de servicios técnicos en campo o taller	• Planificación y asignación de órdenes de trabajo. • Gestión de inventarios de repuestos. • Aplicaciones móviles para actualización en tiempo real. • Seguimiento de actividades técnicas.
Automatización Inteligente	Optimización de procesos mediante tecnologías emergentes	• Clasificación automática de tickets con ML y NLP. • Enrutamiento inteligente de solicitudes. • Reducción de tareas manuales repetitivas. • Mejora de tiempos de respuesta.

Fuentes: (Adinata Arin* et al., s. f., s. f.; Marcapura Campos, 2023; Moreano Anton, 2022; Zangari et al., 2023)
Elaborado por: Urbano K., 2026

2.1.4.2 Ventajas y Desafíos de Implementación

La implementación de plataformas de gestión de servicios conlleva una serie de beneficios que fortalecen la eficiencia operativa, la experiencia del cliente y la automatización de procesos. Sin embargo, también presenta desafíos asociados con la inversión tecnológica, la gestión del cambio organizacional y la dependencia de las herramientas digitales.

La **Tabla 2-4** resume las principales ventajas y desventajas identificadas durante la adopción de estas plataformas, considerando los aspectos operativos, de experiencia del usuario y de automatización.

Tabla 2-4: Ventajas y Desafíos de Implementación de Plataformas de Gestión de Servicios

Aspecto	Ventajas	Desventajas / Desafíos
Eficiencia Operativa	- Reducción de tiempos de resolución (MTTR) y mejor cumplimiento de Acuerdo de Nivel de Servicio (SLAs) gracias a la estandarización de procesos. - Aumento de la productividad operativa mediante flujos de trabajo optimizados.	- Requiere inversión inicial en licencias, configuración y capacitación del personal. - Puede haber resistencia al cambio durante la adopción del sistema.
Experiencia del Cliente	- Mejora la satisfacción y fidelización gracias a la trazabilidad y comunicación proactiva. - Facilita el autoservicio del cliente y la consulta del estado de sus solicitudes.	- Si el sistema no está bien configurado o actualizado, puede generar demoras en la comunicación con el cliente. - Depende de la adopción activa del cliente en el uso de la plataforma.
Automatización	- Disminuye tareas manuales repetitivas mediante reglas de negocio e inteligencia artificial. - Mejora la consistencia en la ejecución de procesos.	- Requiere mantenimiento constante de las reglas y configuraciones para evitar errores. - Puede generar dependencia tecnológica excesiva.

Fuentes: (Espinoza Yunga, 2024; Tedja et al., 2024; Zangari et al., 2023)
Elaborado por: Urbano K., 2026

2.1.5 Hospital del Computador

Hospital del Computador es una empresa privada fundada en el año 2020 en la ciudad de Riobamba, dedicada a la prestación de servicios de soporte técnico, mantenimiento preventivo y correctivo, y comercialización de equipos informáticos. La organización ofrece soluciones tecnológicas orientadas a computadoras portátiles, equipos de escritorio e impresoras, atendiendo tanto a clientes corporativos como a usuarios finales.

La empresa dispone de un taller técnico especializado para el diagnóstico y reparación de equipos informáticos, así como procesos organizados para la gestión de órdenes de servicio, control de equipos y seguimiento de reparaciones. Entre sus actividades principales se incluyen la recepción de equipos, el registro de incidencias, la elaboración de diagnósticos técnicos, la gestión de repuestos y la comunicación con los clientes durante el proceso de reparación.

De igual manera, Hospital del Computador mantiene procedimientos orientados al control operativo y la calidad del servicio, priorizando la trazabilidad de los equipos, la organización de las órdenes de trabajo y el cumplimiento de tiempos de atención. Estas características evidencian la necesidad de contar con herramientas tecnológicas que permitan optimizar la gestión de servicios y mejorar la experiencia del usuario en entornos de soporte técnico.

2.1.5.1 Procesos de gestión de soporte técnico y mantenimiento

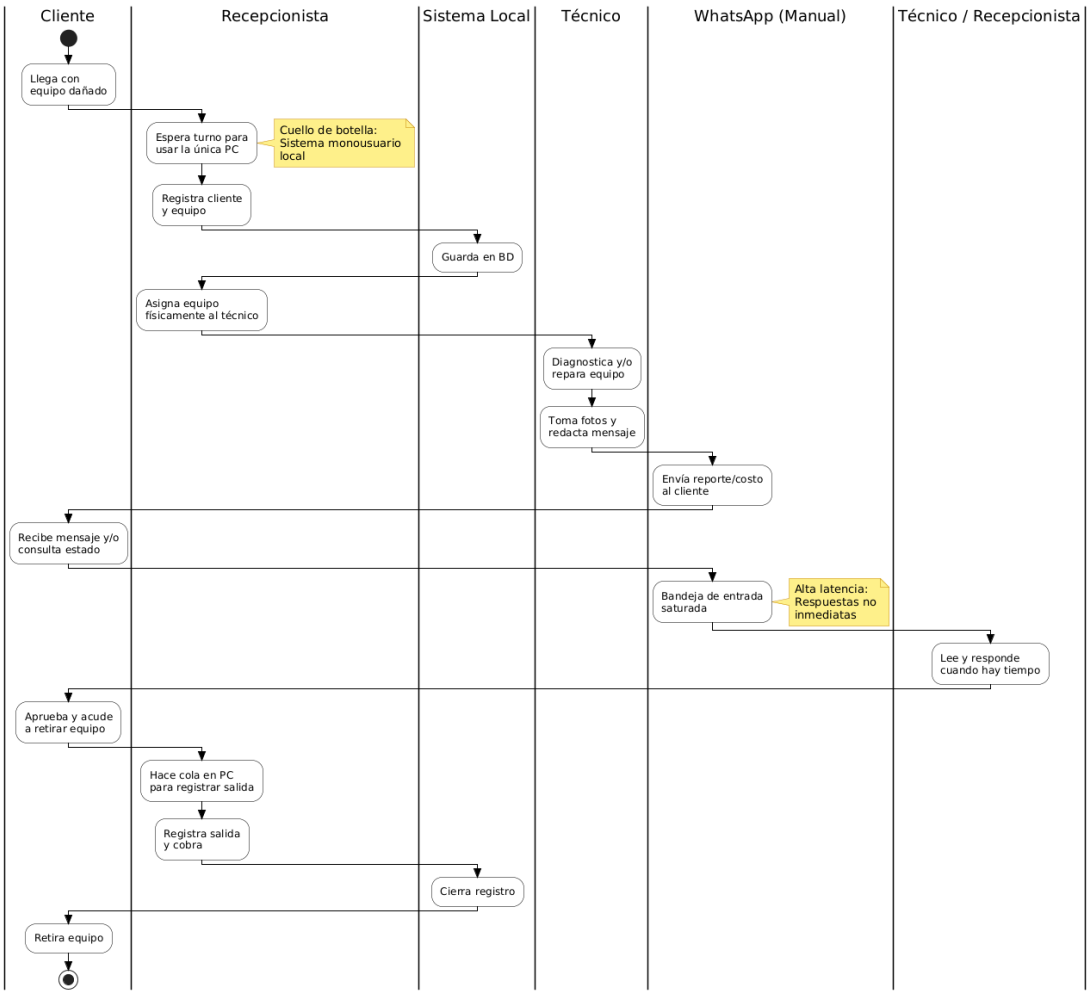


Ilustración 2-1: Ciclo tradicional de atención técnica (Proceso AS-IS)

Elaborado por: Urbano K., 2026

El proceso tradicional de gestión de soporte técnico de Hospital del Computador, como se muestra en la **Ilustración 2-1**, inicia cuando el cliente acude presencialmente a recepción con un equipo informático que presenta alguna falla. Una vez recibido el dispositivo, el recepcionista registra manualmente los datos del cliente y del equipo para posteriormente asignarlo a un técnico encargado de realizar la revisión correspondiente.

En el modelo de trabajo anterior, las actividades se realizaban mediante un sistema local instalado en una sola computadora, lo que generaba diversos inconvenientes operativos. Debido a que no era posible el acceso simultáneo de varios usuarios, el personal debía esperar turnos para registrar información o actualizar estados de las órdenes, ocasionando retrasos en la atención y en los procesos internos.

Además, el seguimiento de los equipos, el envío de diagnósticos y la comunicación de presupuestos se realizaban principalmente mediante WhatsApp. Esta forma de trabajo dificultaba mantener un control organizado de la información, provocando demoras en las respuestas al cliente y limitando la trazabilidad de las reparaciones realizadas.

Con el propósito de solucionar estas limitaciones y centralizar la información del proceso técnico, se planteó una propuesta basada en una aplicación web multiusuario. Para ello, el proceso general fue dividido en distintos subprocesos representados mediante diagramas BPMN (Business Process Model and Notation), permitiendo describir de manera clara las actividades y responsabilidades de cada actor involucrado.

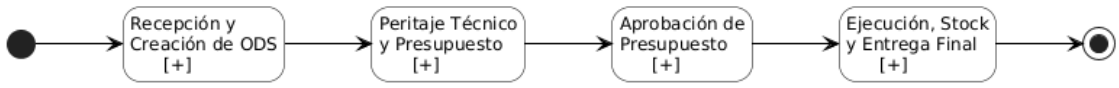


Ilustración 2-2: Subprocesos fundamentales del ecosistema web en Hospital del Computador
Elaborado por: Urbano K., 2026

En la **Ilustración 2-2** se presentan los principales subprocesos que forman parte del sistema web propuesto, abarcando desde el registro del cliente hasta la entrega final del equipo reparado.

2.1.5.2 Subproceso de registro de clientes y validación tributaria

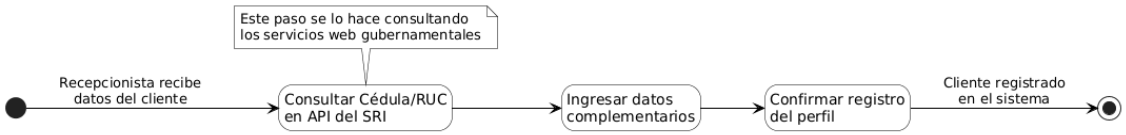


Ilustración 2-3: Subproceso de registro de clientes
Elaborado por: Urbano K., 2026

El subproceso de registro de clientes corresponde a la primera etapa dentro del sistema. Como se observa en la **Ilustración 2-3**, el recepcionista es el encargado de ingresar la información del cliente mediante la interfaz de la aplicación web.

El proceso inicia con la verificación de identidad del usuario a través de su número de cédula o RUC. Posteriormente, el sistema consulta la información tributaria correspondiente para autocompletar datos como nombres o razón social, reduciendo errores en el registro manual.

Una vez verificados los datos, se ingresan otros campos complementarios como correo electrónico, dirección y número telefónico. Finalmente, la información queda almacenada en la base de datos, permitiendo que el cliente pueda registrar órdenes de servicio asociadas a su cuenta.

2.1.5.3 Subproceso *de ingreso de equipos y apertura de ODS*

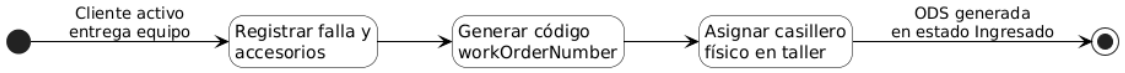


Ilustración 2-4: Subproceso de ingreso de equipos y generación de ods

Elaborado por: Urbano K., 2026

Como se detalla en la **Ilustración 2-4**, una vez registrado el cliente, el recepcionista procede a ingresar la información del equipo que será revisado en el taller técnico.

Durante este proceso se registran datos como marca, modelo, tipo de equipo y accesorios entregados junto al dispositivo. Además, se documentan las fallas o síntomas reportados por el cliente al momento de la recepción.

Al finalizar el registro, el sistema genera automáticamente un número único de orden de servicio (ODS), permitiendo identificar y dar seguimiento al equipo durante todo el proceso de reparación. Posteriormente, la orden se asigna a un casillero disponible dentro del taller y queda en estado "Ingresado" para su revisión técnica.

2.1.5.4 Subproceso de peritaje técnico y cotización de presupuestos

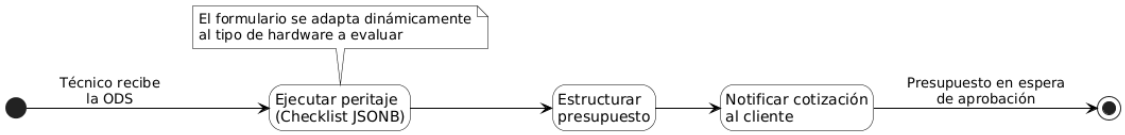


Ilustración 2-5: Subproceso de peritaje técnico y presupuesto

Elaborado por: Urbano K., 2026

Una vez asignada la orden al área técnica, el técnico responsable procede a realizar el diagnóstico del equipo, como se muestra en la **Ilustración 2-5**.

Durante esta etapa, el técnico registra los resultados de las pruebas realizadas y documenta las posibles fallas detectadas según el tipo de dispositivo. Posteriormente, el sistema permite elaborar un presupuesto considerando los repuestos necesarios y los servicios técnicos requeridos para la reparación.

Al guardar la información, el sistema calcula automáticamente los valores correspondientes y genera una cotización en estado "Pendiente de aprobación". Además, el cliente recibe una notificación con un enlace para revisar el presupuesto y decidir si acepta o no la reparación propuesta.

2.1.5.5 Subproceso de ejecución de reparación, control de stock y entrega



Ilustración 2-6: Subproceso de reparación, stock y entrega final

Elaborado por: Urbano K., 2026

Cuando el cliente aprueba el presupuesto, la orden cambia automáticamente al estado "En reparación", permitiendo que el técnico inicie el trabajo correspondiente, como se representa en la **Ilustración 2-6**.

Durante la reparación, el técnico registra las actividades realizadas e incorpora evidencias fotográficas relacionadas con el proceso. Asimismo, el sistema actualiza automáticamente el stock de repuestos utilizados durante la intervención técnica.

Una vez finalizada la reparación y realizadas las pruebas necesarias, el equipo pasa al estado "Terminado". Posteriormente, el sistema notifica al cliente que el dispositivo está listo para ser retirado.

El proceso concluye cuando el cliente recibe el equipo reparado y se registra la entrega final en el sistema. De esta manera, toda la información queda centralizada y disponible para futuras consultas o seguimientos administrativos.

A partir del análisis del proceso tradicional de soporte técnico de Hospital del Computador, fue posible identificar diversas limitaciones relacionadas con el uso de un sistema local y métodos manuales de comunicación. Frente a esta situación, la implementación de una aplicación web

basada en tecnologías como Next.js, NestJS y PostgreSQL permite optimizar el flujo de trabajo, centralizar la información y mejorar el seguimiento de las órdenes de servicio, contribuyendo a una gestión más eficiente y organizada.

2.2 Diseño UX/UI en Aplicaciones Web

El diseño UX/UI en aplicaciones web constituye un componente esencial en el desarrollo de sistemas modernos, ya que integra la experiencia del usuario (UX) y la interfaz de usuario (UI) con el propósito de ofrecer plataformas funcionales, accesibles y visualmente atractivas. Mientras que el UX se orienta en comprender las necesidades, expectativas y comportamientos del usuario para optimizar su experiencia de interacción, el UI se centra en los aspectos gráficos y visuales de la interfaz. La combinación de ambos enfoques permite desarrollar aplicaciones que no solo cumplen objetivos funcionales, sino que también garantizan la satisfacción del usuario final.

En la actualidad, el crecimiento de las tecnologías digitales y la alta competitividad en el desarrollo de aplicaciones web han provocado que las organizaciones prioricen la experiencia del usuario como un factor estratégico dentro de sus sistemas informáticos. Un diseño deficiente puede generar dificultades de navegación, confusión en los procesos, incremento de errores y abandono de las plataformas digitales por parte de los usuarios. Por ello, la implementación de principios UX/UI se ha convertido en un elemento indispensable para mejorar la interacción humano-computadora y asegurar que las aplicaciones sean intuitivas, eficientes y accesibles.

El diseño UX/UI no se limita únicamente a la apariencia visual de un sistema, sino que abarca procesos de investigación, análisis y validación orientados a comprender cómo interactúan los usuarios con una aplicación. En este contexto, el UX se enfoca en aspectos como la arquitectura de la información, la facilidad de navegación, la accesibilidad, la eficiencia y la satisfacción general del usuario, mientras que el UI se concentra en componentes visuales tales como colores, tipografía, iconografía, distribución de elementos y consistencia gráfica. La correcta integración de ambos enfoques permite desarrollar interfaces coherentes que reduzcan la carga cognitiva y faciliten la ejecución de tareas dentro del sistema.

Otro aspecto relevante del diseño UX/UI es su estrecha relación con la accesibilidad digital. Las aplicaciones web modernas deben garantizar que personas con diferentes capacidades físicas, cognitivas o tecnológicas puedan acceder y utilizar sus funcionalidades sin restricciones. En este sentido, la aplicación de principios UX/UI contribuye significativamente al cumplimiento de estándares internacionales como las pautas Web Content Accessibility Guidelines 2.1, promoviendo características como la navegación clara, contraste adecuado de colores, legibilidad

del contenido, estructura semántica y compatibilidad con tecnologías de asistencia. Esto permite que la experiencia de usuario sea inclusiva y adaptable a distintos contextos y dispositivos.

Además, el diseño UX/UI influye directamente en la eficiencia operativa y productividad de las organizaciones. Un sistema diseñado con una adecuada experiencia de usuario reduce el tiempo necesario para completar tareas, minimiza errores de interacción y disminuye la necesidad de soporte técnico o capacitación prolongada. De igual manera, favorece la confianza y satisfacción de los usuarios al ofrecer procesos más claros y consistentes, incrementando así la aceptación y adopción del sistema implementado.

En aplicaciones web orientadas a la gestión de servicios, como el caso de la empresa "Hospital del Computador", el diseño UX/UI adquiere mayor importancia debido a la necesidad de gestionar información, solicitudes y procesos administrativos de forma rápida y organizada. La incorporación de interfaces intuitivas y accesibles facilita la interacción tanto del personal administrativo y técnico como de los clientes, optimizando la comunicación, la navegación y el acceso a la información dentro de la plataforma. Por esta razón, la aplicación de principios UX/UI no solo representa un componente visual, sino también una estrategia orientada a mejorar la funcionalidad, accesibilidad y experiencia general del sistema.

2.2.1 Principios Fundamentales de UX/UI

El diseño UX/UI se rige por una serie de principios fundamentales orientados a crear productos digitales que no solo sean visualmente atractivos, sino también intuitivos, eficientes y satisfactorios para el usuario (Quingue & Sánchez, 2023). La aplicación coherente de estos principios es crucial para el éxito de cualquier aplicación web. A continuación, se detallan algunos de los más relevantes:

- **Centrado en el usuario:** El diseño debe pivotar en torno a las necesidades, objetivos y comportamientos del usuario. La intención es crear soluciones que se adapten a sus expectativas y brinden una experiencia satisfactoria, lo que se logra mediante investigación continua y testing con usuarios reales.
- **Simplicidad:** Un diseño simple mejora la usabilidad al priorizar las funciones esenciales y eliminar elementos superfluos. Esto disminuye la carga cognitiva del usuario y hace que el producto sea más fácil de comprender y usar, facilitando la consecución de sus tareas.

- **Consistencia:** Garantiza que los elementos de la interfaz mantengan patrones similares en toda la aplicación web, incluyendo el uso uniforme de colores, tipografías, íconos, botones, menús y estructuras de navegación.
- **Feedback y respuesta:** El sistema debe proporcionar retroalimentación inmediata y clara sobre las acciones del usuario. Esto se logra mediante elementos visuales como microinteracciones, cambios de estado en los botones o mensajes de confirmación, lo que genera confianza y confirma que la interacción fue exitosa.
- **Jerarquía visual:** Organiza los elementos de la interfaz según el nivel de importancia, guiando la atención del usuario hacia la información más relevante.
- **Accesibilidad:** El diseño debe ser inclusivo, garantizando que usuarios con diversas capacidades y dispositivos puedan utilizar la aplicación sin barreras. Esto implica considerar un alto contraste de colores, tamaños de fuente legibles, navegación mediante teclado y compatibilidad con lectores de pantalla.
- **Eficiencia y facilidad de uso:** El diseño UX/UI eficiente permite que los usuarios completen tareas de manera rápida, intuitiva y con el menor esfuerzo posible.

La integración de estos principios permite desarrollar aplicaciones que son a la vez funcionales y placenteras de usar, sentando las bases para una experiencia de usuario positiva y una interfaz coherente (Quingue & Sánchez, 2023).

2.2.2 Elementos de Diseño de UX/UI

El diseño de Experiencia de Usuario (UX) e Interfaz de Usuario (UI) constituyen dimensiones complementarias en el desarrollo de aplicaciones web. El UX se centra en la interacción completa del usuario, buscando hacerla intuitiva, eficiente y satisfactoria mediante un proceso metódico que antepone necesidades y comportamientos del usuario en todas las etapas del desarrollo (Gonzales Rodriguez, 2021). Por su parte, el UI enfatiza los elementos visuales e interactivos con los que el usuario se comunica directamente, funcionando como la capa estética que da forma tangible a la experiencia digital (Gavila Martínez, 2024).

2.2.2.1 Características Principales

La **Tabla 2-5** presenta los principales aspectos que caracterizan a la UX y la UI dentro del proceso de diseño de interfaces digitales, destacando sus objetivos y contribución al desarrollo de soluciones funcionales y atractivas.

Tabla 2-5: Características Principales de los Aspectos de UX y UI

Aspectos de UX	Aspectos de UI
Investigación de usuarios: Base del proceso que incluye técnicas como entrevistas y creación de user personas para comprender motivaciones y frustraciones de usuarios finales.	**Tipografía legible y jerarquizada**: Facilita el escaneo visual y la comprensión del contenido.
Arquitectura de la información y mapas de experiencia: Organiza el contenido de manera lógica y visualiza los recorridos del usuario para completar tareas, identificando puntos de fricción.	**Paleta de colores coherente y estratégica**: Refuerza la identidad de marca y guía la atención del usuario.
Prototipado y testing: Creación de prototipos, desde esquemas básicos (wireframes) hasta interactivos, para validar flujos de diseño con usuarios reales antes del desarrollo.	**Disposición visual y uso del espacio (layout)**: Organiza elementos en pantalla de forma clara y equilibrada, siguiendo principios de proximidad y alineación

Fuentes: (Castañeda Ramírez, 2025; Gavila Martínez, 2024; Gonzales Rodriguez, 2021)

Elaborado por: Urbano K., 2026

2.2.2.2 Ventajas y Desventajas del Diseño UX/UI

El diseño UX/UI aporta múltiples beneficios en la creación de interfaces centradas en el usuario, mejorando la usabilidad, eficiencia y coherencia visual de los sistemas digitales. No obstante, su aplicación también implica ciertos desafíos relacionados con la inversión en recursos, el tiempo de desarrollo y la diversidad de percepciones de los usuarios.

La **Tabla 2-6** presenta las principales ventajas y desventajas asociadas a la implementación de estrategias de diseño UX/UI, considerando los aspectos funcionales, visuales y operativos que influyen en la experiencia general del usuario.

Tabla 2-6: Ventajas y Desventajas del Diseño UX/UI

Aspecto	Ventajas	Desventajas
Satisfacción del usuario	Mejora la experiencia completa con flujos intuitivos y primera impresión atractiva que genera confianza	Diferentes percepciones de usabilidad y preferencias estéticas entre usuarios
Eficiencia operativa	Reduce errores de uso, disminuye carga de servicio al cliente y facilita navegación coherente	Requiere inversión en investigación, testing y tiempo para refinamiento visual
Consistencia y marca	Establece patrones predecibles y refuerza identidad visual mediante elementos coherentes	Fases de descubrimiento y herramientas especializadas incrementan costos
Balance funcionalidad-estética	Combina usabilidad sólida con atractivo visual para experiencia integral	Riesgo de complejizar interfaces o priorizar apariencia sobre accesibilidad

Fuentes: (Castañeda Ramírez, 2025; Gavila Martínez, 2024; Tituaña et al., 2024)

Elaborado por: Urbano K., 2026

2.2.2.3 Relación del Diseño UX/UI con la Accesibilidad

El diseño UX/UI y la accesibilidad mantienen una relación estrecha dentro del desarrollo de aplicaciones web, debido a que ambos enfoques buscan mejorar la interacción entre el usuario y el sistema. Mientras el diseño UX/UI se enfoca en crear experiencias intuitivas, eficientes y satisfactorias, la accesibilidad procura facilitar el acceso y uso adecuado de la aplicación mediante interfaces claras, organizadas y fáciles de utilizar. La integración de ambos conceptos permite desarrollar plataformas más funcionales, comprensibles y adaptadas a distintos contextos de interacción.

La experiencia de usuario (UX) contribuye a la accesibilidad mediante la organización clara de la información, la simplificación de procesos y la optimización de la navegación dentro del sistema. Un diseño centrado en el usuario permite identificar necesidades y dificultades de interacción, favoreciendo la creación de interfaces comprensibles y eficientes para diferentes perfiles de usuarios. Por su parte, la interfaz de usuario (UI) aporta elementos visuales que facilitan la percepción y utilización del sistema, como el uso adecuado de colores, contraste, tipografía, iconografía y distribución de componentes en pantalla (Gavila Martínez, 2024).

La accesibilidad web se encuentra respaldada por estándares internacionales como las pautas Web Content Accessibility Guidelines 2.1, las cuales establecen criterios orientados a garantizar que los contenidos y funcionalidades web sean perceptibles, operables, comprensibles y robustos. En este contexto, muchos de los principios UX/UI contribuyen directamente al cumplimiento de estas pautas, ya que aspectos como la legibilidad, consistencia visual, jerarquía de información, navegación intuitiva y retroalimentación interactiva mejoran tanto la experiencia del usuario como la facilidad de uso de la plataforma.

Además, la integración de UX/UI con accesibilidad beneficia a los usuarios en general, debido a que una interfaz clara y organizada facilita la comprensión y reduce la complejidad durante la interacción. Esto permite disminuir errores, optimizar tiempos de uso y mejorar la satisfacción de los usuarios al utilizar la aplicación web. Por esta razón, el diseño UX/UI accesible se ha convertido en un elemento fundamental dentro del desarrollo de sistemas modernos orientados a ofrecer experiencias digitales más eficientes, organizadas y fáciles de utilizar (Quingue & Sánchez, 2023).

2.2.2.4 Contribución del Diseño UX/UI para mejorar aplicaciones web

El diseño UX/UI contribuye significativamente al mejoramiento de las aplicaciones web, debido a que permite desarrollar interfaces más intuitivas, funcionales y orientadas a las necesidades de los usuarios. La implementación de principios de experiencia de usuario (UX) e interfaz de usuario (UI) favorece una interacción más eficiente con el sistema, optimizando la navegación, comprensión de contenidos y realización de tareas dentro de la plataforma digital.

Desde la perspectiva del UX, el diseño centrado en el usuario permite analizar comportamientos, necesidades y dificultades de interacción para construir aplicaciones más fáciles de utilizar. Esto ayuda a reducir la complejidad de los procesos, disminuir errores y mejorar la satisfacción general durante el uso del sistema. Además, el UX facilita la organización lógica de la información y la creación de flujos de navegación claros, permitiendo que los usuarios completen sus actividades de manera más rápida y eficiente (Quingue & Sánchez, 2023).

Por otro lado, el diseño UI aporta elementos visuales que fortalecen la claridad y apariencia de la aplicación web. Aspectos como la tipografía legible, el uso adecuado de colores, la jerarquía visual, los iconos y la distribución organizada de los componentes permiten mejorar la comprensión e interacción del usuario con la interfaz. Una interfaz visualmente coherente no solo mejora la estética del sistema, sino que también facilita el aprendizaje y uso de la aplicación (Gavila Martínez, 2024).

Además, la integración de UX/UI contribuye al desarrollo de aplicaciones web accesibles e inclusivas, alineadas con estándares como las pautas Web Content Accessibility Guidelines 2.1. Elementos como la navegación intuitiva, contraste adecuado, consistencia visual y diseño adaptable favorecen el acceso de distintos tipos de usuarios, incluyendo personas con limitaciones visuales, cognitivas o tecnológicas. Esto permite mejorar la experiencia general y ampliar la accesibilidad de la plataforma digital.

En consecuencia, el diseño UX/UI se ha convertido en un componente fundamental dentro del desarrollo web moderno, debido a que no solo mejora la apariencia visual de las aplicaciones, sino también su funcionalidad, accesibilidad y eficiencia operativa. La correcta aplicación de estos principios favorece la creación de sistemas más organizados, comprensibles y centrados en el usuario, incrementando así la calidad y aceptación de las aplicaciones web.

2.3 Calidad de Software

La calidad del software se configura como un elemento crítico en el desarrollo de sistemas tecnológicos, determinándose mediante un proceso integral que abarca múltiples dimensiones del producto. Según Merchán et al. (2024), la calidad del software no se limita únicamente a la ausencia de defectos, sino que implica una evaluación holística que considera aspectos como la funcionalidad, confiabilidad, eficiencia, mantenibilidad y usabilidad. El proceso de garantía de calidad involucra una verificación sistemática que comprende desde la definición precisa de requerimientos, pasando por el diseño arquitectónico, hasta la implementación del código y la ejecución rigurosa de pruebas. Esta perspectiva multidimensional permite identificar y mitigar potenciales desviaciones, asegurando que el producto final cumpla con los estándares técnicos y las expectativas de los usuarios.

En el contexto de los sistemas web, la evaluación de la calidad del software puede centrarse en la accesibilidad del sistema, tomando como referencia los principios establecidos por las Web Content Accessibility Guidelines. En este sentido, se consideran aspectos como la facilidad de acceso al sistema, la claridad en la navegación, la adecuada organización de la información y la interacción con las funcionalidades, enfocados en usuarios en general. Estos elementos permiten verificar que el sistema pueda ser utilizado en distintos contextos de uso, garantizando el acceso y la comprensión de la información sin limitarse a condiciones específicas de los usuarios.

2.3.1 Accesibilidad en Aplicaciones Web

La accesibilidad web se refiere al diseño y desarrollo de sitios y aplicaciones web que permitan a los usuarios acceder, comprender y utilizar la información y funcionalidades de manera clara y sencilla Basurto Sarmiento y Zhizhpon Tacuri (2022). u objetivo es reducir barreras durante la interacción con los sistemas digitales mediante una adecuada organización del contenido, navegación comprensible y facilidad de uso. De esta manera, la accesibilidad también considera diferentes contextos de utilización, como el acceso desde distintos dispositivos, condiciones de iluminación o limitaciones temporales durante la interacción (Villa Puglla, 2025).

2.3.2 Estándar WCAG

Las Web Content Accessibility Guidelines constituyen el estándar internacional desarrollado por el World Wide Web Consortium (W3C) para orientar el diseño y desarrollo de contenido web accesible. Estas pautas establecen un conjunto de criterios técnicos que permiten estructurar de

manera adecuada la información, la navegación y la interacción dentro de los sistemas web (Initiative (WAI), s. f.).

A lo largo del tiempo, las WCAG han evolucionado en distintas versiones (1.0, 2.0, 2.1 y 2.2), incorporando mejoras que responden a las necesidades actuales de los entornos digitales y dispositivos. Su objetivo principal es proporcionar un marco de referencia robusto, aplicable y tecnológicamente neutral que facilite el desarrollo de aplicaciones web con una navegación clara, una organización coherente de la información y un acceso adecuado a sus funcionalidades.

2.3.3 Criterios de WCAG

Las Web Content Accessibility Guidelines se organizan en cuatro principios fundamentales, conocidos como POUR, los cuales constituyen la base para el diseño de aplicaciones web accesibles y estructuradas:

1. **Perceptible:** La información y los componentes de la interfaz deben presentarse de manera clara, permitiendo su correcta identificación y visualización. Esto implica una adecuada jerarquía de contenidos, uso de contrastes y organización visual de los elementos. (Urrego Gómez, 2025)
2. **Operable:** Los componentes de la interfaz y la navegación deben ser utilizables mediante diferentes formas de interacción, garantizando una navegación clara, estructurada y predecible que facilite el acceso a las funcionalidades del sistema.
3. **Comprensible:** La información y el funcionamiento del sistema deben ser claros y fáciles de entender, permitiendo a los usuarios interpretar correctamente el contenido y la lógica de interacción.
4. **Robusto:** El sistema debe funcionar de manera adecuada en distintos entornos, dispositivos y navegadores, asegurando la correcta visualización e interacción con sus funcionalidades (Alvite-Dí-ez & Rojas-Castro, 2022).

En relación con la medición de la accesibilidad, las WCAG establecen tres niveles de conformidad que permiten evaluar el grado de cumplimiento de los criterios definidos:

- **Nivel A:** Nivel básico que garantiza el acceso inicial al contenido.

- **Nivel AA:** Nivel intermedio que asegura una navegación más clara y una mejor organización de la información, siendo el más utilizado como referencia en aplicaciones web.
- **Nivel AAA:** Nivel avanzado que contempla condiciones más estrictas, generalmente aplicadas en contextos específicos.

Las métricas de cumplimiento se basan en la verificación del grado en que un sistema satisface los criterios establecidos para cada nivel. En este sentido, el nivel AA se considera un punto de referencia adecuado para evaluar la accesibilidad en aplicaciones web orientadas a usuarios en general, ya que permite garantizar una navegación clara, una adecuada organización de la información y una interacción comprensible dentro del sistema.

2.3.4 Instrumentos y Herramientas de Evaluación de la Accesibilidad

La evaluación de la accesibilidad web constituye un proceso fundamental para verificar el cumplimiento de las Web Content Accessibility Guidelines (WCAG), mediante el uso de instrumentos que permitan una valoración sistemática y objetiva de una aplicación web (Basurto Sarmiento y Zhizhpon Tacuri, 2022; Villa Puglla, 2025).

Las herramientas de evaluación automática corresponden a aplicaciones o servicios en línea que analizan el código fuente de un sistema para identificar problemas técnicos de accesibilidad de forma rápida. Estas herramientas permiten detectar aspectos como la estructura incorrecta de encabezados, el contraste insuficiente, el uso inadecuado de etiquetas y otros elementos relacionados con la correcta organización del contenido.

Entre las herramientas más utilizadas se encuentran WAVE, axe-core y Lighthouse, las cuales permiten evaluar criterios relacionados con la estructura visual, navegación y organización de la información dentro de una aplicación web. En este sentido, estas herramientas facilitan la generación de reportes que permiten identificar errores y establecer mejoras orientadas al cumplimiento de los estándares WCAG.

2.3.5 Importancia de la Accesibilidad

La implementación de la accesibilidad en aplicaciones web representa un aspecto fundamental en el desarrollo de sistemas, ya que contribuye a mejorar el acceso, la navegación y la comprensión de la información dentro de los entornos digitales. Su aplicación permite reducir barreras en la

interacción con el sistema, facilitando el uso adecuado de sus funcionalidades en distintos contextos.

Entre los principales beneficios de la accesibilidad se destacan:

- **Para los usuarios:** Permite un acceso más claro y organizado a la información, facilitando la navegación y la interacción con el sistema en diferentes situaciones de uso.
- **Estructura y navegación:** La aplicación de criterios de accesibilidad favorece una organización adecuada del contenido, una navegación consistente y una interacción predecible, lo que contribuye a una mejor comprensión del sistema (Jutta, 2020).
- **Disponibilidad del sistema:** La accesibilidad también implica que los sistemas puedan ser utilizados desde distintos dispositivos y entornos, favoreciendo el acceso a la información en cualquier momento y lugar.
- **Buenas prácticas de desarrollo:** Incorporar accesibilidad desde las etapas iniciales del desarrollo permite estructurar mejor el sistema, facilitando su mantenimiento, escalabilidad y adaptación a futuras necesidades tecnológicas (Basurto Sarmiento & Zhizhpon Tacuri, 2022).

2.4 Trabajos relacionados

El desarrollo de aplicaciones web orientadas a la gestión de servicios y automatización de procesos administrativos ha tomado gran relevancia en distintos entornos organizacionales, debido a la necesidad de optimizar tiempos de atención, centralizar información y mejorar la interacción entre usuarios y sistemas. Diversos estudios han abordado problemáticas relacionadas con el control de procesos técnicos, gestión de información y seguimiento de actividades mediante plataformas web desarrolladas con tecnologías modernas y metodologías ágiles.

Rodríguez García y Villa Arévalo (2022) desarrollaron una aplicación web orientada a optimizar la gestión de servicios técnicos mediante la automatización de procesos administrativos y operativos. La propuesta permitió mejorar el seguimiento de actividades, reducir tiempos de respuesta y centralizar la información relacionada con órdenes de servicio y atención al cliente. Asimismo, el estudio destacó la importancia de implementar sistemas web que faciliten la organización de procesos y mejoren la eficiencia operativa dentro de entornos técnicos.

Por su parte, Vergara Zurita y Poma Velastegui (2023) enfatizaron la relevancia de integrar principios de diseño UX/UI dentro del desarrollo de aplicaciones web, con el objetivo de mejorar la interacción entre el usuario y el sistema. Los autores señalaron que aspectos como la

navegación intuitiva, la organización visual de la información y la accesibilidad contribuyen significativamente a optimizar la experiencia del usuario y facilitar el uso de plataformas digitales en distintos contextos organizacionales.

De igual manera, diferentes investigaciones relacionadas con sistemas de gestión web coinciden en que la automatización de procesos, la centralización de información y la implementación de interfaces accesibles permiten mejorar la productividad, reducir errores operativos y optimizar el control de actividades dentro de las organizaciones. Además, el uso de tecnologías web modernas y metodologías ágiles como Scrum favorece el desarrollo de aplicaciones más flexibles, escalables y adaptadas a las necesidades de los usuarios.

Los antecedentes analizados permiten evidenciar la importancia de implementar aplicaciones web orientadas a la gestión de servicios, integrando principios de UX/UI y accesibilidad para mejorar la organización de procesos, la interacción con los usuarios y la eficiencia operativa. En este contexto, la presente investigación se enfoca en el desarrollo de una aplicación web para la gestión de servicios en la empresa “Hospital del Computador”, tomando como referencia los aportes tecnológicos y metodológicos identificados en los estudios revisados.

CAPÍTULO III

3. MARCO METODOLÓGICO

El presente capítulo describe el enfoque metodológico utilizado para el desarrollo de la aplicación web orientada a la gestión de servicios. Se detallan los métodos, técnicas y procedimientos empleados durante las etapas de análisis, diseño, implementación y validación del sistema.

3.1 Tipo de estudio

El presente trabajo de titulación corresponde a un proyecto técnico de carácter aplicativo, orientado al desarrollo de una aplicación web para la gestión de servicios en el ámbito informático. Este tipo de estudio se basa en la aplicación de los conocimientos teóricos y prácticos adquiridos durante la formación profesional, con el propósito de ofrecer una solución tecnológica a una problemática real relacionada con la administración y seguimiento de procesos de reparación de equipos.

El desarrollo se sustenta en el uso de los frameworks NestJS para el backend y Next.js para el frontend, lo que permite implementar una arquitectura moderna, escalable y eficiente. Además, se aplica un enfoque de diseño UX/UI con énfasis en la accesibilidad web, buscando garantizar una experiencia de usuario intuitiva, coherente y adaptable a diferentes perfiles de usuarios. Este enfoque integral contribuye al fortalecimiento de las competencias profesionales y a la optimización de los procesos de gestión mediante el uso de tecnologías web contemporáneas.

3.1.1 Métodos y Técnicas

En función de los objetivos específicos planteados, se establecen los métodos y técnicas que guían el desarrollo del proyecto, desde la recopilación y análisis de información hasta la validación del sistema implementado.

La **Tabla 3-1** presenta de forma detallada los métodos, técnicas y fuentes de información empleadas para cumplir cada uno de los objetivos del presente trabajo.

Tabla 3-1: Métodos, técnicas y fuentes.

OBJETIVOS	MÉTODOS	TÉCNICAS	FUENTE

Identificar el proceso de la gestión de servicios de la empresa Hospital del computador para conocer su funcionamiento.	Cualitativo	Entrevista	Personal del Hospital del Computador
Identificar las características de UX/UI para mejorar la accesibilidad de la aplicación web.	Analítico-Descriptivo	Revisión de documentación Revisión bibliográfica	Artículos académicos, Repositorios, Revistas
Desarrollar los módulos para la aplicación web utilizando las características presentadas en el diseño UX/UI.	Metodología ágil SCRUM	Prototipado Modelado de software Codificación Pruebas iterativas	Product Owner Herramientas de desarrollo Buenas prácticas de desarrollo web
Evaluar la accesibilidad de la aplicación web mediante el estándar Web Content Accessibility Guidelines.	Evaluativo	Evaluación automática de accesibilidad y análisis de reportes WCAG Verificación manual de navegación mediante teclado.	Estándar WCAG 2.1, reportes generados por Lighthouse y WAVE

Elaborado por: Urbano K., 2026

Gracias a los métodos y técnicas aplicados, fue posible comprender el proceso de gestión de servicios de la empresa "Hospital del Computador" y desarrollar una aplicación web alineada con las necesidades operativas de la organización. La aplicación de la metodología SCRUM permitió estructurar el desarrollo del sistema de manera iterativa e incremental, facilitando la implementación progresiva de funcionalidades y características UX/UI orientadas a mejorar la experiencia del usuario. Asimismo, la evaluación mediante las pautas Web Content Accessibility Guidelines (WCAG) y herramientas automáticas de validación permitió verificar criterios de accesibilidad relacionados con la navegación, estructura y organización de la interfaz web.

3.2 Identificar el proceso de gestión de servicios

Con la finalidad de comprender cómo se desarrolla actualmente el proceso de gestión de servicios dentro de la empresa Hospital del Computador, se realizó un análisis del procedimiento utilizado para el registro, seguimiento y control de las órdenes de servicio de equipos informáticos. Esta etapa permitió identificar el flujo de trabajo existente, las actividades realizadas por el personal técnico y administrativo, así como las principales limitaciones presentes durante la gestión de los procesos de reparación.

Actualmente, la empresa utiliza un sistema local alojado en un único computador con una licencia no oficial, lo que genera diversas restricciones operativas. Entre las principales limitaciones se

encuentra la imposibilidad de acceder simultáneamente al sistema desde diferentes estaciones de trabajo, ocasionando tiempos de espera durante el registro de órdenes, actualización de información y atención al cliente. Además, el seguimiento del estado de los equipos se realiza en gran parte mediante mensajes a través de WhatsApp, dificultando la consulta inmediata de información y afectando la eficiencia del proceso de atención.

Para el cumplimiento de este objetivo se aplicó el método analítico, el cual permitió descomponer el proceso de gestión de servicios en actividades más específicas, facilitando el estudio de cada una de las etapas involucradas. Mediante este análisis fue posible identificar necesidades funcionales, problemas operativos y oportunidades de mejora relacionadas con la administración de órdenes de trabajo, control de estados, seguimiento técnico y comunicación con los clientes.

Como técnica de recolección de información se empleó la entrevista semiestructurada dirigida al personal de la empresa, complementada con observación directa del proceso actual y revisión de la documentación utilizada durante la gestión de servicios. Estas técnicas permitieron obtener información detallada sobre el funcionamiento operativo de la empresa, los requerimientos del sistema y las dificultades existentes en el manejo de la información, sirviendo como base para el posterior diseño y desarrollo de la aplicación web.

3.2.1 Planificación

En esta etapa, el objetivo principal de la entrevista fue obtener información relevante sobre cómo se desarrolla actualmente el proceso de gestión de servicios dentro de la empresa Hospital del Computador. Para ello, se seleccionó al personal encargado de la recepción y administración de órdenes de trabajo, así como al área técnica, debido a que poseen un conocimiento directo del flujo operativo relacionado con el ingreso, diagnóstico, seguimiento y entrega de equipos informáticos.

Con la finalidad de abordar los aspectos más importantes del proceso, se elaboró una guía de entrevista basada en distintos ejes temáticos previamente definidos, permitiendo recopilar información relacionada con las actividades operativas, el manejo de órdenes de servicio, la comunicación con los clientes y las limitaciones del sistema actual.

La entrevista fue coordinada previamente con el personal de la empresa y realizada en las instalaciones de Hospital del Computador, garantizando la disponibilidad de los participantes y un ambiente adecuado para el desarrollo de la actividad. De igual manera, se informó a los

entrevistados que la información recopilada sería utilizada únicamente con fines académicos relacionados con el presente trabajo de titulación.

3.2.2 Diseño de entrevista

Para la recopilación de información se planteó una entrevista semiestructurada compuesta por preguntas abiertas, elaboradas con un lenguaje claro y comprensible para los participantes. Las preguntas fueron organizadas en diferentes ejes temáticos con el propósito de analizar de manera ordenada las actividades relacionadas con la gestión de servicios técnicos dentro de la empresa. Los temas abordados, como se evidencian en el **ANEXO A**, fueron los siguientes:

- Proceso de recepción e ingreso de equipos al sistema.
- Registro y actualización de órdenes de servicio.
- Seguimiento del estado de reparación de los equipos.
- Comunicación con los clientes sobre el avance del servicio.
- Gestión de actividades realizadas por los técnicos.
- Limitaciones del sistema actual utilizado por la empresa.
- Opinión sobre la implementación de una aplicación web para mejorar la gestión de servicios.

Esta estructura permitió obtener información organizada y directamente relacionada con los requerimientos funcionales y operativos necesarios para el desarrollo del sistema web.

3.2.3 Aplicación

La entrevista fue realizada al personal administrativo y técnico de la empresa Hospital del Computador en las instalaciones de la organización, en un ambiente adecuado que permitió el desarrollo normal de la actividad y la participación de los entrevistados de manera cómoda y fluida. Durante el desarrollo de la entrevista se registraron las respuestas mediante apuntes estructurados previamente elaborados, evitando interrupciones y permitiendo mantener una conversación continua sobre las actividades y problemáticas presentes en el proceso de gestión de servicios.

La sesión tuvo una duración aproximada de 40 minutos, tiempo en el cual se logró recopilar información relevante sobre el funcionamiento actual de la empresa, las dificultades operativas existentes y las necesidades relacionadas con la automatización de procesos.

3.2.4 Registro

La información obtenida durante la entrevista fue registrada manualmente mediante una hoja de apuntes estructurada en función de los ejes temáticos definidos previamente. Posteriormente, los datos recopilados fueron organizados y analizados con el propósito de identificar los principales procesos, limitaciones y requerimientos relacionados con la gestión de servicios técnicos.

La información recopilada sirvió como base para la definición de funcionalidades, módulos y características necesarias para el desarrollo de la aplicación web.

3.2.5 Resultados obtenidos

A partir de la información recopilada mediante la entrevista y observación del proceso actual, fue posible identificar las principales actividades involucradas en la gestión de servicios de la empresa Hospital del Computador. Entre los hallazgos más relevantes se identificó la dependencia de un sistema local con acceso limitado, la dificultad para realizar consultas simultáneas, la ausencia de seguimiento centralizado de órdenes y la utilización de WhatsApp como medio principal para informar el estado de los equipos a los clientes.

Adicionalmente, se logró establecer de manera clara el flujo operativo relacionado con el ingreso de equipos, asignación de técnicos, actualización de estados y entrega de los dispositivos reparados. Esta información permitió definir los requerimientos funcionales del sistema y sirvió como base para la elaboración de diagramas de procesos, como se muestra en la **Ilustración 2-1.**

3.2.6 Limitaciones encontradas

Durante el proceso de recopilación de información se identificó como principal limitación la falta de documentación formal relacionada con los procedimientos operativos de la empresa, debido a que gran parte de las actividades se desarrollan de manera empírica y basadas en la experiencia del personal. Además, el sistema actualmente utilizado presenta restricciones de acceso y consulta, dificultando la obtención organizada de información histórica sobre las órdenes de servicio y el seguimiento de los procesos realizados.

No obstante, mediante la aplicación de entrevistas, observación directa y revisión del flujo operativo actual, fue posible obtener la información necesaria para comprender el funcionamiento de la gestión de servicios y establecer los requerimientos fundamentales para el desarrollo de la aplicación web.

3.3 Identificar características UX/UI y accesibilidad

Con el propósito de mejorar la experiencia de usuario y garantizar un adecuado nivel de accesibilidad dentro de la aplicación web para la gestión de servicios, se realizó un análisis de diferentes características relacionadas con el diseño UX/UI y las pautas de accesibilidad web. Esta etapa permitió identificar principios, criterios y buenas prácticas orientadas a optimizar la interacción entre los usuarios y el sistema, tomando en consideración las necesidades operativas de clientes, técnicos y administradores de la empresa "Hospital del Computador".

El análisis realizado permitió establecer lineamientos relacionados con la organización visual de la interfaz, navegación intuitiva, consistencia de componentes, legibilidad del contenido y facilidad de uso en diferentes dispositivos. En este sentido, se consideraron criterios asociados a la accesibilidad web, con la finalidad de reducir barreras durante la interacción y mejorar la comprensión del sistema.

3.3.1 Análisis de principios UX/UI

Para el desarrollo de la investigación se analizaron diferentes principios de experiencia de usuario (UX) e interfaz de usuario (UI), orientados a mejorar la interacción de la aplicación web. Este análisis permitió identificar características relacionadas con la estructura visual, organización de contenidos, navegación, jerarquía de información y consistencia de los elementos gráficos del sistema.

Además, se consideraron aspectos enfocados en la optimización de procesos dentro de la aplicación, buscando que las diferentes funcionalidades sean fáciles de comprender y utilizar por los distintos perfiles de usuarios. De esta manera, se establecieron criterios para diseñar interfaces más intuitivas, organizadas y funcionales.

3.3.2 Análisis de accesibilidad web

Como parte del proceso metodológico, se realizó un análisis de las pautas Web Content Accessibility Guidelines (WCAG) 2.1, con el propósito de identificar criterios de accesibilidad aplicables al desarrollo de la aplicación web. Este análisis permitió reconocer aspectos relacionados con perceptibilidad, operabilidad, comprensibilidad y robustez de las interfaces digitales.

Entre los criterios considerados se encuentran el uso adecuado del contraste de colores, la incorporación de texto alternativo en imágenes, la asociación correcta de etiquetas en formularios, la navegación mediante teclado y la identificación adecuada de los elementos interactivos. Estos criterios sirvieron como referencia para orientar el desarrollo y la posterior evaluación de la accesibilidad de la aplicación web conforme a las WCAG 2.1.

3.3.3 Técnicas de recopilación de información

Para la identificación de características UX/UI y criterios de accesibilidad web, se utilizaron técnicas como la revisión bibliográfica y revisión de documentación técnica, permitiendo recopilar, analizar y organizar información relacionada con diseño de interfaces, experiencia de usuario y accesibilidad web aplicada al desarrollo de aplicaciones modernas.

3.3.3.1 Revisión bibliográfica

La revisión bibliográfica permitió recopilar información teórica relacionada con experiencia de usuario (UX), diseño de interfaces (UI) y accesibilidad web. Para ello, se consultaron artículos académicos, revistas científicas y trabajos de investigación publicados en plataformas digitales y repositorios académicos como Google Scholar, Scielo y ResearchGate.

La información obtenida facilitó el análisis de principios relacionados con navegación intuitiva, organización visual, jerarquía de contenidos, legibilidad y diseño responsive aplicados al desarrollo de aplicaciones web.

3.3.3.2 Revisión de documentación

La revisión de documentación se enfocó en el análisis de recursos técnicos y estándares relacionados con accesibilidad web y desarrollo frontend. Para ello, se consultó documentación oficial sobre las pautas WCAG 2.1, guías de accesibilidad web y documentación técnica de herramientas de validación como WAVE y Lighthouse.

Este proceso permitió identificar criterios técnicos y buenas prácticas orientadas a mejorar la accesibilidad, estructura y funcionamiento de la aplicación web durante las etapas de diseño e implementación del sistema.

3.3.4 Fuentes de información

Las fuentes empleadas durante la investigación estuvieron conformadas por artículos académicos, revistas científicas, repositorios digitales y documentación técnica especializada en UX/UI y accesibilidad web. Asimismo, se utilizaron recursos disponibles en plataformas académicas y documentación oficial relacionada con estándares WCAG 2.1 y herramientas de validación de accesibilidad.

La utilización de estas fuentes permitió recopilar información actualizada y confiable, contribuyendo a establecer una base teórica y técnica para la identificación de características UX/UI orientadas a mejorar la accesibilidad de la aplicación web.

3.4 Desarrollo de los módulos para la aplicación web utilizando características UX/UI

Para el desarrollo de la aplicación web orientada a la gestión de servicios de reparación y mantenimiento de equipos informáticos en la empresa "Hospital del Computador", se adoptó la metodología ágil Scrum, debido a que proporciona un marco de trabajo flexible, iterativo e incremental que facilita la organización, planificación y control de las actividades durante el desarrollo del software.

Scrum permite dividir el proyecto en ciclos de trabajo cortos denominados Sprints, dentro de los cuales se desarrollan funcionalidades específicas del sistema de manera progresiva. Esta metodología promueve la colaboración constante entre los integrantes del equipo, la adaptación continua frente a cambios en los requerimientos y la entrega incremental de valor mediante módulos funcionales que pueden ser evaluados y mejorados durante el proceso de desarrollo.

De acuerdo con Schwaber y Sutherland (2020), Scrum se fundamenta en principios de transparencia, inspección y adaptación, permitiendo detectar problemas de forma temprana y optimizar continuamente el desarrollo del producto. Asimismo, este enfoque facilita una gestión más dinámica del proyecto y una mejor comunicación entre los participantes involucrados.

Entre las principales características de Scrum se destacan:

- Desarrollo iterativo e incremental mediante Sprints.
- Organización y priorización de tareas a través del Product Backlog y Sprint Backlog.
- Seguimiento continuo del progreso mediante reuniones diarias.
- Retroalimentación constante y mejora continua del sistema.

- Adaptabilidad frente a cambios o nuevos requerimientos.

La aplicación de esta metodología dentro del proyecto permitió desarrollar los módulos del sistema de manera organizada, incorporando progresivamente las características UX/UI y los criterios de accesibilidad definidos previamente. Además, facilitó validar continuamente el funcionamiento de la aplicación web, optimizar la experiencia de usuario y realizar ajustes oportunos durante las diferentes etapas del desarrollo.

Scrum también permitió mantener un enfoque centrado en las necesidades reales de la empresa "Hospital del Computador", favoreciendo la construcción de una solución tecnológica funcional, escalable y alineada con los procesos de gestión de servicios que actualmente se realizan de manera manual o limitada por el sistema local existente.

A continuación, se describen las diferentes fases y actividades realizadas durante el desarrollo de la aplicación web, incluyendo el análisis, planificación, diseño, implementación, pruebas y validación del sistema.

3.4.1 Fase de análisis y planificación

En esta fase se realizaron las actividades iniciales necesarias para el desarrollo de la aplicación web orientada a la gestión de servicios de reparación y mantenimiento de equipos informáticos en la empresa "Hospital del Computador". Se analizaron los procesos actuales de trabajo, las necesidades de los usuarios y las limitaciones del sistema utilizado actualmente, permitiendo definir los requerimientos funcionales y no funcionales del sistema.

Como parte de esta fase, se identificaron los actores involucrados, los módulos principales que formarían parte de la aplicación web y las tecnologías necesarias para su desarrollo. Esta fase permitió establecer una base organizativa y técnica para la planificación de los Sprints y la construcción progresiva del sistema mediante la metodología Scrum.

3.4.1.1 Identificación de stakeholders

Se identificaron los principales actores involucrados dentro del proceso de gestión de servicios de la empresa "Hospital del Computador", con el propósito de comprender sus necesidades y definir las funcionalidades del sistema de acuerdo con cada perfil de usuario.

Entre los principales stakeholders se identificaron:

- Administrador del sistema.
- Técnicos encargados de las reparaciones.
- Clientes de la empresa.
- Personal administrativo.
- Equipo de desarrollo.

La identificación de estos actores permitió definir los diferentes roles del sistema, establecer permisos de acceso y comprender los procesos que cada usuario realiza dentro de la gestión de servicios.

3.4.1.2 Procesos a automatizar

Durante el análisis se identificaron los principales procesos que actualmente se realizan de manera manual o mediante un sistema local con limitaciones operativas.

Entre los procesos a automatizar se encuentran:

- Registro de órdenes de trabajo.
- Gestión de clientes y equipos.
- Seguimiento del estado de reparación.
- Asignación de técnicos.
- Registro de actividades técnicas.
- Gestión de servicios y repuestos.
- Consulta del estado de reparación por parte del cliente.
- Generación de reportes y control administrativo.

La automatización de estos procesos permitió plantear una solución web centralizada, accesible y orientada a optimizar el flujo de trabajo de la empresa.

3.4.1.3 Levantamiento de Requerimientos

A partir de la información recopilada mediante entrevistas y análisis del proceso actual, se definieron los requerimientos funcionales y no funcionales del sistema.

Entre los requerimientos funcionales identificados se destacan:

- Registro y autenticación de usuarios.
- Gestión de órdenes de trabajo.
- Consulta del estado de reparación.
- Registro de actividades técnicas.
- Gestión de servicios y repuestos.
- Administración de roles y permisos.

Entre los requerimientos no funcionales se establecieron aspectos relacionados con:

- Seguridad mediante autenticación JWT.
- Accesibilidad web.
- Diseño responsive.
- Rendimiento y tiempos de respuesta.
- Organización visual de la interfaz.

Estos requerimientos sirvieron como base para la construcción del *Product Backlog* y la planificación de los módulos del sistema. El catálogo completo y la especificación detallada de los requerimientos, **estructurados mediante Historias de Usuario y Criterios de Aceptación**, se encuentran documentados en la Sección 6 del Manual Técnico (Urbano, 2026, págs. 13-20).

3.4.1.4 Conformación del equipo y roles Scrum

Por medio de la **Tabla 3-2**, se presenta la participación de las personas y su rol, la cual se muestra a continuación:

Tabla 3-2: Equipo de desarrollo

Persona	Rol	Contacto
Ing. Jorge Menéndez	Scrum Master	1803689403 jmenendez@espoch.edu.ec
Kevin Urbano	Development Team	2300439680 kevin.urbano@espoch.edu.ec
Ing. Ernesto Quingue	Product Owner	

Elaborado por: Urbano K., 2026

La definición de estos roles permitió mantener una comunicación constante durante el desarrollo, facilitando la planificación de tareas, el seguimiento de avances y la validación continua de los módulos implementados.

3.4.1.5 Estudio de Factibilidad

Con la finalidad de determinar la viabilidad del desarrollo e implementación de la aplicación web para la gestión de servicios en la empresa "Hospital del Computador", se realizó un estudio de factibilidad enfocado en los aspectos técnicos, operativos y económicos del proyecto. Este análisis permitió verificar si la solución propuesta podía desarrollarse adecuadamente utilizando los recursos disponibles y si respondía a las necesidades reales de la empresa.

Factibilidad técnica

La factibilidad técnica del proyecto se consideró viable debido a que las tecnologías seleccionadas para el desarrollo de la aplicación web son compatibles entre sí y cuentan con amplia documentación y soporte dentro de la comunidad de desarrollo web.

Para el desarrollo del sistema se utilizaron tecnologías como NestJS para el backend, Next.js para el frontend, PostgreSQL como sistema gestor de base de datos y TypeORM como ORM para la gestión de entidades y relaciones. Asimismo, se implementó autenticación mediante JWT para garantizar seguridad en el acceso al sistema.

Además, el equipo de desarrollo cuenta con los conocimientos necesarios para la implementación y mantenimiento de dichas tecnologías, permitiendo desarrollar una solución web funcional, escalable y adaptable a las necesidades de la empresa "Hospital del Computador". Los requerimientos mínimos de hardware, configuraciones de software y estándares de codificación aplicados en el proyecto se describen de manera detallada en la Sección 3 del Manual Técnico (Urbano, 2026, págs. 2-4).

Las tecnologías empleadas fueron:

- Frontend: Next.js, Tailwind CSS y TypeScript.
- Backend: NestJS, TypeORM y JWT.
- Base de datos: PostgreSQL.
- Herramientas complementarias: GitHub, Postman y Visual Studio Code.

Factibilidad operativa

La factibilidad operativa se determinó a partir del análisis del entorno de trabajo y de los procesos que actualmente se realizan dentro de la empresa mediante un sistema local limitado y herramientas externas como WhatsApp.

La aplicación web propuesta se adapta al flujo de trabajo existente, permitiendo optimizar actividades relacionadas con el registro de órdenes de trabajo, seguimiento de reparaciones, gestión de servicios y consulta del estado de equipos por parte de los clientes.

Además, el personal administrativo y técnico posee conocimientos básicos en el uso de sistemas informáticos, lo que facilita la adopción de la plataforma sin requerir procesos complejos de capacitación. De igual manera, el sistema permitirá mejorar la organización de la información y reducir los tiempos de atención ocasionados por el uso del sistema local actual.

Factibilidad económica

La factibilidad económica del proyecto se consideró favorable debido a que el desarrollo de la aplicación web se realizó utilizando tecnologías de código abierto, evitando costos relacionados con licencias de software propietario. El desglose detallado de los costos de recursos técnicos, costos operativos mensuales y el presupuesto general consolidado del desarrollo se especifican en la Sección 4 del Manual Técnico (Urbano, 2026, págs. 5-7).

3.4.1.6 Análisis de Riesgos

Con la finalidad de identificar posibles situaciones que puedan afectar el desarrollo e implementación de la aplicación web para la gestión de servicios en la empresa “Hospital del Computador”, se realizó un análisis de riesgos tomando como referencia los lineamientos generales de la norma ISO 31000:2018. Este proceso permitió anticipar eventos que podrían influir negativamente en el desarrollo, funcionamiento y utilización del sistema, así como establecer medidas preventivas para minimizar su impacto.

El análisis de riesgos se organizó en cuatro etapas: diagnóstico del contexto, identificación de riesgos, evaluación y tratamiento del riesgo. Las matrices de evaluación de probabilidad e impacto (Nivel de Riesgo) y las correspondientes estrategias de mitigación técnica y operativa se encuentran documentadas en la Sección 5 del Manual Técnico (Urbano, 2026, págs. 7-11).

ETAPA I: Diagnóstico del contexto

En esta etapa se analizaron diferentes factores que podrían afectar el correcto desarrollo e implementación del sistema web dentro de la empresa. Entre los principales aspectos identificados se encuentran:

- Dependencia actual de un sistema local limitado.
- Transición de procesos manuales hacia un sistema web.
- Uso frecuente de WhatsApp para el seguimiento de órdenes.
- Posible resistencia al cambio por parte del personal.
- Diferentes niveles de conocimientos tecnológicos entre usuarios.

Este análisis permitió comprender el entorno operativo y tecnológico sobre el cual se desarrollaría la aplicación web.

ETAPA II: Identificación de riesgos

Posteriormente, se identificaron los principales riesgos relacionados con el ciclo de vida del proyecto y la fase operativa. Entre los más relevantes documentados en el manual se encontraron:

- Estimación inadecuada del tiempo del proyecto
- Priorización incorrecta y deficiente elicitación de requerimientos.
- Solicitudes constantes de cambios por parte del cliente.
- Fallas de conectividad a internet en el taller.

Cada uno de estos riesgos fue clasificado según su naturaleza (Proyecto, Técnico), permitiendo establecer medidas de prevención y control.

ETAPA III: Evaluación de riesgos

Una vez identificados los riesgos, se procedió a realizar su evaluación considerando el nivel de probabilidad e impacto que cada uno podría generar. Para ello, se empleó una clasificación cualitativa basada en niveles bajo, medio y alto, asignando valores numéricos para determinar el nivel de riesgo (NR) mediante la relación: Probabilidad × Impacto = Nivel de Riesgo (NR).

Los niveles utilizados fueron los siguientes:

Bajo: 1
Medio: 2
Alto: 3

Posteriormente, los resultados obtenidos fueron clasificados de la siguiente manera:

Bajo: 1 – 2

Medio: 3 – 4

Alto: 6 – 9

Los resultados permitieron identificar los riesgos que requerían mayor atención. Entre los más críticos evaluados se encontraron:

- **Estimación inadecuada del tiempo del proyecto (R1):** Probabilidad alta (3), impacto alta (3), NR = 9 → Muy Alto.
- **Solicitudes constantes de cambios por parte del cliente (R4):** Probabilidad alta (3), impacto alta (3), NR = 9 → Muy Alto.
- **Deficiente elicitación de requerimientos (R3):** Probabilidad alta (3), impacto media (2), NR = 6 → Alto.
- Priorización incorrecta de los requerimientos (R2): Probabilidad media (2), impacto alta (3), NR = 6 → Alto.

La evaluación realizada permitió priorizar los riesgos más críticos y establecer medidas preventivas orientadas a reducir posibles afectaciones.

ETAPA IV: Tratamiento del riesgo

Finalmente, se definieron estrategias orientadas a reducir o controlar los riesgos previamente identificados. Entre las principales medidas de mitigación aplicadas destacan:

- Realización de estimaciones ágiles utilizando Planning Poker y descomposición detallada de tareas por Sprint.
- Aplicación de un método formal de priorización (MoSCoW) y validación continua del Backlog.
- Refuerzo en las sesiones de elicitación mediante el diseño de prototipos visuales rápidos e iterativos.
- Establecimiento de políticas estrictas de control de cambios, congelando el alcance una vez iniciado el Sprint Backlog.
- Definición de criterios de aceptación en formato BDD (Dado/Cuando/Entonces) para asegurar la comprensión de los entregables.

El análisis de riesgos permitió identificar posibles escenarios que podrían afectar el desarrollo y funcionamiento de la aplicación web, facilitando la implementación de medidas preventivas para garantizar un sistema más seguro, estable y alineado con las necesidades de la empresa “Hospital del Computador”.

3.4.1.7 Elaboración del Product Backlog

El Product Backlog se construyó incluyendo tanto historias de usuario como historias técnicas, permitiendo organizar de manera integral las funcionalidades operativas del sistema y los requerimientos técnicos necesarios para su implementación.
Ejemplos:

- **HU-01:** "Inicio de Sesión Seguro" (Usuario). Valor: 5, Estimación: 24 horas (5 Pts).
- **HU-07:** "Apertura de ODS" (Recepcionista). Valor: 5, Estimación: 24 horas (5 Pts).
- **HT-01:** "Esquemas Roles/Permisos BD". Estimación: 18 horas (3 Pts).
- **HT-05:** "Gateway WebSockets NestJS". Estimación: 36 horas (8 Pts).

Este artefacto permitió organizar y priorizar las funcionalidades del sistema, facilitando la planificación progresiva de los módulos y el seguimiento de las actividades definidas para cada Sprint.

Durante esta fase se identificaron los actores involucrados dentro del proceso de gestión de servicios, los requerimientos funcionales y no funcionales del sistema, así como los principales procesos que serían automatizados dentro de la aplicación web. Además, se analizaron aspectos relacionados con la viabilidad del proyecto y los posibles riesgos técnicos y operativos que podrían afectar el desarrollo del sistema.

La información obtenida permitió establecer una estructura inicial para el desarrollo de la aplicación web y definir los elementos necesarios para las siguientes etapas de planificación, implementación y validación del sistema.

3.4.2 Fase de planeación y estimación

En esta fase se definieron y organizaron las historias de usuario e historias técnicas que conformaron el Product Backlog del sistema. Además, se realizó el análisis técnico y estructural de la aplicación web, permitiendo seleccionar las tecnologías, arquitectura y componentes necesarios para el desarrollo de los módulos del sistema.

De igual manera, se elaboró el modelado de datos, la estructura arquitectónica y el prototipado de interfaces, con el propósito de establecer una base sólida para la ejecución de los diferentes sprints planteados dentro de la metodología Scrum. Esta etapa permitió organizar de manera adecuada las funcionalidades prioritarias, definir tiempos estimados de desarrollo y establecer criterios de validación para cada historia. Las fichas detalladas de las historias de usuario bajo el formato BDD (Dado que, Cuando, Entonces) y el Product Backlog consolidado de desarrollo se especifican en la Sección 7 del Manual Técnico (Urbano, 2026, págs. 21-31).

3.4.2.1 Historias de Usuario

Se redactaron un total de 31 historias de usuario (HU-01 a HU-31), las cuales representan las funcionalidades que deben cumplir los diferentes perfiles del sistema: administrador, recepcionista, técnico y cliente. Estas historias fueron construidas a partir de los requerimientos funcionales identificados previamente y se formularon siguiendo el formato:

"Como [tipo de usuario], quiero [acción], para [objetivo o beneficio]".

Cada historia incluyó los siguientes elementos:

- Valor funcional de la historia (escala de 1 a 5).
- Nivel de prioridad (Crítica, Alta, Media o Baja).
- Estimación en puntos de historia y horas aproximadas de desarrollo.
- Criterios de validación claros y verificables.

Entre las historias más relevantes se encuentran:

- **HU-01:** Inicio de sesión seguro (Usuario) – Valor: 5, Estimación: 24 horas (5 puntos).
- **HU-07:** Apertura de órdenes de servicio (Recepcionista) – Valor: 5, Estimación: 24 horas (5 puntos).
- **HU-11:** Registro de peritaje técnico dinámico (Técnico) – Valor: 5, Estimación: 36 horas (8 puntos).
- **HU-18:** Consulta pública del estado de reparación (Cliente) – Valor: 5, Estimación: 24 horas (5 puntos).
- **HU-25:** Generación de presupuestos de reparación (Técnico) – Valor: 5, Estimación: 36 horas (8 puntos).

Estas historias fueron priorizadas con la participación del Product Owner, permitiendo enfocar el desarrollo en las funcionalidades más importantes para la automatización de los procesos de recepción, diagnóstico, reparación y seguimiento de equipos.

3.4.2.2 Historias Técnicas

Con respecto a la estructura interna, seguridad y soporte técnico del sistema, se plantearon 7 historias técnicas (HT-01 a HT-07), orientadas a garantizar el correcto funcionamiento de la aplicación web. Entre las historias técnicas más relevantes se encuentran:

- **HT-01:** Diseño de esquemas de roles y permisos en la base de datos – Estimación: 18 horas (3 puntos).
- **HT-02:** Implementación de guards y control de acceso en NestJS – Estimación: 24 horas (5 puntos).
- **HT-03:** Configuración del sistema de autenticación con NextAuth.js – Estimación: 24 horas (5 puntos).
- **HT-05:** Implementación del gateway de WebSockets en NestJS – Estimación: 36 horas (8 puntos).

Cada historia técnica fue evaluada considerando criterios de complejidad, integración, seguridad y viabilidad técnica, utilizando los mismos parámetros empleados para las historias de usuario.

3.4.2.3 Estimación de Historias

La estimación de las historias se realizó utilizando puntos de historia basados en la secuencia de Fibonacci (1, 2, 3, 5 y 8 puntos), permitiendo medir el nivel de complejidad, esfuerzo y tiempo requerido para cada funcionalidad.

Posteriormente, los puntos de historia fueron asociados a un tiempo aproximado de desarrollo y pruebas en horas. Para realizar esta estimación se consideraron aspectos como:

- Nivel de complejidad técnica de la funcionalidad.
- Integración con servicios externos como Brevo y SRI.
- Tiempo requerido para pruebas funcionales y validaciones.
- Experiencia previa del equipo con tecnologías como NestJS, TypeORM y Socket.io.

Las historias con mayor complejidad alcanzaron estimaciones de 8 puntos (aproximadamente 36 horas), principalmente aquellas relacionadas con funcionalidades dinámicas, tiempo real o procesos complejos de negocio, como el módulo de peritaje técnico o las notificaciones mediante WebSockets. Por otro lado, las historias de menor complejidad fueron estimadas entre 1 y 2 puntos (6 a 12 horas), enfocándose en funcionalidades CRUD básicas o configuraciones internas del sistema.

3.4.2.4 Definición de Criterios de Aceptación

Los criterios de aceptación fueron redactados utilizando un lenguaje claro y verificable, permitiendo validar el correcto funcionamiento de cada historia de usuario e historia técnica desarrollada.

Para facilitar el proceso de validación, se empleó el enfoque BDD (Behavior Driven Development), estructurando los criterios bajo el esquema:

- Dado
- Cuando
- Entonces

Por ejemplo, para la historia HU-07: "Apertura de órdenes de servicio", se establecieron los siguientes criterios:

- Validar los campos obligatorios correspondientes al cliente, equipo y problema reportado.
- Permitir autocompletar los datos del cliente mediante consulta al SRI utilizando cédula o RUC.
- Generar un número único y correlativo de orden de servicio.
- Mostrar un mensaje de confirmación al finalizar correctamente el registro del equipo.

Estos criterios permitieron verificar el cumplimiento funcional de cada módulo durante las fases de desarrollo y pruebas.

3.4.2.5 Análisis del Sistema

Durante esta etapa se analizaron los componentes técnicos, estructurales y arquitectónicos necesarios para el funcionamiento de la aplicación web.

a) Estilo arquitectónico

Se adoptó una arquitectura en capas, separando el sistema en tres niveles principales:

- Capa de presentación (Frontend): encargada de la interacción con el usuario mediante interfaces desarrolladas en Next.js.
- Capa de lógica de negocio (Backend): responsable de procesar reglas de negocio y servicios REST utilizando NestJS.
- Capa de acceso a datos: encargada de la comunicación con PostgreSQL mediante TypeORM.

Esta arquitectura fue seleccionada debido a su mantenibilidad, escalabilidad y facilidad para realizar pruebas e integraciones futuras.

b) Tecnologías seleccionadas

Para el desarrollo del sistema se seleccionaron tecnologías modernas y compatibles entre sí:

- Frontend: Next.js, Tailwind CSS y Framer Motion.
- Backend: NestJS con TypeORM.
- Comunicación: Axios y Socket.io.
- Base de datos: PostgreSQL.

Estas herramientas permitieron desarrollar una aplicación modular, escalable y adaptable a futuras mejoras.

c) Diagrama de componentes

Se elaboró un diagrama de componentes para representar la estructura general del sistema y la interacción entre sus módulos principales.

Dentro de este diagrama se representaron módulos como:

- Autenticación.
- Órdenes de servicio.

- Inventario.
- Presupuestos.
- Notificaciones.
- Gestión de usuarios y roles.

Además, se representó la comunicación entre frontend, backend y base de datos.

d) Análisis de estructura técnica

Se diseñó un diagrama de despliegue con el objetivo de representar la distribución física del sistema y sus componentes principales:

- Cliente web ejecutado desde el navegador.
- Servidor backend ejecutando NestJS sobre Node.js.
- Servidor PostgreSQL para persistencia de datos.
- Servicios externos como Brevo y SRI.

e) Modelos de base de datos

- **Modelo lógico**: Representando entidades, atributos y relaciones entre tablas.
- **Modelo físico**: Implementado en PostgreSQL mediante entidades TypeORM y restricciones de integridad relacional.

f) Diccionario de datos

Se elaboró un diccionario de datos como guía técnica para el desarrollo del sistema, documentando cada tabla y sus respectivos atributos.

Cada tabla incluyó información referente a:

- Nombre del campo.
- Tipo de dato.
- Descripción.
- Llave primaria o foránea.
- Restricciones y validaciones.

3.4.2.6 Maquetado de Interfaces

Se desarrollaron prototipos de baja fidelidad utilizando Balsamiq Wireframes, permitiendo definir la distribución de los componentes visuales y la navegación general del sistema antes de iniciar la implementación. Este proceso permitió:

- Visualizar la estructura de las interfaces.
- Validar la navegación entre módulos.
- Obtener retroalimentación temprana sobre la distribución visual.
- Incorporar criterios UX/UI y accesibilidad definidos previamente.

Las interfaces contemplaron pantallas para los distintos perfiles del sistema: administrador, recepcionista, técnico, cliente.

3.4.2.7 Selección de Herramientas de Desarrollo

Para facilitar el desarrollo colaborativo y el control del proyecto se seleccionaron diferentes herramientas de apoyo:

- **Visual Studio Code:** entorno de desarrollo principal.
- **Git y GitHub:** control de versiones del proyecto.
- **Postman**: pruebas y validación de endpoints REST.
- **npm**: gestión de dependencias y paquetes del sistema.

La utilización de estas herramientas permitió mantener un flujo de trabajo organizado, facilitando el desarrollo, pruebas y mantenimiento de la aplicación web.

3.4.3 Fase de implementación

La fase de implementación correspondió al proceso de construcción y desarrollo funcional de la aplicación web, en donde se ejecutaron las actividades definidas previamente dentro del Sprint Backlog. Durante esta etapa se desarrollaron progresivamente los diferentes módulos del sistema mediante iteraciones cortas organizadas en sprints, permitiendo incorporar funcionalidades de manera incremental y validarlas continuamente conforme avanzaba el proyecto.

Además, en esta fase se realizó la integración entre el frontend y backend, la configuración de la base de datos relacional, la implementación de mecanismos de autenticación y control de acceso, y la aplicación de buenas prácticas de programación orientadas a la mantenibilidad, seguridad y escalabilidad del sistema. Los detalles físicos de diseño como los diagramas UML de componentes y despliegue (Sección 8), principios de diseño visual y wireframes de baja fidelidad (Sección 9) y el diccionario de datos relacional (Sección 10) se encuentran documentados en el Manual Técnico (Urbano, 2026, págs. 32-69).

3.4.3.1 Implementación por Sprints

El desarrollo de la aplicación web se organizó en 8 sprints de dos semanas cada uno, distribuyendo las historias de usuario e historias técnicas según su prioridad y dependencia funcional. Esta planificación permitió desarrollar el sistema de manera progresiva, asegurando entregas funcionales al finalizar cada sprint.

Las principales actividades desarrolladas fueron las siguientes:

- **Sprint 1**: Diseño de la base de datos relacional y configuración inicial de entidades con TypeORM (HT-01), implementación de guards en NestJS (HT-02), desarrollo del inicio de sesión seguro (HU-01), protección de rutas REST (HU-04) y validación de permisos por roles (HU-06).

- **Sprint 2:** Configuración de NextAuth.js en el frontend (HT-03), implementación de seeders para carga inicial de datos (HT-07), registro e invitación de usuarios (HU-02), recuperación de contraseñas (HU-03) y asignación de múltiples roles (HU-05).

- **Sprint 3:** Desarrollo del módulo principal de órdenes de servicio, incluyendo apertura de ODS (HU-07), generación automática de códigos secuenciales (HU-08), asignación de niveles de prioridad (HU-09), registro de fallas reportadas (HU-10) y asignación de casilleros (HU-13).

- **Sprint 4:** Implementación del borrado lógico de registros (HT-04), desarrollo del módulo de peritaje técnico mediante checklist dinámico (HU-11), registro de actividades técnicas (HU-12) y carga de evidencias multimedia (HU-14).

- **Sprint 5:** Desarrollo del módulo de presupuestos (HU-25), aprobación interna de reparaciones (HU-26), aceptación de presupuestos por parte del cliente (HU-27), registro de auditoría de estados (HU-15) y consulta pública del estado de órdenes (HU-18).

- **Sprint 6:** Implementación del módulo de inventario y almacén, incluyendo gestión de repuestos (HU-19), control de partes físicas (HU-20), registro de compras (HU-21 y HU-22), anulación de compras (HU-23) y ajustes manuales de stock (HU-24).

- **Sprint 7:** Configuración del gateway WebSocket en NestJS (HT-05), integración de sockets en frontend (HT-06), desarrollo de vistas filtradas por roles (HU-17), notificaciones en tiempo real (HU-30) y alertas de stock mínimo (HU-31).

- **Sprint 8:** Desarrollo de dashboards estadísticos (HU-28), generación de reportes exportables (HU-29), desactivación lógica de órdenes obsoletas (HU-16), optimización responsive de interfaces y pruebas finales del sistema.

La organización del proyecto mediante sprints permitió mantener un flujo de trabajo continuo, facilitar el seguimiento de avances y realizar ajustes progresivos en función de la retroalimentación obtenida durante el desarrollo.

3.4.3.2 Control de Versiones con GitHub

Para la administración y control del código fuente se empleó Git junto con la plataforma GitHub. Esta herramienta permitió gestionar adecuadamente las diferentes versiones del sistema, mantener un historial de cambios y facilitar el trabajo colaborativo durante el desarrollo del proyecto.

Se trabajó con diferentes ramas de desarrollo, entre ellas:

- **main**: Contenía la versión estable y funcional del sistema.
- **Rama frontend**: Destinada al desarrollo de interfaces, componentes visuales y funcionalidades implementadas en Next.js.
- **Rama backend**: Utilizada para el desarrollo de la lógica de negocio, endpoints REST, autenticación y conexión con la base de datos en NestJS.

Las funcionalidades desarrolladas fueron integradas progresivamente mediante Pull Requests, permitiendo revisar cambios antes de incorporarlos a la rama principal del proyecto.

3.4.3.3 Integración de Frontend (Next.js) y Backend (NestJS)

La aplicación fue desarrollada bajo una arquitectura de tres capas (N-capas), distribuyendo las responsabilidades de presentación, lógica de negocio y persistencia de datos de manera independiente. El frontend fue construido utilizando Next.js, mientras que el backend) se implementó con NestJS, ejecutándose en un nodo de servidor de aplicaciones que expone una API REST organizada por módulos funcionales

Entre los módulos implementados se incluyeron:

- Autenticación y usuarios.
- Órdenes de servicio.
- Presupuestos.
- Inventario de repuestos.
- Notificaciones.
- Gestión de roles y permisos.

La comunicación entre ambas capas se realizó mediante solicitudes HTTP utilizando Axios como cliente de consumo de APIs. Además, se implementó autenticación basada en JSON Web Tokens (JWT), permitiendo proteger rutas y restringir funcionalidades según el rol del usuario autenticado.

Para fortalecer la seguridad del sistema se emplearon guards y decoradores de roles en NestJS, garantizando un control de acceso adecuado dentro de la aplicación.

3.4.3.4 Conexión con la Base de Datos (TypeORM y PostgreSQL)

La persistencia de la información se gestionó mediante PostgreSQL como sistema gestor de base de datos relacional y TypeORM como ORM encargado de mapear las entidades del sistema.

Las entidades fueron definidas mediante clases en TypeORM, permitiendo representar las tablas, relaciones y restricciones de la base de datos desde el backend. Para la administración de cambios estructurales se utilizaron migraciones, facilitando la sincronización controlada del esquema relacional.

Entre las actividades realizadas se incluyeron:

- Generación de migraciones para creación y modificación de tablas.
- Ejecución de migraciones en PostgreSQL.
- Inicialización de datos mediante seeders.
- Implementación de operaciones CRUD seguras y validadas.

De igual manera, se aplicaron relaciones entre entidades, restricciones de integridad y mecanismos de borrado lógico para garantizar la consistencia de la información almacenada.

3.4.3.5 Reuniones Diarias (Daily Scrum)

Durante el desarrollo de cada sprint se realizaron reuniones diarias de seguimiento con una duración aproximada de 10 a 15 minutos. Estas reuniones permitieron supervisar el avance del proyecto y coordinar las actividades pendientes. En cada reunión se abordaron aspectos como:

- Actividades completadas durante la jornada anterior.
- Tareas planificadas para el día.
- Problemas o bloqueos técnicos encontrados durante el desarrollo.
- Coordinación de actividades entre frontend y backend.

La aplicación de reuniones Daily Scrum facilitó la comunicación constante entre los integrantes del equipo y permitió detectar inconvenientes de manera temprana.

3.4.3.6 Documentación del Código Fuente y Buenas Prácticas

Durante el desarrollo del sistema se aplicaron buenas prácticas de programación orientadas a mejorar la organización, mantenibilidad y comprensión del código fuente. Entre las principales prácticas implementadas se encuentran:

- Uso de comentarios estructurados en servicios, controladores y métodos relevantes.
- Aplicación de convenciones de nomenclatura como camelCase y PascalCase.
- Organización modular del backend y frontend.
- Validación de datos mediante class-validator y formularios controlados.
- Encriptación de contraseñas utilizando bcrypt.
- Manejo centralizado de excepciones y respuestas HTTP.
- Separación de responsabilidades entre componentes, servicios y controladores.

Estas prácticas permitieron mantener una estructura de código clara, reutilizable y alineada con estándares modernos de desarrollo web.

3.4.4 Fase de revisión y retrospectiva

Al finalizar cada sprint se realizaron sesiones de revisión orientadas a verificar el correcto funcionamiento de las funcionalidades desarrolladas y validar el cumplimiento de los requerimientos definidos en el Product Backlog. Durante estas revisiones se presentaron los avances alcanzados en cada iteración, permitiendo obtener retroalimentación sobre el comportamiento del sistema, la experiencia de usuario y la integración entre módulos.

Además, estas actividades permitieron detectar oportunidades de mejora, corregir inconsistencias funcionales y ajustar determinados procesos antes de continuar con los siguientes ciclos de desarrollo.

3.4.4.1 Evaluación del Cumplimiento de Criterios de Aceptación

Para cada historia de usuario e historia técnica se verificó el cumplimiento de los criterios de aceptación definidos durante la fase de planificación. Estas validaciones permitieron asegurar que las funcionalidades implementadas respondieran correctamente a los requerimientos operativos del sistema.

Entre las validaciones realizadas se destacan las siguientes:

- En la historia **HU-07 "Apertura de ODS"**, se comprobó que el sistema permitiera registrar correctamente la orden de servicio, validando la información obligatoria del cliente, equipo y problema reportado. Además, se verificó la consulta automática de datos mediante el número de cédula o RUC.
- En la historia **HU-27 "Aceptación Cliente Online"**, se verificó que el cliente pudiera aprobar o rechazar el presupuesto desde la interfaz externa y que dicha acción actualizara automáticamente el estado de la orden en el sistema.
- En la historia **HU-30 "Notificaciones en Tiempo Real"**, se comprobó el envío de alertas automáticas relacionadas con cambios de estado, asignaciones y aprobación de presupuestos mediante WebSockets.

La validación continua de criterios de aceptación permitió mantener la calidad funcional del sistema y garantizar que cada incremento desarrollado cumpliera con los objetivos definidos para el proyecto.

3.4.4.2 Retrospectiva del Sprint

Después de cada revisión se realizaron reuniones de retrospectiva con el propósito de analizar el desempeño del equipo durante el sprint, identificar dificultades encontradas y proponer mejoras para las siguientes iteraciones.

Análisis de aspectos positivos

- La arquitectura modular implementada con NestJS y TypeORM facilitó el desarrollo independiente de funcionalidades en frontend y backend.
- La organización de tareas mediante sprints permitió mantener un seguimiento constante del avance del proyecto.
- El uso de componentes reutilizables en Next.js agilizó el desarrollo de interfaces y mejoró la consistencia visual del sistema.

Problemas identificados

- Surgieron retrasos menores relacionados con integraciones externas y validaciones de servicios de consulta.
- Algunas funcionalidades requirieron ajustes adicionales debido a cambios en requerimientos durante el desarrollo.
- Se identificaron mejoras necesarias en determinadas validaciones y procesos de actualización en tiempo real.

Acciones de mejora continua

- Reforzar las pruebas funcionales antes de finalizar cada sprint.
- Mantener reuniones de seguimiento para resolver bloqueos técnicos de forma temprana.
- Optimizar la planificación de historias que involucren integraciones externas o funcionalidades en tiempo real.
- La aplicación de retrospectivas permitió mejorar continuamente la organización del trabajo y optimizar el desarrollo progresivo de la aplicación web.

La aplicación de retrospectivas permitió mejorar continuamente la organización del trabajo y optimizar el desarrollo progresivo de la aplicación web.

3.4.5 Fase de lanzamiento

La fase de lanzamiento estuvo orientada a preparar el sistema para su ejecución en un entorno funcional, garantizando la correcta configuración de los servicios necesarios para el funcionamiento de la aplicación web. Durante esta etapa también se realizaron pruebas finales de funcionamiento y rendimiento con el fin de validar la estabilidad del sistema.

3.4.5.1 Preparación del entorno de ejecución

Para ejecutar la aplicación web correctamente en un entorno local se realizaron las siguientes actividades:

- Instalación de Node.js y PostgreSQL como dependencias principales del sistema.
- Configuración del entorno de desarrollo mediante Visual Studio Code.
- Clonación y configuración del repositorio del proyecto.
- Creación y configuración de archivos de variables de entorno para frontend y backend.
- Ejecución de migraciones y sincronización de la base de datos PostgreSQL.
- Inicialización de datos base mediante seeders.
- Levantamiento de los servicios backend y frontend utilizando los scripts de ejecución correspondientes.
- Acceso al sistema mediante la dirección local configurada para la aplicación web.
- Estas actividades permitieron disponer de un entorno estable para la ejecución y validación del sistema desarrollado.

3.4.5.2 Pruebas finales del sistema

Se realizaron pruebas finales sobre los diferentes módulos de la aplicación web con el propósito de verificar el correcto funcionamiento de las funcionalidades implementadas y validar el cumplimiento de los criterios definidos durante el desarrollo del sistema.

Las pruebas funcionales contemplaron procesos relacionados con autenticación de usuarios, registro y seguimiento de órdenes de servicio, generación de presupuestos, gestión de inventario, control de estados y consulta externa de información por parte de los clientes.

Adicionalmente, se efectuaron pruebas de accesibilidad web tomando como referencia las pautas establecidas por las Web Content Accessibility Guidelines (WCAG). Para ello, se utilizaron herramientas automáticas de evaluación como Lighthouse y WAVE, permitiendo identificar aspectos relacionados con:

- Contraste de colores en la interfaz.
- Estructuración correcta de encabezados y elementos HTML.
- Navegación y jerarquía de componentes.
- Etiquetado adecuado de formularios y botones.
- Compatibilidad básica con criterios de accesibilidad y buenas prácticas web.

De igual manera, se realizaron revisiones sobre la adaptabilidad visual de la interfaz en diferentes tamaños de pantalla y dispositivos, verificando la correcta visualización de los componentes desarrollados.

Los resultados obtenidos permitieron comprobar que la aplicación web presentaba un funcionamiento adecuado y cumplía con criterios básicos de accesibilidad y organización estructural definidos para el sistema.

3.4.5.3 Publicación del producto o despliegue final

Una vez finalizado el desarrollo y validación del sistema, se realizó el despliegue final de la aplicación web en un entorno local de ejecución. Para ello, se documentó el procedimiento necesario para la instalación, configuración y puesta en marcha del sistema desde equipos autorizados.

De igual manera, se efectuó la entrega oficial de la aplicación web desarrollada para la empresa "Hospital del Computador", incluyendo el código fuente y la documentación técnica correspondiente. La evidencia de este proceso se presenta en el **ANEXO B.**

3.5 Evaluación de la accesibilidad de la aplicación web

El cuarto objetivo del proyecto tuvo como finalidad evaluar la accesibilidad de la aplicación web desarrollada tomando como referencia las pautas establecidas en las **Web Content Accessibility Guidelines (WCAG 2.1)**, considerando el nivel de conformidad **AA**, con el propósito de verificar el cumplimiento de los criterios de accesibilidad aplicables a las interfaces del sistema. La

evaluación se enfocó en aspectos relacionados con los principios de **perceptibilidad, operabilidad, comprensibilidad y robustez**, definidos por dicho estándar.

Para este proceso se aplicó el método evaluativo, el cual permitió determinar el nivel de cumplimiento de los criterios de accesibilidad mediante la utilización de herramientas automáticas de evaluación, complementadas con pruebas manuales de navegación utilizando teclado. Los resultados obtenidos sirvieron para identificar barreras de accesibilidad, implementar las correcciones correspondientes y verificar las mejoras alcanzadas en la aplicación web.

3.5.1 Selección de interfaces para la evaluación

Debido a que la aplicación web desarrollada está conformada por diferentes módulos asociados a distintos perfiles de usuario, se seleccionaron las interfaces más representativas para realizar la evaluación de accesibilidad. La selección se efectuó considerando aquellas vistas que concentran las principales funcionalidades del sistema y que forman parte de los flujos de interacción más frecuentes durante su utilización.

Tabla 3-3 Interfaces seleccionadas para la evaluación de accesibilidad

Código	Interfaz evaluada	Perfil relacionado
IE-01	Inicio de sesión	General
IE-02	Dashboard administrativo	Administrador
IE-03	Registro de órdenes de servicio	Recepcionista
IE-04	Módulo de diagnóstico técnico	Técnico
IE-05	Consulta pública del estado de reparación	Cliente

Fuente: Aplicación web desarrollada.

Elaborado por: Urbano K., 2026

La evaluación de estas interfaces permitió verificar el cumplimiento de los criterios de accesibilidad definidos en las WCAG 2.1, nivel de conformidad AA, sobre las funcionalidades más representativas de la aplicación web, garantizando que el proceso de evaluación contemplara los principales escenarios de uso para cada perfil de usuario.

3.5.2 Evaluación automática de accesibilidad

Con el propósito de verificar el cumplimiento de los criterios de accesibilidad establecidos en las **Web Content Accessibility Guidelines (WCAG 2.1)**, se realizó una evaluación automática de las interfaces seleccionadas mediante herramientas especializadas de validación. Estas herramientas permitieron identificar barreras de accesibilidad, errores de implementación y

oportunidades de mejora relacionadas con la estructura y presentación de la información en la aplicación web.

Las herramientas utilizadas durante el proceso de evaluación fueron:

- **Lighthouse.**
- **WAVE Web Accessibility Evaluation Tool.**

A través de estas herramientas se analizaron, entre otros, los siguientes aspectos de accesibilidad:

- Contraste de colores.
- Jerarquía de encabezados.
- Uso de etiquetas semánticas y estructura HTML.
- Accesibilidad de formularios.
- Uso de texto alternativo en imágenes.
- Identificación adecuada de botones, enlaces y elementos interactivos.

De manera complementaria, se realizaron pruebas manuales de navegación utilizando únicamente el teclado, con el propósito de verificar el acceso a las principales funcionalidades de la aplicación, el orden lógico del foco y la correcta interacción con botones, enlaces, formularios y demás elementos interactivos.

3.5.3 Análisis de los reportes de evaluación de accesibilidad

Los reportes generados por las herramientas **Lighthouse** y **WAVE** fueron analizados con el propósito de determinar el nivel de cumplimiento de los criterios de accesibilidad considerados durante la evaluación de la aplicación web, tomando como referencia las pautas establecidas en las **WCAG 2.1**.

La información obtenida permitió identificar las principales barreras de accesibilidad presentes en las interfaces evaluadas, tales como problemas de contraste, ausencia de etiquetas descriptivas, deficiencias en la estructura semántica y otros aspectos relacionados con la accesibilidad de los componentes de la interfaz. Con base en estos hallazgos, se implementaron las correcciones correspondientes y se realizaron nuevas evaluaciones para verificar las mejoras alcanzadas.

Posteriormente, los resultados obtenidos fueron organizados en tablas y figuras para facilitar su análisis e interpretación en el capítulo de resultados, permitiendo evidenciar el impacto de las mejoras implementadas sobre el nivel de accesibilidad de la aplicación web.

3.5.4 Verificación manual de la navegación mediante teclado

Como complemento a la evaluación automática realizada mediante las herramientas Lighthouse y WAVE, se efectuó una verificación manual de la navegación utilizando únicamente el teclado. Esta actividad tuvo como propósito comprobar que las principales funcionalidades de la aplicación web pudieran ser utilizadas sin depender del uso del ratón, verificando el cumplimiento de los criterios de accesibilidad aplicables establecidos en las WCAG 2.1.

Durante la verificación manual se comprobó el comportamiento de los elementos interactivos presentes en las interfaces seleccionadas, evaluando aspectos relacionados con el orden de navegación, la visibilidad del foco y la posibilidad de acceder correctamente a los diferentes componentes de la aplicación.

Tabla 3-4 Aspectos verificados durante la navegación mediante teclado

Aspecto verificado	**Descripción**
Navegación mediante la tecla **Tab**	Se verificó que fuera posible recorrer los elementos interactivos utilizando únicamente el teclado.
Orden lógico del foco	Se comprobó que la secuencia de navegación siguiera un orden coherente entre los diferentes controles de la interfaz.
Visibilidad del foco	Se verificó que el elemento con foco pudiera identificarse visualmente durante la navegación.
Acceso a formularios	Se comprobó el acceso a campos de entrada, listas desplegables y botones mediante teclado.
Activación de elementos interactivos	Se verificó el funcionamiento de botones, enlaces y demás controles interactivos mediante teclado.

Fuente: Aplicación web desarrollada.
Elaborado por: Urbano K., 2026

3.5.5 Criterios de evaluación de accesibilidad

Para evaluar el nivel de accesibilidad de la aplicación web se establecieron indicadores de evaluación basados en los criterios definidos en las Web Content Accessibility Guidelines (WCAG 2.1), con énfasis en el nivel de conformidad AA. Estos indicadores permitieron verificar el grado de cumplimiento de los criterios de accesibilidad aplicables a las interfaces seleccionadas

mediante la utilización de las herramientas Lighthouse, WAVE y la verificación manual de navegación utilizando teclado.

La evaluación se estructuró considerando los cuatro principios fundamentales definidos por las WCAG 2.1: **Perceptible, Operable, Comprensible y Robusto**. Para cada principio se consideraron los criterios de conformidad más relevantes asociados a las características de la aplicación web y a los hallazgos identificados durante el proceso de evaluación.

Tabla 3-5 Criterios de accesibilidad evaluados e indicadores de verificación

Principio WCAG 2.1	Criterio de conformidad	Indicador de evaluación	Herramienta utilizada
Perceptible	1.1.1 Contenido no textual	Presencia de texto alternativo en imágenes	WAVE
Perceptible	1.4.3 Contraste mínimo	Relación de contraste adecuada entre texto y fondo	WAVE / Lighthouse
Operable	2.1.1 Teclado	Navegación mediante teclado sin uso del ratón	Verificación manual
Operable	2.4.3 Orden del foco	Secuencia lógica de navegación con tecla Tab	Verificación manual
Comprensible	3.3.1 Identificación de errores	Detección de errores en formularios	WAVE
Comprensible	3.3.2 Etiquetas o instrucciones	Asociación correcta entre labels y campos de formulario	WAVE
Robusto	4.1.2 Nombre, función y valor	Identificación adecuada de componentes interactivos mediante atributos de accesibilidad	WAVE

Fuente: WCAG 2.1, Lighthouse y WAVE

Elaborado por: Urbano K., 2026

Como complemento a los criterios anteriores, Lighthouse proporciona un puntaje general de accesibilidad comprendido entre 0 y 100 puntos, el cual fue utilizado como indicador cuantitativo para interpretar el nivel de accesibilidad alcanzado en cada una de las interfaces evaluadas. La interpretación adoptada para la presente investigación se presenta en la Tabla 3.5.

Tabla 3-6: Interpretación del puntaje de accesibilidad

Puntaje	Nivel de accesibilidad
0 – 49	Bajo
50 – 89	Medio
90 – 100	Alto

Fuente: Lighthouse Accessibility Score

Elaborado por: Urbano K., 2026

Por su parte, los reportes generados por WAVE permitieron identificar errores y advertencias relacionados con los criterios de accesibilidad evaluados, facilitando la detección de barreras presentes en las interfaces de la aplicación web. La información obtenida mediante ambas herramientas sirvió como base para el análisis e interpretación de los resultados presentados en el capítulo siguiente.

3.5.6 Criterios de conformidad WCAG

La evaluación de accesibilidad de la aplicación web se realizó tomando como referencia los niveles de conformidad establecidos por las Web Content Accessibility Guidelines (WCAG 2.1), los cuales permiten clasificar el grado de cumplimiento de los criterios de accesibilidad en tres niveles progresivos: A, AA y AAA.

Estos niveles representan distintos grados de exigencia en materia de accesibilidad, desde requisitos básicos hasta estándares más estrictos, permitiendo determinar el nivel de cumplimiento de una aplicación web respecto a los criterios establecidos por la WCAG.

Tabla 3-7: Niveles de conformidad WCAG 2.1

Nivel	Descripción
A	Nivel mínimo de accesibilidad. Incluye los requisitos básicos que eliminan barreras críticas de acceso al contenido.
AA	Nivel intermedio y recomendado. Considera criterios que mejoran significativamente la experiencia de usuario y es el estándar adoptado en la mayoría de normativas web.
AAA	Nivel avanzado de accesibilidad. Incluye criterios más estrictos que no siempre son aplicables a todos los tipos de contenido o contextos.

Fuente: WCAG 2.1
Elaborado por: Urbano K., 2026

La evaluación desarrollada en el presente proyecto se enfocó en los criterios de conformidad de nivel **AA**, considerando que este nivel constituye la referencia principal dentro de las WCAG 2.1 para la verificación del cumplimiento de accesibilidad en aplicaciones web.

3.5.7 Fuentes utilizadas

Para el proceso de evaluación de accesibilidad se utilizaron diversas fuentes normativas y herramientas técnicas que permitieron realizar el análisis de cumplimiento de los criterios establecidos en las Web Content Accessibility Guidelines (WCAG 2.1).

En primer lugar, se tomó como referencia el estándar WCAG 2.1, el cual constituye el marco normativo principal para la evaluación de accesibilidad en aplicaciones web.

Para la evaluación automatizada de las interfaces se utilizaron las siguientes herramientas:

- **Lighthouse (Chrome DevTools)**, empleada para la generación de auditorías automáticas de accesibilidad y la obtención de puntajes de cumplimiento.
- **WAVE Web Accessibility Evaluation Tool**, utilizada para la identificación de errores, advertencias y barreras de accesibilidad en la estructura de las interfaces.

De manera complementaria, se realizaron pruebas de verificación manual mediante navegación con teclado, con el objetivo de validar el comportamiento de los elementos interactivos en escenarios de uso real.

Finalmente, los reportes generados por Lighthouse y WAVE fueron utilizados como evidencia técnica para el análisis de resultados y la interpretación del nivel de cumplimiento de accesibilidad alcanzado por la aplicación web.

CAPÍTULO IV

4. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS

En el presente capítulo se exponen los resultados obtenidos durante el proceso de evaluación de accesibilidad realizado sobre la aplicación web desarrollada. Para ello se utilizaron herramientas automáticas de validación compatibles con el estándar WCAG 2.1, permitiendo identificar el nivel de accesibilidad alcanzado en las diferentes interfaces del sistema.

Los resultados obtenidos mediante Lighthouse y WAVE fueron organizados y analizados con el propósito de determinar el cumplimiento de buenas prácticas relacionadas con accesibilidad web, contraste visual, estructura semántica, navegación e interacción de los usuarios con la interfaz.

4.1 Resultados del proceso de evaluación de accesibilidad web

4.1.1 Interfaces evaluadas

Para la evaluación de accesibilidad se seleccionaron las principales interfaces del sistema, considerando los módulos más representativos y utilizados por los diferentes perfiles de usuario.

La selección de estas interfaces se realizó en función de los flujos de interacción más relevantes dentro del sistema, con el objetivo de evaluar el cumplimiento de los criterios de accesibilidad definidos en las Web Content Accessibility Guidelines (WCAG 2.1), nivel de conformidad AA, y las métricas establecidas en el capítulo metodológico.

Tabla 4-1: Interfaces evaluadas

Código	Interfaz evaluada	Perfil relacionado
IE-01	Inicio de sesión	General
IE-02	Dashboard administrativo	Administrador
IE-03	Registro de órdenes de servicio	Recepcionista
IE-04	Módulo de diagnóstico técnico	Técnico
IE-05	Consulta pública del estado de reparación	Cliente

Fuente: Aplicación web desarrollada.

Elaborado por: Urbano K., 2026

4.1.2 Resultados de la evaluación de accesibilidad web

La evaluación de accesibilidad de la aplicación web se realizó mediante un enfoque complementario de carácter iterativo, basado en los criterios establecidos en las Web Content Accessibility Guidelines (WCAG 2.1), nivel de conformidad AA, y las métricas definidas en el capítulo metodológico.

En una primera etapa, se utilizó la herramienta WAVE con el objetivo de identificar barreras de accesibilidad cualitativas presentes en las interfaces del sistema, considerando aspectos relacionados con los principios de perceptibilidad, operabilidad, comprensibilidad y robustez. Posteriormente, se implementaron las correcciones correspondientes en el código fuente de la aplicación, con base en los hallazgos obtenidos en la evaluación inicial. Estas mejoras estuvieron orientadas a reducir las barreras detectadas y asegurar el cumplimiento progresivo de los criterios de accesibilidad definidos en la WCAG 2.1.

Finalmente, se aplicó la herramienta Lighthouse con el propósito de obtener una medición cuantitativa del nivel de accesibilidad alcanzado en cada uno de los módulos evaluados, permitiendo validar el impacto de las mejoras implementadas.

A continuación, se detalla el flujo metodológico iterativo seguido para la detección y corrección de barreras de accesibilidad y la obtención de las métricas finales de conformidad:

Desarrollo inicial de los módulos de la aplicación web

↓

Evaluación diagnóstica inicial con la herramienta WAVE

↓

Identificación de errores específicos (etiquetas, contrastes, etc.)

↓

Corrección y depuración de barreras de accesibilidad en el código

↓

Reevaluación del sistema y pruebas manuales de navegación por teclado

↓

Medición cuantitativa final con la herramienta Lighthouse

↓

Obtención y registro de resultados de conformidad (89% - 98%)

4.1.3 Fase 1: Evaluación diagnóstica con WAVE

En esta primera fase se realizó un análisis diagnóstico de las interfaces del sistema mediante la herramienta WAVE, con el objetivo de identificar barreras de accesibilidad presentes en la implementación inicial de la aplicación web, en concordancia con los criterios establecidos en las WCAG 2.1.

El análisis permitió detectar diversas barreras de accesibilidad relacionadas con los principios de perceptibilidad, operabilidad, comprensibilidad y robustez, afectando distintos componentes de la interfaz del sistema.

Entre los principales hallazgos identificados se encuentran:

- **Contraste insuficiente de color (WCAG 1.4.3):** presente en el Dashboard administrativo debido al uso de textos con baja diferenciación cromática respecto al fondo.
- **Ausencia de texto alternativo (WCAG 1.1.1):** detectada en imágenes del módulo de diagnóstico técnico utilizadas para la visualización de evidencias.
- **Etiquetas de formulario faltantes (WCAG 3.3.2):** identificadas en el inicio de sesión y en campos de búsqueda, afectando la interpretación de los controles.
- **Jerarquía incorrecta de encabezados (WCAG 1.3.1):** observada en el Dashboard al omitir niveles intermedios en la estructura semántica.
- **Elementos interactivos sin descripción (WCAG 4.1.2):** presentes en botones con iconografía sin atributos de accesibilidad como aria-label.

A partir de estos hallazgos, se llevó a cabo un proceso de corrección en el código fuente de la aplicación, orientado a eliminar las barreras de accesibilidad identificadas. Este proceso incluyó la mejora del contraste visual en estilos CSS, la incorporación de texto alternativo en elementos gráficos, la vinculación adecuada de etiquetas en formularios, la reorganización de la estructura semántica del HTML y la adición de atributos de accesibilidad en componentes interactivos.

La siguiente ilustración muestra el reporte generado por la herramienta WAVE antes de la implementación de las correcciones, evidenciando las barreras de accesibilidad identificadas en la interfaz evaluada:

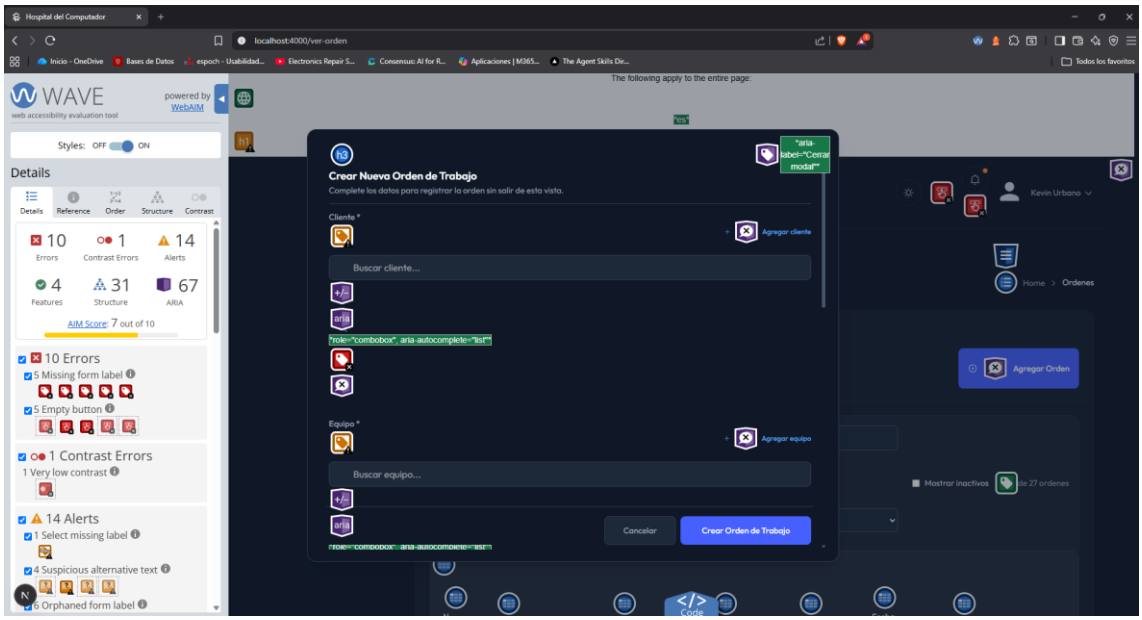


Ilustración 4-1: Reporte WAVE Página Orden de servicio

Elaborado por: Urbano K., 2026

Posteriormente, una vez implementadas las mejoras, se realizó una nueva validación del sistema, observándose una reducción significativa de errores de accesibilidad, lo que permitió continuar con la evaluación cuantitativa mediante Lighthouse.

4.1.4 Fase 2: Evaluación final con Lighthouse

Una vez implementadas las mejoras derivadas del diagnóstico inicial realizado mediante la herramienta WAVE, se procedió a ejecutar una evaluación final utilizando Lighthouse, con el propósito de obtener una medición cuantitativa del nivel de accesibilidad alcanzado en las interfaces del sistema, en relación con los criterios establecidos en las Web Content Accessibility Guidelines (WCAG 2.1), nivel de conformidad AA.

Esta evaluación permitió verificar el impacto de las correcciones aplicadas sobre los componentes de la interfaz, evidenciando una mejora significativa en los niveles de cumplimiento de accesibilidad.

Los resultados obtenidos reflejaron puntuaciones comprendidas entre 89/100 y 98/100 en los distintos módulos evaluados, lo que indica un alto grado de conformidad respecto a los criterios analizados por la herramienta.

Con el propósito de documentar el proceso de evaluación realizado, la herramienta Lighthouse permitió generar reportes detallados para cada una de las interfaces seleccionadas durante la

investigación. Estos reportes contienen el puntaje de accesibilidad obtenido, las auditorías ejecutadas automáticamente y el detalle de los criterios evaluados, constituyendo la evidencia técnica del proceso de validación desarrollado. En la presente sección se presenta un reporte representativo correspondiente a una de las interfaces evaluadas. Los reportes completos de las cinco interfaces analizadas se incluyen en los **Anexos B** al **F** como evidencia complementaria del proceso de evaluación de accesibilidad.

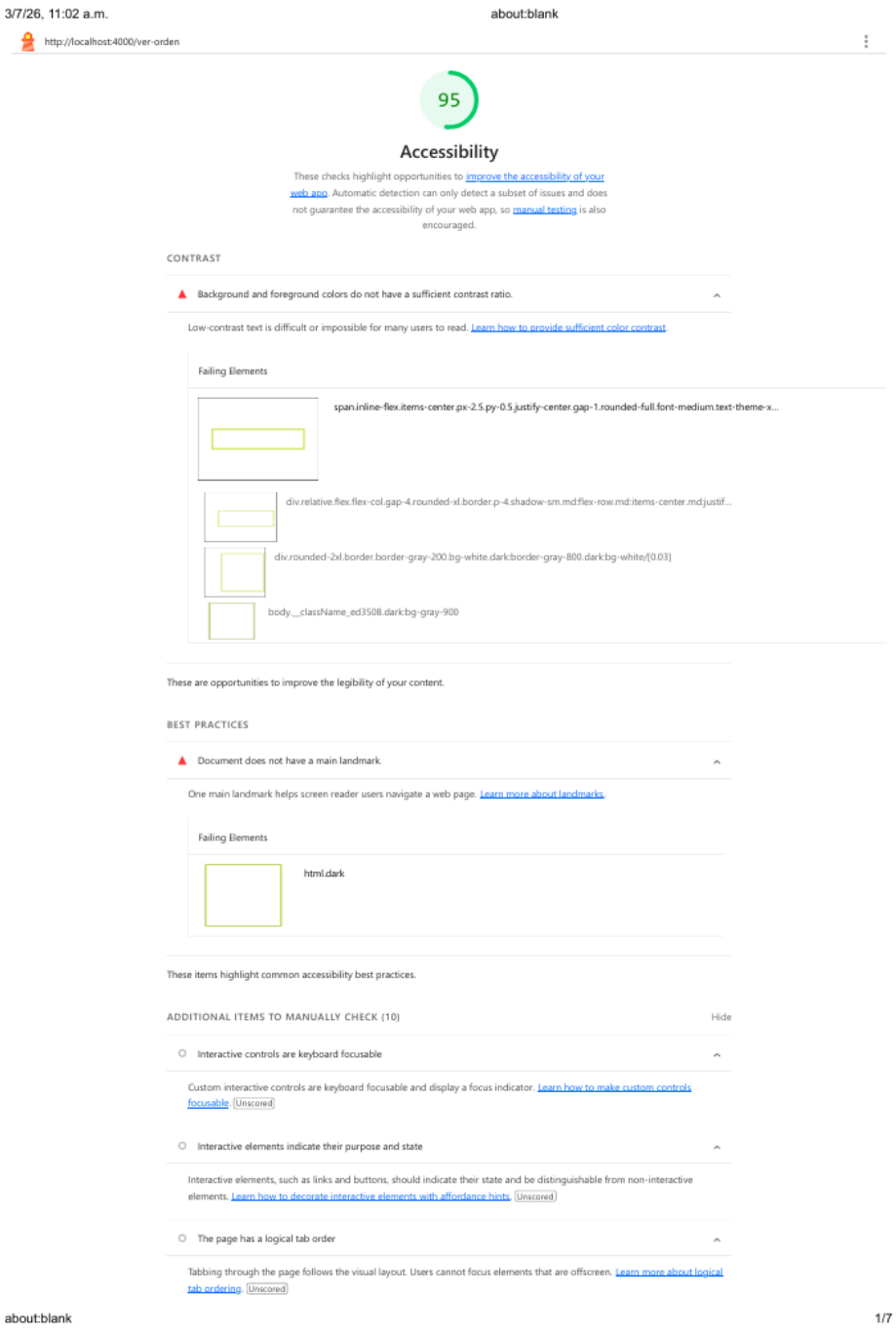


Ilustración 4-2: Reporte Lighthouse – Orden de servicio

Elaborado por: Urbano K., 2026

Para el análisis global de la evaluación se elaboró un consolidado de resultados, el cual resume los errores detectados inicialmente por WAVE, las acciones de mitigación implementadas y los puntajes finales obtenidos mediante Lighthouse.

Tabla 4-2: Consolidado de Resultados de Accesibilidad y Mitigación

Módulo	**Hallazgos identificados por WAVE**	**Acciones de mitigación aplicadas**	**Puntaje Lighthouse**
Inicio de sesión	3 hallazgos relacionados con contraste y etiquetas ausentes	Se incorporaron etiquetas explícitas en formularios y se ajustaron colores para cumplir contraste WCAG 1.4.3	89
Dashboard	5 hallazgos relacionados con jerarquía semántica y elementos interactivos	Se corrigió la estructura semántica del DOM y se añadieron atributos aria-label en botones iconográficos	94
Registro de orden de servicio	2 hallazgos relacionados con estructura de formularios y tablas	Se mejoró la estructura semántica de formularios y la presentación de datos en tablas	95
Módulo de diagnóstico técnico	6 hallazgos relacionados con imágenes sin descripción y contraste	Se incorporaron textos alternativos en imágenes y ajustes de contraste en elementos visuales	94
Portal del cliente	4 hallazgos relacionados con navegación y elementos interactivos	Se optimizó la navegación mediante teclado y se implementó un focus-ring visible conforme WCAG AA	98

Fuente: Reportes automatizados WAVE y Lighthouse

Reporte Lighthouse Final - Panel

Los resultados evidencian una mejora general en los niveles de accesibilidad de la aplicación web, lo cual confirma que las correcciones implementadas en el código fuente permitieron reducir las barreras identificadas en la fase diagnóstica.

Es importante señalar que los resultados obtenidos mediante Lighthouse corresponden exclusivamente a los criterios de accesibilidad evaluados automáticamente por la herramienta, por lo que fueron complementados con el análisis cualitativo realizado mediante WAVE y la verificación manual basada en las pautas WCAG 2.1.

4.1.5 Análisis de navegación y accesibilidad manual

Los resultados obtenidos durante la evaluación permitieron determinar que la aplicación web cumple con los criterios de accesibilidad establecidos en el estándar WCAG 2.1, nivel de conformidad AA, en relación con los aspectos evaluados mediante verificación manual y herramientas automáticas.

Las pruebas realizadas mediante navegación con teclado y revisión estructural de la interfaz evidenciaron el cumplimiento de los siguientes aspectos de accesibilidad:

- **Contraste y legibilidad visual:** cumplimiento de la relación mínima de contraste de 4.5:1 para texto normal, conforme al criterio WCAG 1.4.3.
- **Uso de etiquetas semánticas:** correcta estructuración del documento mediante elementos HTML5 como <header>, <nav>, <main>, <footer> y <section>, conforme al criterio WCAG 1.3.1.
- **Accesibilidad de formularios:** asociación adecuada entre etiquetas <label> y campos de entrada mediante el atributo correspondiente, conforme al criterio WCAG 3.3.2.
- **Navegación mediante teclado:** correcta secuencia de enfoque y acceso a elementos interactivos mediante teclado, conforme al criterio WCAG 2.1.1.

De manera complementaria, las observaciones identificadas en las fases de evaluación automatizada mediante WAVE y Lighthouse fueron corregidas progresivamente durante el proceso de desarrollo, lo que permitió reducir las barreras de accesibilidad inicialmente detectadas y mejorar el cumplimiento general de los criterios evaluados.

CONCLUSIONES

- Mediante el análisis de la información proporcionada por la empresa Hospital del Computador fue posible identificar y comprender el proceso de gestión de servicios técnicos, permitiendo determinar las necesidades operativas y las problemáticas existentes en el manejo de órdenes y seguimiento de equipos. Este análisis constituyó la base para plantear una solución tecnológica alineada con los procesos reales de la empresa, contribuyendo a una mejor organización y control de las actividades técnicas.

- A través de la investigación de lineamientos de accesibilidad y diseño de interfaces web, se identificaron características orientadas al cumplimiento de las pautas WCAG 2.1, considerando aspectos como el contraste visual, la navegación mediante teclado, la estructura semántica y la adaptabilidad responsiva. La incorporación de estos criterios permitió establecer lineamientos de desarrollo que favorecen la reducción de barreras de acceso y la mejora de la calidad de las interfaces del sistema.

- Se logró desarrollar una aplicación web multirol para la gestión de servicios técnicos utilizando Next.js, NestJS y PostgreSQL, automatizando procesos relacionados con órdenes de servicio, diagnósticos, presupuestos, inventario y seguimiento de reparaciones. La implementación del sistema contribuye a optimizar la administración de la información, reducir procesos manuales y facilitar el seguimiento de las actividades realizadas sobre los equipos ingresados a la empresa.

- Se evaluó el nivel de accesibilidad de la aplicación web con base en las pautas WCAG 2.1, nivel de conformidad AA, obteniéndose puntuaciones superiores a 89/100 en las evaluaciones realizadas mediante Lighthouse y minimizando las alertas semánticas detectadas por WAVE. Los resultados obtenidos evidencian un alto grado de cumplimiento de los criterios de accesibilidad considerados en la investigación, favoreciendo una interacción más accesible y una mejor compatibilidad con diferentes entornos de navegación.

- En términos generales, el desarrollo e implementación de la aplicación web permitió cumplir el objetivo principal de la investigación al proporcionar una solución tecnológica para la gestión de servicios del Hospital del Computador, integrando criterios de accesibilidad web desde las etapas de desarrollo y contribuyendo a la optimización de los procesos administrativos y técnicos de la empresa.

RECOMENDACIONES

- Se recomienda realizar un levantamiento detallado de información sobre los procesos operativos antes del desarrollo del sistema, permitiendo identificar correctamente las necesidades de gestión de servicios, los flujos de trabajo y las problemáticas presentes en los procesos manuales.

- Es aconsejable definir lineamientos de diseño UX/UI orientados a accesibilidad desde las etapas iniciales del proyecto, considerando aspectos como contraste visual, navegación mediante teclado, estructura semántica y adaptabilidad responsiva conforme a las pautas WCAG 2.1.

- Se recomienda utilizar arquitecturas modulares y tecnologías compatibles entre frontend y backend para facilitar la integración, mantenimiento y escalabilidad de aplicaciones web multirol orientadas a la gestión de servicios.

- Cuando se realicen evaluaciones de accesibilidad web, se recomienda complementar las herramientas automáticas como Lighthouse y WAVE con validaciones manuales de navegación e interacción, permitiendo obtener resultados más precisos sobre el cumplimiento de las pautas WCAG 2.1 y la experiencia de uso del sistema.

BIBLIOGRAFÍA

Adinata Arin*, I., Fitria Murad, D., & Harco Leslie Hendric Spits Warnars, H. S. W. (s. f.). *Digital Twin Integration in Field Service Management: A Hybrid Systematic Literature Review and Bibliometric Analysis (2017-2024) | IIETA*. https://doi.org/10.18280/ijsdp.200801

Alvite-Dí-ez, M.-L., & Rojas-Castro, A. (2022). Ediciones digitales académicas: Concepto, estándares de calidad y software de publicación. *Profesional de la información*, *31*(2). https://doi.org/10.3145/epi.2022.mar.16

Armijos Ortega, L. M., Velez Macas, C. A., Lojan Cueva, E. L., Armijos Ortega, L. M., Velez Macas, C. A., & Lojan Cueva, E. L. (2024). Estudio de la adopción de metodologías ágiles en proyectos de desarrollo de software en la región 7 del Ecuador. *Revista Espacios*, *45*(4), 73-84. https://doi.org/10.48082/espacios-a24v45n04p06

Basurto Sarmiento, D. B., & Zhizhpon Tacuri, C. E. (2022, julio). *Repositorio Institucional de la Universidad Politécnica Salesiana: Diseño, desarrollo e implementación de una herramienta web de análisis estadístico y monitoreo para evaluar la accesibilidad en páginas web según los criterios de conformidad de la WCAG 2.1*. https://dspace.ups.edu.ec/handle/123456789/23410?locale=es&utm_source=chatgpt.com

Berenguel Gómez, J. L. (2024). *Desarrollo de aplicaciones web distribuidas*. Ediciones Paraninfo, S.A.

Betancur Morales, K. (2022). *Comparativo metodológico y financiero entre las metodologías tradicionales y ágiles estipuladas para el diseño, formulación y gestión de proyectos de software*. https://repository.eia.edu.co/entities/publication/38c80246-d734-49c3-bc69-1ae5233cf1d4

Caldwell, B., Cooper, M., Guarino Reid, L., & Vanderheiden, G. (2008). *Web Content Accessibility Guidelines (WCAG) 2.0*. https://www.w3.org/TR/2008/REC-WCAG20-20081211/

Castañeda Ramírez, A. F. (2025). *Diseño UX/UI para la aplicación web Espora*. https://hdl.handle.net/10495/46666

Castillo Barreiro, L. C. (2025). *Aplicación web para la gestión de servicios y reservas en la peluquería 3 Ases.* https://repositorio.upse.edu.ec/handle/46000/14158

Espinoza Yunga, D. J. (2024). *Propuesta de un modelo de mesa de servicios utilizando buenas prácticas de TI basado en ITIL 4. Caso de estudio: Gobierno Autónomo Descentralizado Municipal del Cantón Pindal*. https://repositorio.puce.edu.ec/handle/123456789/45132

García Costa, N. (2025). *Desarrollo de una plataforma web full stack para la digitalización y gestión de un taller de Imaginería religiosa y arte sacro*. https://riunet.upv.es/handle/10251/223223

Gavila Martínez, J. (2024). *Diseño de la identidad corporativa, diseño de producto y diseño UX/UI de la página web para una marca tecnológica*. https://riunet.upv.es/handle/10251/212197

Gavilánez Álvarez, O. D., Layedra Larrea, N. P., & Ramos Valencia, M. V. (2022). Análisis comparativo de Patrones de Diseño de Software. *Polo del Conocimiento: Revista científico - profesional*, *7*(7 (JULIO 2022)), 2146-2165.

Gonzales Rodriguez, J. I. (2021). *Diseño web basado en UX/UI para mejorar la mala experiencia en el comercio electrónico en las empresas de retail de Lima Metropolitana*. https://repositorio.usil.edu.pe/entities/publication/b48b258d-3d21-4019-b2aa-55a4ed78d9e0

Initiative (WAI), W. W. A. (s. f.). *What's New in WCAG 2.2*. Web Accessibility Initiative (WAI). Recuperado 7 de octubre de 2025, de https://www.w3.org/WAI/standards-guidelines/wcag/new-in-22/

Jocham, R., Coleman, J., & Sutherland, J. (s. f.). *Scrum Guide Expansion Pack*.

Jutta, K. (2020, mayo 19). *Meeting Web Accessibility Requirements—A Case Study*. https://aaltodoc.aalto.fi/items/7e2ef822-cccb-4194-9a53-11e252dd7c36?utm_source=chatgpt.com

Krug, S. (2014). Don't make me think, revisited: A common sense approach to Web usability. *Choice Reviews Online*, *51*(11), 51-6218-51-6218. https://doi.org/10.5860/CHOICE.51-6218

L. F. Santiago González, Mendoza, J. C. H., Luna, Y. M., Rodríguez, A. J. R., Requena, D. T. V., Rodríguez, J. L. M., & Navejar, J. G. M. (2021). Aplicación Web Basada en el Patrón de Arquitectura de Software Modelo-Vista- Controlador (MVC) para Incrementar el Desempeño Académico en la Asignatura de Matemáticas Básicas: Web Application Based on the Model-View-Controller (MVC) Software Architecture Pattern to Increase Academic Performance in the Basic Mathematics Course. *Tecnología Educativa Revista CONAIC*, *8*(1), 7-21. https://doi.org/10.32671/terc.v8i1.187

Latorre Munar, S., Jiménez Ramírez, S. A., & Ibáñez Capacho, J. S. (2024). *Inteligencia Artificial Generativa para Arquitectura de Software*. https://hdl.handle.net/1992/75468

Mahiques Fenollar, M. (2024). *Desarrollo de una aplicación web full-stack para pruebas regresivas mediante una metodología de programación asistida por IA: Análisis comparativo con el desarrollo tradicional*. ix. https://doi.org/10.4995/ia.2014.3293

Marcapura Campos, A. M. (2023). *Implementación de un sistema de mesa de ayuda para gestión de incidencias de la empresa Resemin, 2022*. https://doi.org/10.21142/tl.2023.2842

Martínez Padua, J. V., Quitian Monroy, J. S., Castiblanco Jiménez, I. A., Martínez Padua, J. V., Quitian Monroy, J. S., & Castiblanco Jiménez, I. A. (2022). Caracterización y comparación de metodologías ágiles y tradicionales de desarrollo de producto. *Ciencia e Ingeniería Neogranadina*, *32*(2), 9-26. https://doi.org/10.18359/rcin.5168

Merchán-Narváez, N. J., Palma-Peralta, E. E., & Poma-Japón, D. X. (2024). Comparación de metodologías ágiles para el desarrollo de software. *MQRInvestigar*, *8*(1), 5052-5074. https://doi.org/10.56048/MQR20225.8.1.2024.5052-5074

Morales Carrillo, J., Cedeño Valarezo, L., Cajape Bravo, J. S., & Ormaza Calderón, J. G. (2021, julio 1). *Metodologías de desarrollo de software y su ámbito de aplicación: Una revisión sistemática*. https://www.proquest.com/openview/937c0ffcf92bf2c2761d76165f34090e/1?pq-origsite=gscholar&cbl=1006393

Moreano Anton, K. E. (2022). ITIL en la gestión de servicios en el área de tecnologías de información en la empresa SOLUTEAM S.A.C. Santa Anita – 2021. *Repositorio Institucional - UCV*. https://repositorio.ucv.edu.pe/handle/20.500.12692/93219

Morocho-Yunga, L. E., & Poma-Japón, D. X. (2025). Desarrollo de un sitio web y aplicación móvil para la gestión de servicios internacionales Soel&Tech. *MQRInvestigar*, *9*(1), e251-e251. https://doi.org/10.56048/MQR20225.9.1.2025.e251

NestJS. (s. f.). NestJS - A Progressive Node.Js Framework. Recuperado 13 de octubre de 2025, de https://nestjs.com

Next.js. (s. f.). Recuperado 13 de octubre de 2025, de https://nextjs.org/

Palomeque-Zambrano, E. E., & Campoverde-Molina, M. (2023). Propuesta de una arquitectura de software para el desarrollo de sitios web accesibles. *MQRInvestigar*, *7*(3), 1458-1474. https://doi.org/10.56048/MQR20225.7.3.2023.1458-1474

Pérez Ibarra, S. G., Quispe, J. R., Mullicundo, F. F., & Lamas, D. A. (2021). *Herramientas y tecnologías para el desarrollo web desde el FrontEnd al BackEnd*. XXIII Workshop de Investigadores en Ciencias de la Computación (WICC 2021, Chilecito, La Rioja). http://sedici.unlp.edu.ar/handle/10915/120476

Piñeiro Gomez, Jose Manuel. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S.A.

Quingue, E. I., & Sánchez, K. A. (2023). *REDISEÑO E IMPLEMENTACIÓN DEL SISTEMA WEB DEL GAD DEL CANTÓN GUAMOTE, COMO MEDIO DE INFORMACIÓN TURÍSTICA, COMUNICACIÓN DE OBRAS Y ACTIVIDADES REALIZADAS USANDO DISEÑO UX/UI*.

Riano Nossa, N. D. (2021). *Estudio comparativo de metodologías tradicionales y ágiles aplicadas en la gestión de proyectos*. https://repository.upb.edu.co/handle/20.500.11912/9611

Rojas Torres, J. M. (2023). HERRAMIENTA CASE GXC PARA GENERAR CÓDIGO BAJO LA ARQUITECTURA DE PROGRAMACIÓN EN 3 CAPAS. *Repositorio institucional - ULP*. https://repositorio.ulp.edu.pe/handle/ULP/48

Salgado Reyes, N. E., Guamba Gómez, A., & Guerrero Flores, R. (2024). El impacto de la tecnología de la información en la gestión empresarial. *Nexus Research Journal*, *3*(2), 17-34.

Soriano Sanchis, S. (2025). *Desarrollo de una herramienta para la creación dinámica de interfaces CRUD con Node.js y React*. https://hdl.handle.net/10609/152069

Sotomayor, C., & Alonso, I. J. A. (2023). Desarrollo de una aplicación web utilizando Nextjs y metodologías agiles para mejorar los procesos administrativos internos y el manejo de finanzas de restaurantes del distrito de Villa el Salvador. *Revista Científica: BIOTECH AND ENGINEERING*, *3*(1). https://doi.org/10.52248/eb.Vol3Iss1.87

Tedja, B., Al Musadieq, M., Kusumawati, A., & Yulianto, E. (2024). Systematic literature review using PRISMA: Exploring the influence of service quality and perceived value on satisfaction and intention to continue relationship. *Future Business Journal*, *10*(1), 39. https://doi.org/10.1186/s43093-024-00326-4

Tituaña, J., Ojeda Carrasco, J. L., Girón Rodríguez, J. P., Bonilla Granja, D. S., & Jara Moya, S. (2024). Análisis de Frameworks Frontend para Aplicar UX/IU en el Desarrollo Web: Una Revisión Sistemática. *Ciencia Latina: Revista Multidisciplinar*, *8*(3), 841-862.

Toro, M. A. R., Herrera, J. H., Salinas, L. S. V., & Chávez, J. M. M. (2023). Construyendo una web API con la arquitectura n capas. *Multidisciplinas de la Ingeniería*, *11*(18), 14-25. https://doi.org/10.29105/mdi.v11i18.274

Urrego Gómez, Y. S. (2025). *Implementación del principio de perceptibilidad del estándar WCAG 2.1 en la plataforma web del proyecto Inclusión Lectora de la carrera de Computación* [Universidad Nacional de Loja. Facultad de la Energía, las Industrias y los Recursos Naturales no Renovables. Carrera de Computación.]. https://dspace.unl.edu.ec/handle/123456789/33654

Villa Puglla, F. N. (2025). *Evaluación de accesibilidad web de universidades de Ecuador en el top del QS ranking* [PUCE - Ambato]. https://repositorio.puce.edu.ec/handle/123456789/46620

Web Content Accessibility Guidelines (WCAG) 2.1. (s. f.). Recuperado 10 de noviembre de 2025, de https://www.w3.org/TR/WCAG21/

WebAIM web accessibility in mind. (2025, marzo 31). *The WebAIM Million*. The WebAIM Million The 2025 report on the accessibility of the top 1,000,000 home pages. https://webaim.org/projects/million/

Widianto, A., & Subriadi, A. P. (2022). IT service management evaluation method based on content, context, and process approach: A literature review. *Procedia Computer Science, Sixth Information Systems International Conference (ISICO 2021)*, *197*, 410-419. https://doi.org/10.1016/j.procs.2021.12.157

Zangari, A., Marcuzzo, M., Schiavinato, M., Gasparetto, A., & Albarelli, A. (2023). Ticket automation: An insight into current research with applications to multi-level classification scenarios. *Expert Systems with Applications*, *225*, 119984. https://doi.org/10.1016/j.eswa.2023.119984

ANEXOS

ANEXO A: ENTREVISTA PARA IDENTIFICAR EL PROCESO DE GESTIÓN DE SERVICIOS EN HOSPITAL DEL COMPUTADOR

Fecha de la entrevista:

Sábado 21 de junio de 2025

Lugar:
Instalaciones de Hospital del Computador

Entrevistado:
Ing. Ernesto Quingue - Personal administrativo y técnico con conocimiento directo del proceso de gestión de servicios y reparación de equipos informáticos.

Preguntas y respuestas

1. **¿Cómo se realiza actualmente el ingreso de un equipo al taller?**

   El cliente entrega el equipo en recepción y se registran manualmente los datos del cliente, características del equipo y problema reportado en el sistema local.

2. **¿Qué información se registra cuando ingresa un equipo?**

   Se registran datos como nombre del cliente, número telefónico, tipo de equipo, marca, modelo, número de serie, falla reportada y accesorios entregados junto al equipo.

3. **¿Cómo se asigna una orden de trabajo al equipo ingresado?**

   El sistema genera una orden de trabajo y posteriormente el equipo es asignado a un técnico dependiendo del tipo de reparación o disponibilidad.

4. **¿Cómo se realiza el seguimiento del estado de reparación?**

   El técnico actualiza verbalmente o mediante anotaciones el avance del trabajo y la información se modifica manualmente en el sistema local.

5. **¿Qué dificultades presenta el sistema actual utilizado por la empresa?**

   El sistema solamente puede utilizarse desde un computador, no permite acceso simultáneo y genera retrasos cuando varias personas necesitan utilizarlo al mismo tiempo.

6. **¿Cómo consultan los clientes el estado de sus equipos?**

Generalmente los clientes realizan consultas mediante llamadas telefónicas o mensajes por WhatsApp.

7. **¿Qué problemas genera el seguimiento mediante WhatsApp?**

En ocasiones la información tarda en responderse, se pierden mensajes o no se dispone inmediatamente del estado actualizado del equipo.

8. **¿Cómo se registran las actividades realizadas por los técnicos?**

Las actividades se registran manualmente dentro del sistema o mediante observaciones escritas relacionadas con el proceso de reparación.

9. **¿Cómo se controla el uso de repuestos durante una reparación?**

El técnico informa los repuestos utilizados y estos son registrados manualmente para el control del inventario y costos.

10. **¿Cómo se calcula el costo total del servicio?**

Se consideran los costos de mano de obra y repuestos utilizados durante la reparación del equipo.

11. **¿Cómo se informa al cliente sobre presupuestos o cambios en la reparación?**

La comunicación se realiza principalmente por llamadas telefónicas o mensajes de WhatsApp.

12. **¿Qué sucede cuando varios empleados necesitan acceder al sistema al mismo tiempo?**

Deben esperar turnos para utilizar el computador donde está instalado el sistema, generando retrasos en la atención.

13. **¿Cómo se realiza la entrega del equipo al cliente?**

Una vez finalizada la reparación, se informa al cliente y se procede a entregar el equipo verificando la orden de trabajo correspondiente.

14. **¿Cómo se almacenan los registros históricos de reparaciones?**

Los registros quedan almacenados en el sistema local, aunque el acceso y consulta pueden resultar limitados.

15. ¿Qué dificultades existen al buscar información de órdenes antiguas?

En algunos casos la búsqueda puede ser lenta o complicada debido a la organización limitada de la información.

16. ¿Cómo se controla el estado de los equipos dentro del taller?

Se utilizan estados definidos manualmente según el avance del proceso de reparación.

17. ¿Qué tipo de reportes consideran necesarios para mejorar la gestión?

Reportes relacionados con órdenes ingresadas, equipos entregados, reparaciones pendientes, ingresos económicos y seguimiento técnico.

18. ¿Cuáles son los principales problemas del proceso actual de gestión de servicios?

Los principales problemas son la demora en consultas, limitaciones del sistema local, dificultad de acceso simultáneo y falta de seguimiento automatizado para clientes.

19. ¿Qué beneficios esperan obtener con una aplicación web?

Se espera mejorar la organización, agilizar procesos, centralizar información y permitir consultas en tiempo real sobre el estado de los equipos.

20. ¿Considera importante que los clientes puedan consultar el estado de sus equipos desde internet?

Sí, porque permitiría mejorar la atención al cliente y reducir consultas repetitivas por llamadas o mensajes.

21. ¿Cómo considera que una aplicación web podría mejorar el trabajo del personal?

Facilitaría el acceso simultáneo al sistema, optimizaría el registro de información y mejoraría el control de las órdenes de servicio y reparaciones.

ANEXO B: ACTA DE ENTREGA AL CLIENTE

ACTA DE ENTREGA DE PRODUCTO SOFTWARE

En la ciudad de RIOBAMBA, a los 6 días del mes de JULIO de 2026, se reúnen las siguientes personas:

1. Por la Empresa "Hospital del Computador":

- **Nombre:** Ernesto Israel Quingue Marcatoma
- **Cargo:** Gerente Propietario del Hospital del Computador (Product Owner)
- **Documento de Identidad:** 0605376300

2. Por el Desarrollador del Sistema:

- **Nombre:** Kevin Alexander Urbano Lombeida
- **Cargo / Rol:** Desarrollador
- **Documento de Identidad:** 2300439680

I. OBJETO DEL ACTA

Entregar formalmente a la empresa "Hospital del Computador" el producto software denominado:

- **Nombre del Software:** Aplicación Web para la Gestión de Servicios
- **Versión:** 1.0
- **Descripción breve:** El producto software es una plataforma web cliente-servidor diseñada para la empresa "Hospital del Computador" que permite la gestión integral de órdenes de servicio (ODS), control de inventario de repuestos, peritaje técnico mediante checklist dinámico y envío de alertas en tiempo real vía WebSockets. Ha sido construida sobre una arquitectura modular utilizando Next.js y NestJS, integrando principios de diseño UX/UI y pautas de accesibilidad WCAG 2.1 (Nivel AA) para optimizar la interacción de clientes y personal técnico.

II. COMPONENTES ENTREGADOS

A continuación, se relacionan los entregables que conforman el producto software:

Nº	Entregable	Descripción	Fecha Entrega	Observaciones
1	Código fuente	Repositorio Git con versión final del Frontend y Backend.	2026-07-06	Ninguna
2	Manual Técnico	Documentación de arquitectura, diccionario de base de datos y diseño UML.	2026-07-06	Ninguna
3	Entorno de Despliegue	Credenciales, variables de entorno y scripts de ejecución.	2026-07-06	Ninguna
4	Informe de Pruebas	Resultados de auditorías automatizadas (Lighthouse / WAVE).	2026-07-06	Ninguna

III. CRITERIOS DE ACEPTACIÓN

La empresa "Hospital del Computador" declara haber verificado los siguientes criterios y acepta el producto software:

- Cumplimiento de los requisitos funcionales y no funcionales establecidos en el *Product Backlog* y el manual técnico.
- Instalación, despliegue y conexión a la base de datos PostgreSQL correctos en el entorno de producción.
- Recepción de la documentación técnica y manuales de forma completa y legible.

IV. OBSERVACIONES GENERALES

La aplicación web para la gestión de servicios ha sido finalizada adecuadamente y se ha entregado a la empresa "Hospital del Computador" de manera satisfactoria. Dicha aplicación ha sido elaborada como parte de la formación académica y trabajo de titulación en la carrera de Software, perteneciente a la Escuela Superior Politécnica de Chimborazo (ESPOCH).

V. FIRMAS

En señal de conformidad, firman la presente acta:

ENTREGA (Por el Desarrollador):

Nombre: Kevin Alexander Urbano Lombeida

Cargo: Desarrollador de Software

Firma: 

Fecha: 2026-07-06

RECIBE (Por el Cliente):

Nombre: Ernesto Israel Quingue Marcatoma

Cargo: Gerente Propietario

Firma: 

Fecha: 2026-07-06

ANEXO C: Reporte Lighthouse – Orden de Servicio.

http://localhost:4000/ver-orden

95

Accessibility

These checks highlight opportunities to improve the accessibility of your web app. Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so manual testing is also encouraged.

CONTRAST

▲ Background and foreground colors do not have a sufficient contrast ratio.

Low-contrast text is difficult or impossible for many users to read. Learn how to provide sufficient color contrast.

Failing Elements



span.inline-flex.items-center.px-2.5.py-0.5.justify-center.gap-1.rounded-full.font-medium.text-theme-x...



div.relative.flex.flex-col.gap-4.rounded-xl.border.p-4.shadow-sm.md:flex-row.md:items-center.md:justif...



div.rounded-2xl.border.border-gray-200.bg-white.dark:border-gray-800.dark:bg-white/[0.03]

body.__className_ed3508.dark:bg-gray-900

These are opportunities to improve the legibility of your content.

BEST PRACTICES

▲ Document does not have a main landmark.

One main landmark helps screen reader users navigate a web page. Learn more about landmarks.

Failing Elements

html.dark

These items highlight common accessibility best practices.

ADDITIONAL ITEMS TO MANUALLY CHECK (10) Hide

○ Interactive controls are keyboard focusable

Custom interactive controls are keyboard focusable and display a focus indicator. Learn how to make custom controls focusable. Unscored

○ Interactive elements indicate their purpose and state

Interactive elements, such as links and buttons, should indicate their state and be distinguishable from non-interactive elements. Learn how to decorate interactive elements with affordance hints. Unscored

○ The page has a logical tab order

Tabbing through the page follows the visual layout. Users cannot focus elements that are offscreen. Learn more about logical tab ordering. Unscored

- Visual order on the page follows DOM order

  DOM order matches the visual order, improving navigation for assistive technology. Learn more about DOM and visual ordering. Unscored

- User focus is not accidentally trapped in a region

  A user can tab into and out of any control or region without accidentally trapping their focus. Learn how to avoid focus traps. Unscored

- The user's focus is directed to new content added to the page

  If new content, such as a dialog, is added to the page, the user's focus is directed to it. Learn how to direct focus to new content. Unscored

- HTML5 landmark elements are used to improve navigation

  Landmark elements (`<main>`, `<nav>`, etc.) are used to improve the keyboard navigation of the page for assistive technology. Learn more about landmark elements. Unscored

- Offscreen content is hidden from assistive technology

  Offscreen content is hidden with display: none or aria-hidden=true. Learn how to properly hide offscreen content. Unscored

- Custom controls have associated labels

  Custom interactive controls have associated labels, provided by aria-label or aria-labelledby. Learn more about custom controls and labels. Unscored

- Custom controls have ARIA roles

  Custom interactive controls have appropriate ARIA roles. Learn how to add roles to custom controls. Unscored

These items address areas which an automated testing tool cannot cover. Learn more in our guide on conducting an accessibility review.

PASSED AUDITS (23) Hide

- `[aria-*]` attributes match their roles

  Each ARIA `role` supports a specific subset of `aria-*` attributes. Mismatching these invalidates the `aria-*` attributes. Learn how to match ARIA attributes to their roles.

- `[aria-hidden="true"]` is not present on the document `<body>`

  Assistive technologies, like screen readers, work inconsistently when `aria-hidden="true"` is set on the document <body>. Learn how `aria-hidden` affects the document body.

- `[role]`s have all required `[aria-*]` attributes

  Some ARIA roles have required attributes that describe the state of the element to screen readers. Learn more about roles and required attributes.

- `[role]` values are valid

  ARIA roles must have valid values in order to perform their intended accessibility functions. Learn more about valid ARIA roles.

- `[aria-*]` attributes have valid values

  Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid values. Learn more about valid values for ARIA attributes.

- `[aria-*]` attributes are valid and not misspelled

  Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid names. Learn more about valid ARIA

attributes.

Buttons have an accessible name

When a button doesn't have an accessible name, screen readers announce it as "button", making it unusable for users who rely on screen readers. Learn how to make buttons more accessible.

Image elements have `[alt]` attributes

Informative elements should aim for short, descriptive alternate text. Decorative elements can be ignored with an empty alt attribute. Learn more about the `alt` attribute.

Form elements have associated labels

Labels ensure that form controls are announced properly by assistive technologies, like screen readers. Learn more about form element labels.

`[user-scalable="no"]` is not used in the `<meta name="viewport">` element and the `[maximum-scale]` attribute is not less than 5.

Disabling zooming is problematic for users with low vision who rely on screen magnification to properly see the contents of a web page. Learn more about the viewport meta tag.

Select elements have associated label elements.

Form elements without effective labels can create frustrating experiences for screen reader users. Learn more about the `select` element.

ARIA attributes are used as specified for the element's role

Some ARIA attributes are only allowed on an element under certain conditions. Learn more about conditional ARIA attributes.

`[aria-hidden="true"]` elements do not contain focusable descendents

Focusable descendents within an `[aria-hidden="true"]` element prevent those interactive elements from being available to users of assistive technologies like screen readers. Learn how `aria-hidden` affects focusable elements.

Elements use only permitted ARIA attributes

Using ARIA attributes in roles where they are prohibited can mean that important information is not communicated to users of assistive technologies. Learn more about prohibited ARIA roles.

Document has a `<title>` element

The title gives screen reader users an overview of the page, and search engine users rely on it heavily to determine if a page is relevant to their search. Learn more about document titles.

`<html>` element has a `[lang]` attribute

If a page doesn't specify a `lang` attribute, a screen reader assumes that the page is in the default language that the user chose when setting up the screen reader. If the page isn't actually in the default language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute.

`<html>` element has a valid value for its `[lang]` attribute

Specifying a valid BCP 47 language helps screen readers announce text properly. Learn how to use the `lang` attribute.

Links have a discernible name

Link text (and alternate text for images, when used as links) that is discernible, unique, and focusable improves the navigation experience for screen reader users. Learn how to make links accessible.

Lists contain only `<li>` elements and script supporting elements (`<script>` and `<template>`).

Screen readers have a specific way of announcing lists. Ensuring proper list structure aids screen reader output. Learn more about proper list structure.

List items (`<li>`) are contained within `<ul>`, `<ol>` or `<menu>` parent elements

Screen readers require list items (`<li>`) to be contained within a parent `<ul>`, `<ol>` or `<menu>` to be announced properly. Learn more about proper list structure.

Touch targets have sufficient size and spacing.

Touch targets with sufficient size and spacing help users who may have difficulty targeting small controls to activate the targets. Learn more about touch targets.

Heading elements appear in a sequentially-descending order

Properly ordered headings that do not skip levels convey the semantic structure of the page, making it easier to navigate and understand when using assistive technologies. Learn more about heading order.

Deprecated ARIA roles were not used

Deprecated ARIA roles may not be processed correctly by assistive technology. Learn more about deprecated ARIA roles.

NOT APPLICABLE (38)

Hide

`[accesskey]` values are unique

Access keys let users quickly focus a part of the page. For proper navigation, each access key must be unique. Learn more about access keys. Unscored

`button`, `link`, and `menuitem` elements have accessible names

When an element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to make command elements more accessible. Unscored

Elements with `role="dialog"` or `role="alertdialog"` have accessible names.

ARIA dialog elements without accessible names may prevent screen readers users from discerning the purpose of these elements. Learn how to make ARIA dialog elements more accessible. Unscored

ARIA input fields have accessible names

When an input field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about input field labels. Unscored

ARIA `meter` elements have accessible names

When a meter element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name meter elements. Unscored

ARIA `progressbar` elements have accessible names

When a progressbar element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to label progressbar elements. Unscored

Elements with an ARIA `[role]` that require children to contain a specific `[role]` have all required children.

Some ARIA parent roles must contain specific child roles to perform their intended accessibility functions. Learn more about roles and required children elements. Unscored

`[role]`s are contained by their required parent element

Some ARIA child roles must be contained by specific parent roles to properly perform their intended accessibility functions. Learn more about ARIA roles and required parent element. Unscored

Elements with the `role=text` attribute do not have focusable descendents.

Adding `role=text` around a text node split by markup enables VoiceOver to treat it as one phrase, but the element's focusable descendents will not be announced. Learn more about the `role=text` attribute. Unscored

- ARIA toggle fields have accessible names

  When a toggle field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about toggle fields. Unscored

- ARIA `tooltip` elements have accessible names

  When a tooltip element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `tooltip` elements. Unscored

- ARIA `treeitem` elements have accessible names

  When a `treeitem` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about labeling `treeitem` elements. Unscored

- The page contains a heading, skip link, or landmark region

  Adding ways to bypass repetitive content lets keyboard users navigate the page more efficiently. Learn more about bypass blocks. Unscored

- `<dl>`'s contain only properly-ordered `<dt>` and `<dd>` groups, `<script>`, `<template>` or `<div>` elements.

  When definition lists are not properly marked up, screen readers may produce confusing or inaccurate output. Learn how to structure definition lists correctly. Unscored

- Definition list items are wrapped in `<dl>` elements

  Definition list items (<dt> and <dd>) must be wrapped in a parent <dl> element to ensure that screen readers can properly announce them. Learn how to structure definition lists correctly. Unscored

- ARIA IDs are unique

  The value of an ARIA ID must be unique to prevent other instances from being overlooked by assistive technologies. Learn how to fix duplicate ARIA IDs. Unscored

- No form fields have multiple labels

  Form fields with multiple labels can be confusingly announced by assistive technologies like screen readers which use either the first, the last, or all of the labels. Learn how to use form labels. Unscored

- `<frame>` or `<iframe>` elements have a title

  Screen reader users rely on frame titles to describe the contents of frames. Learn more about frame titles. Unscored

- `<html>` element has an `[xml:lang]` attribute with the same base language as the `[lang]` attribute.

  If the webpage does not specify a consistent language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute. Unscored

- Input buttons have discernible text.

  Adding discernable and accessible text to input buttons may help screen reader users understand the purpose of the input button. Learn more about input buttons. Unscored

- `<input type="image">` elements have `[alt]` text

  When an image is being used as an `<input>` button, providing alternative text can help screen reader users understand the purpose of the button. Learn about input image alt text. Unscored

- Links are distinguishable without relying on color.

  Low-contrast text is difficult or impossible for many users to read. Link text that is discernible improves the experience for users with low vision. Learn how to make links distinguishable. Unscored

- The document does not use `<meta http-equiv="refresh">`

Users do not expect a page to refresh automatically, and doing so will move focus back to the top of the page. This may create a frustrating or confusing experience. Learn more about the refresh meta tag. Unscored

- `<object>` elements have alternate text

Screen readers cannot translate non-text content. Adding alternate text to `<object>` elements helps screen readers convey meaning to users. Learn more about alt text for `object` elements. Unscored

- Skip links are focusable.

Including a skip link can help users skip to the main content to save time. Learn more about skip links. Unscored

- No element has a `[tabindex]` value greater than 0

A value greater than 0 implies an explicit navigation ordering. Although technically valid, this often creates frustrating experiences for users who rely on assistive technologies. Learn more about the `tabindex` attribute. Unscored

- Cells in a `<table>` element that use the `[headers]` attribute refer to table cells within the same table.

Screen readers have features to make navigating tables easier. Ensuring `<td>` cells using the `[headers]` attribute only refer to other cells in the same table may improve the experience for screen reader users. Learn more about the `headers` attribute. Unscored

- `<th>` elements and elements with `[role="columnheader"/"rowheader"]` have data cells they describe.

Screen readers have features to make navigating tables easier. Ensuring table headers always refer to some set of cells may improve the experience for screen reader users. Learn more about table headers. Unscored

- `[lang]` attributes have a valid value

Specifying a valid BCP 47 language on elements helps ensure that text is pronounced correctly by a screen reader. Learn how to use the `lang` attribute. Unscored

- `<video>` elements contain a `<track>` element with `[kind="captions"]`

When a video provides a caption it is easier for deaf and hearing impaired users to access its information. Learn more about video captions. Unscored

- `autocomplete` attributes are used correctly

The `autocomplete` attribute values must be valid and correctly applied for screen readers to function correctly. Learn more about valid autocomplete values. Unscored

- Elements with `role="none"` or `role="presentation"` do not have conflicts

There are certain cases where the semantic role of an element with `role="none"` or `role="presentation"` does not resolve to none or presentation. To ensure the element remains removed from the accessibility tree, you should not add any global ARIA attributes to the element or make it focusable. Learn more about presentation role conflict. Unscored

- SVG elements with an `img` role have an accessible text alternative

Ensures SVG elements with an `img`, `graphics-document` or `graphics-symbol` role have an accessible text alternative. Learn more about SVG alt text. Unscored

- Tables have different content in the summary attribute and `<caption>`.

The summary attribute should describe the table structure, while `<caption>` should have the onscreen title. Accurate table mark-up helps users of screen readers. Learn more about summary and caption. Unscored

- All heading elements contain content.

A heading with no content or inaccessible text prevent screen reader users from accessing information on the page's structure. Learn more about headings. Unscored

- Uses ARIA roles only on compatible elements

Many HTML elements can only be assigned certain ARIA roles. Using ARIA roles where they are not allowed can interfere with the accessibility of the web page. Learn more about ARIA roles. Unscored

Image elements do not have `[alt]` attributes that are redundant text.

Informative elements should aim for short, descriptive alternative text. Alternative text that is exactly the same as the text adjacent to the link or image is potentially confusing for screen reader users, because the text will be read twice. Learn more about the `alt` attribute. Unscored

Identical links have the same purpose.

Links with the same destination should have the same description, to help users understand the link's purpose and decide whether to follow it. Learn more about identical links. Unscored

Captured at Jul 3, 2026, 11:02 AM GMT-5
Initial page load
Emulated Desktop with Lighthouse 13.3.0
Custom throttling
Single page session
Using Chromium 150.0.0.0 with devtools

Generated by **Lighthouse** 13.3.0 | File an issue

ANEXO D: Reporte Lighthouse – Inicio de sesión.

http://localhost:4000/signin

89

Accessibility

These checks highlight opportunities to improve the accessibility of your web app. Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so manual testing is also encouraged.

NAMES AND LABELS

▲ Form elements do not have associated labels

Labels ensure that form controls are announced properly by assistive technologies, like screen readers. Learn more about form element labels.

Failing Elements



input.w-5.h-5.appearance-none.cursor-pointer.dark:border-gray-700.border.border-gray-300.checked:...



label.flex.items-center.space-x-3.group.cursor-pointer

These are opportunities to improve the semantics of the controls in your application. This may enhance the experience for users of assistive technology, like a screen reader.

CONTRAST

▲ Links rely on color to be distinguishable.

Low-contrast text is difficult or impossible for many users to read. Link text that is discernible improves the experience for users with low vision. Learn how to make links distinguishable.

Failing Elements



a.text-brand-500.hover:text-brand-600.dark:text-brand-400

p.text-sm.font-normal.text-center.text-gray-700.dark:text-gray-400.sm:text-start

These are opportunities to improve the legibility of your content.

BEST PRACTICES

▲ Document does not have a main landmark.

One main landmark helps screen reader users navigate a web page. Learn more about landmarks.

Failing Elements



html.dark

Failing Elements



These items highlight common accessibility best practices.

ADDITIONAL ITEMS TO MANUALLY CHECK (10) Hide

- Interactive controls are keyboard focusable

  Custom interactive controls are keyboard focusable and display a focus indicator. Learn how to make custom controls focusable. Unscored

- Interactive elements indicate their purpose and state

  Interactive elements, such as links and buttons, should indicate their state and be distinguishable from non-interactive elements. Learn how to decorate interactive elements with affordance hints. Unscored

- The page has a logical tab order

  Tabbing through the page follows the visual layout. Users cannot focus elements that are offscreen. Learn more about logical tab ordering. Unscored

- Visual order on the page follows DOM order

  DOM order matches the visual order, improving navigation for assistive technology. Learn more about DOM and visual ordering. Unscored

- User focus is not accidentally trapped in a region

  A user can tab into and out of any control or region without accidentally trapping their focus. Learn how to avoid focus traps. Unscored

- The user's focus is directed to new content added to the page

  If new content, such as a dialog, is added to the page, the user's focus is directed to it. Learn how to direct focus to new content. Unscored

- HTML5 landmark elements are used to improve navigation

  Landmark elements (`<main>`, `<nav>`, etc.) are used to improve the keyboard navigation of the page for assistive technology. Learn more about landmark elements. Unscored

- Offscreen content is hidden from assistive technology

  Offscreen content is hidden with display: none or aria-hidden=true. Learn how to properly hide offscreen content. Unscored

- Custom controls have associated labels

  Custom interactive controls have associated labels, provided by aria-label or aria-labelledby. Learn more about custom controls and labels. Unscored

- Custom controls have ARIA roles

  Custom interactive controls have appropriate ARIA roles. Learn how to add roles to custom controls. Unscored

These items address areas which an automated testing tool cannot cover. Learn more in our guide on conducting an accessibility review.

PASSED AUDITS (21) Hide

`[aria-*]` attributes match their roles

Each ARIA `role` supports a specific subset of `aria-*` attributes. Mismatching these invalidates the `aria-*` attributes. Learn how to match ARIA attributes to their roles.

`[aria-hidden="true"]` is not present on the document `<body>`

Assistive technologies, like screen readers, work inconsistently when `aria-hidden="true"` is set on the document `<body>`. Learn how `aria-hidden` affects the document body.

`[role]`s have all required `[aria-*]` attributes

Some ARIA roles have required attributes that describe the state of the element to screen readers. Learn more about roles and required attributes.

`[role]` values are valid

ARIA roles must have valid values in order to perform their intended accessibility functions. Learn more about valid ARIA roles.

`[aria-*]` attributes have valid values

Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid values. Learn more about valid values for ARIA attributes.

`[aria-*]` attributes are valid and not misspelled

Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid names. Learn more about valid ARIA attributes.

Buttons have an accessible name

When a button doesn't have an accessible name, screen readers announce it as "button", making it unusable for users who rely on screen readers. Learn how to make buttons more accessible.

Image elements have `[alt]` attributes

Informative elements should aim for short, descriptive alternate text. Decorative elements can be ignored with an empty alt attribute. Learn more about the `alt` attribute.

`[user-scalable="no"]` is not used in the `<meta name="viewport">` element and the `[maximum-scale]` attribute is not less than 5.

Disabling zooming is problematic for users with low vision who rely on screen magnification to properly see the contents of a web page. Learn more about the viewport meta tag.

ARIA attributes are used as specified for the element's role

Some ARIA attributes are only allowed on an element under certain conditions. Learn more about conditional ARIA attributes.

`[aria-hidden="true"]` elements do not contain focusable descendents

Focusable descendents within an `[aria-hidden="true"]` element prevent those interactive elements from being available to users of assistive technologies like screen readers. Learn how `aria-hidden` affects focusable elements.

Elements use only permitted ARIA attributes

Using ARIA attributes in roles where they are prohibited can mean that important information is not communicated to users of assistive technologies. Learn more about prohibited ARIA roles.

Background and foreground colors have a sufficient contrast ratio

Low-contrast text is difficult or impossible for many users to read. Learn how to provide sufficient color contrast.

Document has a `<title>` element

The title gives screen reader users an overview of the page, and search engine users rely on it heavily to determine if a page is relevant to their search. Learn more about document titles.

`<html>` element has a `[lang]` attribute

If a page doesn't specify a `lang` attribute, a screen reader assumes that the page is in the default language that the user chose when setting up the screen reader. If the page isn't actually in the default language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute.

`<html>` element has a valid value for its `[lang]` attribute

Specifying a valid BCP 47 language helps screen readers announce text properly. Learn how to use the `lang` attribute.

Links have a discernible name

Link text (and alternate text for images, when used as links) that is discernible, unique, and focusable improves the navigation experience for screen reader users. Learn how to make links accessible.

Touch targets have sufficient size and spacing.

Touch targets with sufficient size and spacing help users who may have difficulty targeting small controls to activate the targets. Learn more about touch targets.

Heading elements appear in a sequentially-descending order

Properly ordered headings that do not skip levels convey the semantic structure of the page, making it easier to navigate and understand when using assistive technologies. Learn more about heading order.

Deprecated ARIA roles were not used

Deprecated ARIA roles may not be processed correctly by assistive technology. Learn more about deprecated ARIA roles.

`autocomplete` attributes are used correctly

The `autocomplete` attribute values must be valid and correctly applied for screen readers to function correctly. Learn more about valid autocomplete values.

NOT APPLICABLE (39) Hide

`[accesskey]` values are unique

Access keys let users quickly focus a part of the page. For proper navigation, each access key must be unique. Learn more about access keys. Unscored

`button`, `link`, and `menuitem` elements have accessible names

When an element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to make command elements more accessible. Unscored

Elements with `role="dialog"` or `role="alertdialog"` have accessible names.

ARIA dialog elements without accessible names may prevent screen readers users from discerning the purpose of these elements. Learn how to make ARIA dialog elements more accessible. Unscored

ARIA input fields have accessible names

When an input field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable

for users who rely on screen readers. Learn more about input field labels. Unscored

- ARIA `meter` elements have accessible names

  When a meter element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `meter` elements. Unscored

- ARIA `progressbar` elements have accessible names

  When a `progressbar` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to label `progressbar` elements. Unscored

- Elements with an ARIA `[role]` that require children to contain a specific `[role]` have all required children.

  Some ARIA parent roles must contain specific child roles to perform their intended accessibility functions. Learn more about roles and required children elements. Unscored

- `[role]`s are contained by their required parent element

  Some ARIA child roles must be contained by specific parent roles to properly perform their intended accessibility functions. Learn more about ARIA roles and required parent element. Unscored

- Elements with the `role=text` attribute do not have focusable descendents.

  Adding `role=text` around a text node split by markup enables VoiceOver to treat it as one phrase, but the element's focusable descendents will not be announced. Learn more about the `role=text` attribute. Unscored

- ARIA toggle fields have accessible names

  When a toggle field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about toggle fields. Unscored

- ARIA `tooltip` elements have accessible names

  When a tooltip element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `tooltip` elements. Unscored

- ARIA `treeitem` elements have accessible names

  When a `treeitem` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about labeling `treeitem` elements. Unscored

- The page contains a heading, skip link, or landmark region

  Adding ways to bypass repetitive content lets keyboard users navigate the page more efficiently. Learn more about bypass blocks. Unscored

- `<dl>`'s contain only properly-ordered `<dt>` and `<dd>` groups, `<script>`, `<template>` or `<div>` elements.

  When definition lists are not properly marked up, screen readers may produce confusing or inaccurate output. Learn how to structure definition lists correctly. Unscored

- Definition list items are wrapped in `<dl>` elements

  Definition list items (<dt> and <dd>) must be wrapped in a parent <dl> element to ensure that screen readers can properly announce them. Learn how to structure definition lists correctly. Unscored

- ARIA IDs are unique

  The value of an ARIA ID must be unique to prevent other instances from being overlooked by assistive technologies. Learn how to fix duplicate ARIA IDs. Unscored

- No form fields have multiple labels

  Form fields with multiple labels can be confusingly announced by assistive technologies like screen readers which use either the first, the last, or all of the labels. Learn how to use form labels. Unscored

- `<frame>` or `<iframe>` elements have a title

  Screen reader users rely on frame titles to describe the contents of frames. Learn more about frame titles. Unscored

- `<html>` element has an `[xml:lang]` attribute with the same base language as the `[lang]` attribute.

  If the webpage does not specify a consistent language, then the screen reader might not announce the page's text correctly. Learn more about the lang attribute. Unscored

- Input buttons have discernible text.

  Adding discernable and accessible text to input buttons may help screen reader users understand the purpose of the input button. Learn more about input buttons. Unscored

- `<input type="image">` elements have `[alt]` text

  When an image is being used as an `<input>` button, providing alternative text can help screen reader users understand the purpose of the button. Learn about input image alt text. Unscored

- Lists contain only `<li>` elements and script supporting elements (`<script>` and `<template>`).

  Screen readers have a specific way of announcing lists. Ensuring proper list structure aids screen reader output. Learn more about proper list structure. Unscored

- List items (`<li>`) are contained within `<ul>`, `<ol>` or `<menu>` parent elements

  Screen readers require list items (`<li>`) to be contained within a parent `<ul>`, `<ol>` or `<menu>` to be announced properly. Learn more about proper list structure. Unscored

- The document does not use `<meta http-equiv="refresh">`

  Users do not expect a page to refresh automatically, and doing so will move focus back to the top of the page. This may create a frustrating or confusing experience. Learn more about the refresh meta tag. Unscored

- `<object>` elements have alternate text

  Screen readers cannot translate non-text content. Adding alternate text to `<object>` elements helps screen readers convey meaning to users. Learn more about alt text for `object` elements. Unscored

- Select elements have associated label elements.

  Form elements without effective labels can create frustrating experiences for screen reader users. Learn more about the `select` element. Unscored

- Skip links are focusable.

  Including a skip link can help users skip to the main content to save time. Learn more about skip links. Unscored

- No element has a `[tabindex]` value greater than 0

  A value greater than 0 implies an explicit navigation ordering. Although technically valid, this often creates frustrating experiences for users who rely on assistive technologies. Learn more about the `tabindex` attribute. Unscored

- Cells in a `<table>` element that use the `[headers]` attribute refer to table cells within the same table.

  Screen readers have features to make navigating tables easier. Ensuring `<td>` cells using the `[headers]` attribute only refer to other cells in the same table may improve the experience for screen reader users. Learn more about the headers

attribute. Unscored

- `<th>` elements and elements with `[role="columnheader"/"rowheader"]` have data cells they describe.

  Screen readers have features to make navigating tables easier. Ensuring table headers always refer to some set of cells may improve the experience for screen reader users. Learn more about table headers. Unscored

- `[lang]` attributes have a valid value

  Specifying a valid BCP 47 language on elements helps ensure that text is pronounced correctly by a screen reader. Learn how to use the lang attribute. Unscored

- `<video>` elements contain a `<track>` element with `[kind="captions"]`

  When a video provides a caption it is easier for deaf and hearing impaired users to access its information. Learn more about video captions. Unscored

- Elements with `role="none"` or `role="presentation"` do not have conflicts

  There are certain cases where the semantic role of an element with `role="none"` or `role="presentation"` does not resolve to none or presentation. To ensure the element remains removed from the accessibility tree, you should not add any global ARIA attributes to the element or make it focusable. Learn more about presentation role conflict. Unscored

- SVG elements with an `img` role have an accessible text alternative

  Ensures SVG elements with an `img`, `graphics-document` or `graphics-symbol` role have an accessible text alternative. Learn more about SVG alt text. Unscored

- Tables have different content in the summary attribute and `<caption>`.

  The summary attribute should describe the table structure, while `<caption>` should have the onscreen title. Accurate table mark-up helps users of screen readers. Learn more about summary and caption. Unscored

- All heading elements contain content.

  A heading with no content or inaccessible text prevent screen reader users from accessing information on the page's structure. Learn more about headings. Unscored

- Uses ARIA roles only on compatible elements

  Many HTML elements can only be assigned certain ARIA roles. Using ARIA roles where they are not allowed can interfere with the accessibility of the web page. Learn more about ARIA roles. Unscored

- Image elements do not have `[alt]` attributes that are redundant text.

  Informative elements should aim for short, descriptive alternative text. Alternative text that is exactly the same as the text adjacent to the link or image is potentially confusing for screen reader users, because the text will be read twice. Learn more about the `alt` attribute. Unscored

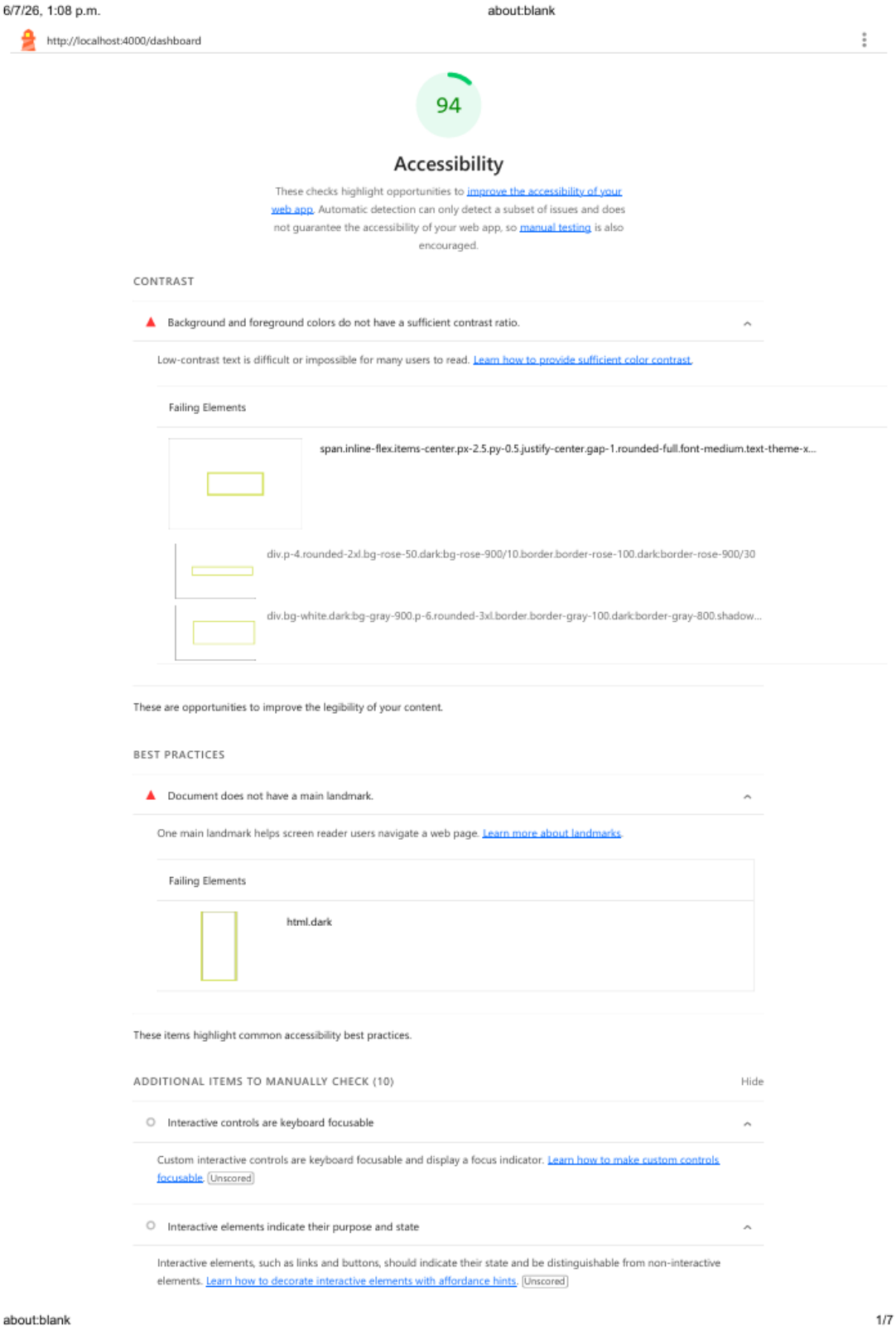
- Identical links have the same purpose.

  Links with the same destination should have the same description, to help users understand the link's purpose and decide whether to follow it. Learn more about identical links. Unscored

Captured at Jul 6, 2026, 1:02 PM GMT-5
Initial page load
Emulated Desktop with Lighthouse 13.3.0
Custom throttling
Single page session
Using Chromium 150.0.0.0 with devtools

Generated by **Lighthouse** 13.3.0 | File an issue

ANEXO E: Reporte Lighthouse – Dashboard.



- The page has a logical tab order

  Tabbing through the page follows the visual layout. Users cannot focus elements that are offscreen. Learn more about logical tab ordering. Unscored

- Visual order on the page follows DOM order

  DOM order matches the visual order, improving navigation for assistive technology. Learn more about DOM and visual ordering. Unscored

- User focus is not accidentally trapped in a region

  A user can tab into and out of any control or region without accidentally trapping their focus. Learn how to avoid focus traps. Unscored

- The user's focus is directed to new content added to the page

  If new content, such as a dialog, is added to the page, the user's focus is directed to it. Learn how to direct focus to new content. Unscored

- HTML5 landmark elements are used to improve navigation

  Landmark elements (`<main>`, `<nav>`, etc.) are used to improve the keyboard navigation of the page for assistive technology. Learn more about landmark elements. Unscored

- Offscreen content is hidden from assistive technology

  Offscreen content is hidden with display: none or aria-hidden=true. Learn how to properly hide offscreen content. Unscored

- Custom controls have associated labels

  Custom interactive controls have associated labels, provided by aria-label or aria-labelledby. Learn more about custom controls and labels. Unscored

- Custom controls have ARIA roles

  Custom interactive controls have appropriate ARIA roles. Learn how to add roles to custom controls. Unscored

These items address areas which an automated testing tool cannot cover. Learn more in our guide on conducting an accessibility review.

PASSED AUDITS (22) Hide

- `[aria-*]` attributes match their roles

  Each ARIA `role` supports a specific subset of `aria-*` attributes. Mismatching these invalidates the `aria-*` attributes. Learn how to match ARIA attributes to their roles.

- `[aria-hidden="true"]` is not present on the document `<body>`

  Assistive technologies, like screen readers, work inconsistently when `aria-hidden="true"` is set on the document <body>. Learn how `aria-hidden` affects the document body.

- `[role]`s have all required `[aria-*]` attributes

  Some ARIA roles have required attributes that describe the state of the element to screen readers. Learn more about roles and required attributes.

- `[role]` values are valid

ARIA roles must have valid values in order to perform their intended accessibility functions. Learn more about valid ARIA roles.

`[aria-*]` attributes have valid values

Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid values. Learn more about valid values for ARIA attributes.

`[aria-*]` attributes are valid and not misspelled

Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid names. Learn more about valid ARIA attributes.

Buttons have an accessible name

When a button doesn't have an accessible name, screen readers announce it as "button", making it unusable for users who rely on screen readers. Learn how to make buttons more accessible.

Image elements have `[alt]` attributes

Informative elements should aim for short, descriptive alternate text. Decorative elements can be ignored with an empty alt attribute. Learn more about the `alt` attribute.

`[user-scalable="no"]` is not used in the `<meta name="viewport">` element and the `[maximum-scale]` attribute is not less than 5.

Disabling zooming is problematic for users with low vision who rely on screen magnification to properly see the contents of a web page. Learn more about the viewport meta tag.

ARIA attributes are used as specified for the element's role

Some ARIA attributes are only allowed on an element under certain conditions. Learn more about conditional ARIA attributes.

`[aria-hidden="true"]` elements do not contain focusable descendents

Focusable descendents within an `[aria-hidden="true"]` element prevent those interactive elements from being available to users of assistive technologies like screen readers. Learn how `aria-hidden` affects focusable elements.

Elements use only permitted ARIA attributes

Using ARIA attributes in roles where they are prohibited can mean that important information is not communicated to users of assistive technologies. Learn more about prohibited ARIA roles.

Document has a `<title>` element

The title gives screen reader users an overview of the page, and search engine users rely on it heavily to determine if a page is relevant to their search. Learn more about document titles.

`<html>` element has a `[lang]` attribute

If a page doesn't specify a `lang` attribute, a screen reader assumes that the page is in the default language that the user chose when setting up the screen reader. If the page isn't actually in the default language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute.

`<html>` element has a valid value for its `[lang]` attribute

Specifying a valid BCP 47 language helps screen readers announce text properly. Learn how to use the `lang` attribute.

Links have a discernible name

Link text (and alternate text for images, when used as links) that is discernible, unique, and focusable improves the navigation experience for screen reader users. Learn how to make links accessible.

Lists contain only `<li>` elements and script supporting elements (`<script>` and `<template>`).

Screen readers have a specific way of announcing lists. Ensuring proper list structure aids screen reader output. Learn more about proper list structure.

List items (`<li>`) are contained within `<ul>`, `<ol>` or `<menu>` parent elements

Screen readers require list items (`<li>`) to be contained within a parent `<ul>`, `<ol>` or `<menu>` to be announced properly. Learn more about proper list structure.

Touch targets have sufficient size and spacing.

Touch targets with sufficient size and spacing help users who may have difficulty targeting small controls to activate the targets. Learn more about touch targets.

Cells in a `<table>` element that use the `[headers]` attribute refer to table cells within the same table.

Screen readers have features to make navigating tables easier. Ensuring `<td>` cells using the `[headers]` attribute only refer to other cells in the same table may improve the experience for screen reader users. Learn more about the `headers` attribute.

Heading elements appear in a sequentially-descending order

Properly ordered headings that do not skip levels convey the semantic structure of the page, making it easier to navigate and understand when using assistive technologies. Learn more about heading order.

Deprecated ARIA roles were not used

Deprecated ARIA roles may not be processed correctly by assistive technology. Learn more about deprecated ARIA roles.

NOT APPLICABLE (39) Hide

`[accesskey]` values are unique

Access keys let users quickly focus a part of the page. For proper navigation, each access key must be unique. Learn more about access keys. Unscored

`button`, `link`, and `menuitem` elements have accessible names

When an element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to make command elements more accessible. Unscored

Elements with `role="dialog"` or `role="alertdialog"` have accessible names.

ARIA dialog elements without accessible names may prevent screen readers users from discerning the purpose of these elements. Learn how to make ARIA dialog elements more accessible. Unscored

ARIA input fields have accessible names

When an input field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about input field labels. Unscored

ARIA `meter` elements have accessible names

When a meter element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `meter` elements. Unscored

- ARIA `progressbar` elements have accessible names

  When a `progressbar` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to label `progressbar` elements. Unscored

- Elements with an ARIA `[role]` that require children to contain a specific `[role]` have all required children.

  Some ARIA parent roles must contain specific child roles to perform their intended accessibility functions. Learn more about roles and required children elements. Unscored

- `[role]`s are contained by their required parent element

  Some ARIA child roles must be contained by specific parent roles to properly perform their intended accessibility functions. Learn more about ARIA roles and required parent element. Unscored

- Elements with the `role=text` attribute do not have focusable descendents.

  Adding `role=text` around a text node split by markup enables VoiceOver to treat it as one phrase, but the element's focusable descendents will not be announced. Learn more about the `role=text` attribute. Unscored

- ARIA toggle fields have accessible names

  When a toggle field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about toggle fields. Unscored

- ARIA `tooltip` elements have accessible names

  When a tooltip element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `tooltip` elements. Unscored

- ARIA `treeitem` elements have accessible names

  When a `treeitem` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about labeling `treeitem` elements. Unscored

- The page contains a heading, skip link, or landmark region

  Adding ways to bypass repetitive content lets keyboard users navigate the page more efficiently. Learn more about bypass blocks. Unscored

- `<dl>`'s contain only properly-ordered `<dt>` and `<dd>` groups, `<script>`, `<template>` or `<div>` elements.

  When definition lists are not properly marked up, screen readers may produce confusing or inaccurate output. Learn how to structure definition lists correctly. Unscored

- Definition list items are wrapped in `<dl>` elements

  Definition list items (<dt> and <dd>) must be wrapped in a parent <dl> element to ensure that screen readers can properly announce them. Learn how to structure definition lists correctly. Unscored

- ARIA IDs are unique

  The value of an ARIA ID must be unique to prevent other instances from being overlooked by assistive technologies. Learn how to fix duplicate ARIA IDs. Unscored

- No form fields have multiple labels

  Form fields with multiple labels can be confusingly announced by assistive technologies like screen readers which use either the first, the last, or all of the labels. Learn how to use form labels. Unscored

- `<frame>` or `<iframe>` elements have a title

  Screen reader users rely on frame titles to describe the contents of frames. Learn more about frame titles. Unscored

- `<html>` element has an `[xml:lang]` attribute with the same base language as the `[lang]` attribute.

  If the webpage does not specify a consistent language, then the screen reader might not announce the page's text correctly. Learn more about the lang attribute. Unscored

- Input buttons have discernible text.

  Adding discernable and accessible text to input buttons may help screen reader users understand the purpose of the input button. Learn more about input buttons. Unscored

- `<input type="image">` elements have `[alt]` text

  When an image is being used as an `<input>` button, providing alternative text can help screen reader users understand the purpose of the button. Learn about input image alt text. Unscored

- Form elements have associated labels

  Labels ensure that form controls are announced properly by assistive technologies, like screen readers. Learn more about form element labels. Unscored

- Links are distinguishable without relying on color.

  Low-contrast text is difficult or impossible for many users to read. Link text that is discernible improves the experience for users with low vision. Learn how to make links distinguishable. Unscored

- The document does not use `<meta http-equiv="refresh">`

  Users do not expect a page to refresh automatically, and doing so will move focus back to the top of the page. This may create a frustrating or confusing experience. Learn more about the refresh meta tag. Unscored

- `<object>` elements have alternate text

  Screen readers cannot translate non-text content. Adding alternate text to `<object>` elements helps screen readers convey meaning to users. Learn more about alt text for `object` elements. Unscored

- Select elements have associated label elements.

  Form elements without effective labels can create frustrating experiences for screen reader users. Learn more about the `select` element. Unscored

- Skip links are focusable.

  Including a skip link can help users skip to the main content to save time. Learn more about skip links. Unscored

- No element has a `[tabindex]` value greater than 0

  A value greater than 0 implies an explicit navigation ordering. Although technically valid, this often creates frustrating experiences for users who rely on assistive technologies. Learn more about the `tabindex` attribute. Unscored

- `<th>` elements and elements with `[role="columnheader"/"rowheader"]` have data cells they describe.

  Screen readers have features to make navigating tables easier. Ensuring table headers always refer to some set of cells may improve the experience for screen reader users. Learn more about table headers. Unscored

- `[lang]` attributes have a valid value

Specifying a valid BCP 47 language on elements helps ensure that text is pronounced correctly by a screen reader. Learn how to use the lang attribute. Unscored

○ `<video>` elements contain a `<track>` element with `[kind="captions"]`

When a video provides a caption it is easier for deaf and hearing impaired users to access its information. Learn more about video captions. Unscored

○ `autocomplete` attributes are used correctly

The `autocomplete` attribute values must be valid and correctly applied for screen readers to function correctly. Learn more about valid autocomplete values. Unscored

○ Elements with `role="none"` or `role="presentation"` do not have conflicts

There are certain cases where the semantic role of an element with `role="none"` or `role="presentation"` does not resolve to none or presentation. To ensure the element remains removed from the accessibility tree, you should not add any global ARIA attributes to the element or make it focusable. Learn more about presentation role conflict. Unscored

○ SVG elements with an `img` role have an accessible text alternative

Ensures SVG elements with an `img`, `graphics-document` or `graphics-symbol` role have an accessible text alternative. Learn more about SVG alt text. Unscored

○ Tables have different content in the summary attribute and `<caption>`.

The summary attribute should describe the table structure, while `<caption>` should have the onscreen title. Accurate table mark-up helps users of screen readers. Learn more about summary and caption. Unscored

○ All heading elements contain content.

A heading with no content or inaccessible text prevent screen reader users from accessing information on the page's structure. Learn more about headings. Unscored

○ Uses ARIA roles only on compatible elements

Many HTML elements can only be assigned certain ARIA roles. Using ARIA roles where they are not allowed can interfere with the accessibility of the web page. Learn more about ARIA roles. Unscored

○ Image elements do not have `[alt]` attributes that are redundant text.

Informative elements should aim for short, descriptive alternative text. Alternative text that is exactly the same as the text adjacent to the link or image is potentially confusing for screen reader users, because the text will be read twice. Learn more about the `alt` attribute. Unscored

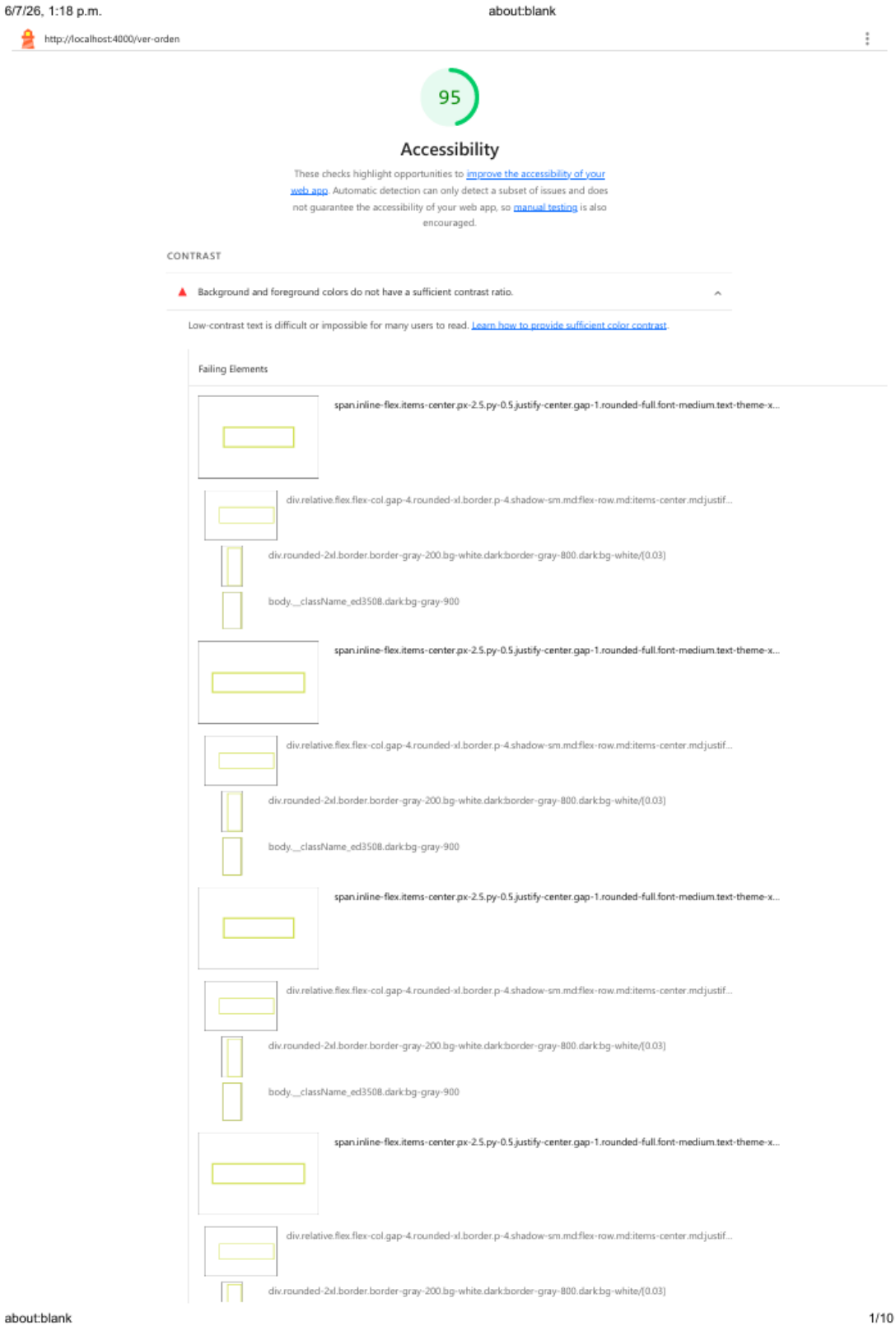
○ Identical links have the same purpose.

Links with the same destination should have the same description, to help users understand the link's purpose and decide whether to follow it. Learn more about identical links. Unscored

Captured at Jul 6, 2026, 1:03 PM GMT-5
Initial page load
Emulated Desktop with Lighthouse 13.3.0
Custom throttling
Single page session
Using Chromium 150.0.0.0 with devtools

Generated by **Lighthouse** 13.3.0 | File an issue

ANEXO F: Reporte Lighthouse – Módulo de diagnóstico técnico.



Failing Elements



Failing Elements



body.__className_ed3508.dark:bg-gray-900

span.inline-flex.items-center.px-2.5.py-0.5.justify-center.gap-1.rounded-full.font-medium.text-theme-x...



div.relative.flex.flex-col.gap-4.rounded-xl.border.p-4.shadow-sm.md:flex-row.md:items-center.md:justif...



div.rounded-2xl.border.border-gray-200.bg-white.dark:border-gray-800.dark:bg-white/[0.03]

body.__className_ed3508.dark:bg-gray-900

span.inline-flex.items-center.px-2.5.py-0.5.justify-center.gap-1.rounded-full.font-medium.text-theme-x...



div.relative.flex.flex-col.gap-4.rounded-xl.border.p-4.shadow-sm.md:flex-row.md:items-center.md:justif...



div.rounded-2xl.border.border-gray-200.bg-white.dark:border-gray-800.dark:bg-white/[0.03]



body.__className_ed3508.dark:bg-gray-900



span.inline-flex.items-center.px-2.5.py-0.5.justify-center.gap-1.rounded-full.font-medium.text-theme-x...



div.relative.flex.flex-col.gap-4.rounded-xl.border.p-4.shadow-sm.md:flex-row.md:items-center.md:justif...



div.rounded-2xl.border.border-gray-200.bg-white.dark:border-gray-800.dark:bg-white/[0.03]

body.__className_ed3508.dark:bg-gray-900



span.inline-flex.items-center.px-2.5.py-0.5.justify-center.gap-1.rounded-full.font-medium.text-theme-x...



div.relative.flex.flex-col.gap-4.rounded-xl.border.p-4.shadow-sm.md:flex-row.md:items-center.md:justif...



div.rounded-2xl.border.border-gray-200.bg-white.dark:border-gray-800.dark:bg-white/[0.03]

body.__className_ed3508.dark:bg-gray-900

span.inline-flex.items-center.px-2.5.py-0.5.justify-center.gap-1.rounded-full.font-medium.text-theme-x...



div.relative.flex.flex-col.gap-4.rounded-xl.border.p-4.shadow-sm.md:flex-row.md:items-center.md:justif...



div.rounded-2xl.border.border-gray-200.bg-white.dark:border-gray-800.dark:bg-white/[0.03]

Failing Elements



These are opportunities to improve the legibility of your content.

BEST PRACTICES

▲ Document does not have a main landmark. ^

One main landmark helps screen reader users navigate a web page. Learn more about landmarks.

Failing Elements

These items highlight common accessibility best practices.

ADDITIONAL ITEMS TO MANUALLY CHECK (10) Hide

○ Interactive controls are keyboard focusable ^

Custom interactive controls are keyboard focusable and display a focus indicator. Learn how to make custom controls focusable. Unscored

○ Interactive elements indicate their purpose and state ^

Interactive elements, such as links and buttons, should indicate their state and be distinguishable from non-interactive elements. Learn how to decorate interactive elements with affordance hints. Unscored

○ The page has a logical tab order ^

Tabbing through the page follows the visual layout. Users cannot focus elements that are offscreen. Learn more about logical tab ordering. Unscored

- Visual order on the page follows DOM order

  DOM order matches the visual order, improving navigation for assistive technology. Learn more about DOM and visual ordering. Unscored

- User focus is not accidentally trapped in a region

  A user can tab into and out of any control or region without accidentally trapping their focus. Learn how to avoid focus traps. Unscored

- The user's focus is directed to new content added to the page

  If new content, such as a dialog, is added to the page, the user's focus is directed to it. Learn how to direct focus to new content. Unscored

- HTML5 landmark elements are used to improve navigation

  Landmark elements (`<main>`, `<nav>`, etc.) are used to improve the keyboard navigation of the page for assistive technology. Learn more about landmark elements. Unscored

- Offscreen content is hidden from assistive technology

  Offscreen content is hidden with display: none or aria-hidden=true. Learn how to properly hide offscreen content. Unscored

- Custom controls have associated labels

  Custom interactive controls have associated labels, provided by aria-label or aria-labelledby. Learn more about custom controls and labels. Unscored

- Custom controls have ARIA roles

  Custom interactive controls have appropriate ARIA roles. Learn how to add roles to custom controls. Unscored

These items address areas which an automated testing tool cannot cover. Learn more in our guide on conducting an accessibility review.

PASSED AUDITS (23) Hide

- `[aria-*]` attributes match their roles

  Each ARIA `role` supports a specific subset of `aria-*` attributes. Mismatching these invalidates the `aria-*` attributes. Learn how to match ARIA attributes to their roles.

- `[aria-hidden="true"]` is not present on the document `<body>`

  Assistive technologies, like screen readers, work inconsistently when `aria-hidden="true"` is set on the document <body>. Learn how `aria-hidden` affects the document body.

- `[role]`s have all required `[aria-*]` attributes

  Some ARIA roles have required attributes that describe the state of the element to screen readers. Learn more about roles and required attributes.

- `[role]` values are valid

  ARIA roles must have valid values in order to perform their intended accessibility functions. Learn more about valid ARIA roles.

- `[aria-*]` attributes have valid values

  Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid values. Learn more about valid values for ARIA attributes.

- `[aria-*]` attributes are valid and not misspelled

  Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid names. Learn more about valid ARIA attributes.

Buttons have an accessible name

When a button doesn't have an accessible name, screen readers announce it as "button", making it unusable for users who rely on screen readers. Learn how to make buttons more accessible.

Image elements have `[alt]` attributes

Informative elements should aim for short, descriptive alternate text. Decorative elements can be ignored with an empty alt attribute. Learn more about the `alt` attribute.

Form elements have associated labels

Labels ensure that form controls are announced properly by assistive technologies, like screen readers. Learn more about form element labels.

`[user-scalable="no"]` is not used in the `<meta name="viewport">` element and the `[maximum-scale]` attribute is not less than 5.

Disabling zooming is problematic for users with low vision who rely on screen magnification to properly see the contents of a web page. Learn more about the viewport meta tag.

Select elements have associated label elements.

Form elements without effective labels can create frustrating experiences for screen reader users. Learn more about the `select` element.

ARIA attributes are used as specified for the element's role

Some ARIA attributes are only allowed on an element under certain conditions. Learn more about conditional ARIA attributes.

`[aria-hidden="true"]` elements do not contain focusable descendents

Focusable descendents within an `[aria-hidden="true"]` element prevent those interactive elements from being available to users of assistive technologies like screen readers. Learn how `aria-hidden` affects focusable elements.

Elements use only permitted ARIA attributes

Using ARIA attributes in roles where they are prohibited can mean that important information is not communicated to users of assistive technologies. Learn more about prohibited ARIA roles.

Document has a `<title>` element

The title gives screen reader users an overview of the page, and search engine users rely on it heavily to determine if a page is relevant to their search. Learn more about document titles.

`<html>` element has a `[lang]` attribute

If a page doesn't specify a `lang` attribute, a screen reader assumes that the page is in the default language that the user chose when setting up the screen reader. If the page isn't actually in the default language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute.

`<html>` element has a valid value for its `[lang]` attribute

Specifying a valid BCP 47 language helps screen readers announce text properly. Learn how to use the `lang` attribute.

Links have a discernible name

Link text (and alternate text for images, when used as links) that is discernible, unique, and focusable improves the navigation experience for screen reader users. Learn how to make links accessible.

Lists contain only `<li>` elements and script supporting elements (`<script>` and `<template>`).

Screen readers have a specific way of announcing lists. Ensuring proper list structure aids screen reader output. Learn more about proper list structure.

List items (`<li>`) are contained within `<ul>`, `<ol>` or `<menu>` parent elements

Screen readers require list items (`<li>`) to be contained within a parent `<ul>`, `<ol>` or `<menu>` to be announced properly. Learn more about proper list structure.

Touch targets have sufficient size and spacing.

Touch targets with sufficient size and spacing help users who may have difficulty targeting small controls to activate the targets. Learn more about touch targets.

Heading elements appear in a sequentially-descending order

Properly ordered headings that do not skip levels convey the semantic structure of the page, making it easier to navigate and understand when using assistive technologies. Learn more about heading order.

Deprecated ARIA roles were not used

Deprecated ARIA roles may not be processed correctly by assistive technology. Learn more about deprecated ARIA roles.

NOT APPLICABLE (38) Hide

`[accesskey]` values are unique

Access keys let users quickly focus a part of the page. For proper navigation, each access key must be unique. Learn more about access keys. Unscored

`button`, `link`, and `menuitem` elements have accessible names

When an element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to make command elements more accessible. Unscored

Elements with `role="dialog"` or `role="alertdialog"` have accessible names.

ARIA dialog elements without accessible names may prevent screen readers users from discerning the purpose of these elements. Learn how to make ARIA dialog elements more accessible. Unscored

ARIA input fields have accessible names

When an input field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about input field labels. Unscored

ARIA `meter` elements have accessible names

When a meter element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name meter elements. Unscored

ARIA `progressbar` elements have accessible names

When a progressbar element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to label progressbar elements. Unscored

Elements with an ARIA `[role]` that require children to contain a specific `[role]` have all required children.

Some ARIA parent roles must contain specific child roles to perform their intended accessibility functions. Learn more about roles and required children elements. Unscored

`[role]`s are contained by their required parent element

Some ARIA child roles must be contained by specific parent roles to properly perform their intended accessibility functions. Learn more about ARIA roles and required parent element. Unscored

Elements with the `role=text` attribute do not have focusable descendents.

Adding `role=text` around a text node split by markup enables VoiceOver to treat it as one phrase, but the element's focusable descendents will not be announced. Learn more about the `role=text` attribute. Unscored

- ARIA toggle fields have accessible names

  When a toggle field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about toggle fields. Unscored

- ARIA `tooltip` elements have accessible names

  When a tooltip element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `tooltip` elements. Unscored

- ARIA `treeitem` elements have accessible names

  When a `treeitem` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about labeling `treeitem` elements. Unscored

- The page contains a heading, skip link, or landmark region

  Adding ways to bypass repetitive content lets keyboard users navigate the page more efficiently. Learn more about bypass blocks. Unscored

- `<dl>`'s contain only properly-ordered `<dt>` and `<dd>` groups, `<script>`, `<template>` or `<div>` elements.

  When definition lists are not properly marked up, screen readers may produce confusing or inaccurate output. Learn how to structure definition lists correctly. Unscored

- Definition list items are wrapped in `<dl>` elements

  Definition list items (<dt> and <dd>) must be wrapped in a parent <dl> element to ensure that screen readers can properly announce them. Learn how to structure definition lists correctly. Unscored

- ARIA IDs are unique

  The value of an ARIA ID must be unique to prevent other instances from being overlooked by assistive technologies. Learn how to fix duplicate ARIA IDs. Unscored

- No form fields have multiple labels

  Form fields with multiple labels can be confusingly announced by assistive technologies like screen readers which use either the first, the last, or all of the labels. Learn how to use form labels. Unscored

- `<frame>` or `<iframe>` elements have a title

  Screen reader users rely on frame titles to describe the contents of frames. Learn more about frame titles. Unscored

- `<html>` element has an `[xml:lang]` attribute with the same base language as the `[lang]` attribute.

  If the webpage does not specify a consistent language, then the screen reader might not announce the page's text correctly. Learn more about the lang attribute. Unscored

- Input buttons have discernible text.

  Adding discernable and accessible text to input buttons may help screen reader users understand the purpose of the input button. Learn more about input buttons. Unscored

- `<input type="image">` elements have `[alt]` text

  When an image is being used as an <input> button, providing alternative text can help screen reader users understand the purpose of the button. Learn about input image alt text. Unscored

- Links are distinguishable without relying on color.

  Low-contrast text is difficult or impossible for many users to read. Link text that is discernible improves the experience for users with low vision. Learn how to make links distinguishable. Unscored

- The document does not use `<meta http-equiv="refresh">`

Users do not expect a page to refresh automatically, and doing so will move focus back to the top of the page. This may create a frustrating or confusing experience. Learn more about the refresh meta tag. Unscored

○ `<object>` elements have alternate text

Screen readers cannot translate non-text content. Adding alternate text to `<object>` elements helps screen readers convey meaning to users. Learn more about alt text for `object` elements. Unscored

○ Skip links are focusable.

Including a skip link can help users skip to the main content to save time. Learn more about skip links. Unscored

○ No element has a `[tabindex]` value greater than 0

A value greater than 0 implies an explicit navigation ordering. Although technically valid, this often creates frustrating experiences for users who rely on assistive technologies. Learn more about the `tabindex` attribute. Unscored

○ Cells in a `<table>` element that use the `[headers]` attribute refer to table cells within the same table.

Screen readers have features to make navigating tables easier. Ensuring `<td>` cells using the `[headers]` attribute only refer to other cells in the same table may improve the experience for screen reader users. Learn more about the `headers` attribute. Unscored

○ `<th>` elements and elements with `[role="columnheader"/"rowheader"]` have data cells they describe.

Screen readers have features to make navigating tables easier. Ensuring table headers always refer to some set of cells may improve the experience for screen reader users. Learn more about table headers. Unscored

○ `[lang]` attributes have a valid value

Specifying a valid BCP 47 language on elements helps ensure that text is pronounced correctly by a screen reader. Learn how to use the `lang` attribute. Unscored

○ `<video>` elements contain a `<track>` element with `[kind="captions"]`

When a video provides a caption it is easier for deaf and hearing impaired users to access its information. Learn more about video captions. Unscored

○ `autocomplete` attributes are used correctly

The `autocomplete` attribute values must be valid and correctly applied for screen readers to function correctly. Learn more about valid autocomplete values. Unscored

○ Elements with `role="none"` or `role="presentation"` do not have conflicts

There are certain cases where the semantic role of an element with `role="none"` or `role="presentation"` does not resolve to none or presentation. To ensure the element remains removed from the accessibility tree, you should not add any global ARIA attributes to the element or make it focusable. Learn more about presentation role conflict. Unscored

○ SVG elements with an `img` role have an accessible text alternative

Ensures SVG elements with an `img`, `graphics-document` or `graphics-symbol` role have an accessible text alternative. Learn more about SVG alt text. Unscored

○ Tables have different content in the summary attribute and `<caption>`.

The summary attribute should describe the table structure, while `<caption>` should have the onscreen title. Accurate table mark-up helps users of screen readers. Learn more about summary and caption. Unscored

○ All heading elements contain content.

A heading with no content or inaccessible text prevent screen reader users from accessing information on the page's structure. Learn more about headings. Unscored

○ Uses ARIA roles only on compatible elements

Many HTML elements can only be assigned certain ARIA roles. Using ARIA roles where they are not allowed can interfere with the accessibility of the web page. Learn more about ARIA roles. Unscored

Image elements do not have `[alt]` attributes that are redundant text.

Informative elements should aim for short, descriptive alternative text. Alternative text that is exactly the same as the text adjacent to the link or image is potentially confusing for screen reader users, because the text will be read twice. Learn more about the `alt` attribute. Unscored

Identical links have the same purpose.

Links with the same destination should have the same description, to help users understand the link's purpose and decide whether to follow it. Learn more about identical links. Unscored

Captured at Jul 6, 2026, 1:17 PM GMT-5
Initial page load
Emulated Desktop with Lighthouse 13.3.0
Custom throttling
Single page session
Using Chromium 150.0.0.0 with devtools

Generated by **Lighthouse** 13.3.0 | File an issue

ANEXO G: Reporte Lighthouse – Consulta pública del estado de reparación.

http://localhost:4000/consulta

98

Accessibility

These checks highlight opportunities to improve the accessibility of your web app. Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so manual testing is also encouraged.

NAVIGATION

▲ Heading elements are not in a sequentially-descending order

Properly ordered headings that do not skip levels convey the semantic structure of the page, making it easier to navigate and understand when using assistive technologies. Learn more about heading order.

Failing Elements



h4.text-sm.font-semibold.tracking-wider.uppercase.text-gray-900.dark:text-white

These are opportunities to improve keyboard navigation in your application.

ADDITIONAL ITEMS TO MANUALLY CHECK (10) Hide

○ Interactive controls are keyboard focusable

Custom interactive controls are keyboard focusable and display a focus indicator. Learn how to make custom controls focusable. Unscored

○ Interactive elements indicate their purpose and state

Interactive elements, such as links and buttons, should indicate their state and be distinguishable from non-interactive elements. Learn how to decorate interactive elements with affordance hints. Unscored

○ The page has a logical tab order

Tabbing through the page follows the visual layout. Users cannot focus elements that are offscreen. Learn more about logical tab ordering. Unscored

- Visual order on the page follows DOM order

  DOM order matches the visual order, improving navigation for assistive technology. Learn more about DOM and visual ordering. Unscored

- User focus is not accidentally trapped in a region

  A user can tab into and out of any control or region without accidentally trapping their focus. Learn how to avoid focus traps. Unscored

- The user's focus is directed to new content added to the page

  If new content, such as a dialog, is added to the page, the user's focus is directed to it. Learn how to direct focus to new content. Unscored

- HTML5 landmark elements are used to improve navigation

  Landmark elements (`<main>`, `<nav>`, etc.) are used to improve the keyboard navigation of the page for assistive technology. Learn more about landmark elements. Unscored

- Offscreen content is hidden from assistive technology

  Offscreen content is hidden with display: none or aria-hidden=true. Learn how to properly hide offscreen content. Unscored

- Custom controls have associated labels

  Custom interactive controls have associated labels, provided by aria-label or aria-labelledby. Learn more about custom controls and labels. Unscored

- Custom controls have ARIA roles

  Custom interactive controls have appropriate ARIA roles. Learn how to add roles to custom controls. Unscored

These items address areas which an automated testing tool cannot cover. Learn more in our guide on conducting an accessibility review.

PASSED AUDITS (23) Hide

- `[aria-*]` attributes match their roles

  Each ARIA `role` supports a specific subset of `aria-*` attributes. Mismatching these invalidates the `aria-*` attributes. Learn how to match ARIA attributes to their roles.

`[aria-hidden="true"]` is not present on the document `<body>`

Assistive technologies, like screen readers, work inconsistently when `aria-hidden="true"` is set on the document `<body>`. Learn how `aria-hidden` affects the document body.

`[role]`s have all required `[aria-*]` attributes

Some ARIA roles have required attributes that describe the state of the element to screen readers. Learn more about roles and required attributes.

`[role]` values are valid

ARIA roles must have valid values in order to perform their intended accessibility functions. Learn more about valid ARIA roles.

`[aria-*]` attributes have valid values

Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid values. Learn more about valid values for ARIA attributes.

`[aria-*]` attributes are valid and not misspelled

Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid names. Learn more about valid ARIA attributes.

Buttons have an accessible name

When a button doesn't have an accessible name, screen readers announce it as "button", making it unusable for users who rely on screen readers. Learn how to make buttons more accessible.

Form elements have associated labels

Labels ensure that form controls are announced properly by assistive technologies, like screen readers. Learn more about form element labels.

`[user-scalable="no"]` is not used in the `<meta name="viewport">` element and the `[maximum-scale]` attribute is not less than 5.

Disabling zooming is problematic for users with low vision who rely on screen magnification to properly see the contents of a web page. Learn more about the viewport meta tag.

ARIA attributes are used as specified for the element's role

Some ARIA attributes are only allowed on an element under certain conditions. Learn more about conditional ARIA attributes.

`[aria-hidden="true"]` elements do not contain focusable descendents

Focusable descendents within an `[aria-hidden="true"]` element prevent those interactive elements from being available to users of assistive technologies like screen readers. Learn how `aria-hidden` affects focusable elements.

Elements use only permitted ARIA attributes

Using ARIA attributes in roles where they are prohibited can mean that important information is not communicated to users of assistive technologies. Learn more about prohibited ARIA roles.

Background and foreground colors have a sufficient contrast ratio

Low-contrast text is difficult or impossible for many users to read. Learn how to provide sufficient color contrast.

Document has a `<title>` element

The title gives screen reader users an overview of the page, and search engine users rely on it heavily to determine if a page is relevant to their search. Learn more about document titles.

`<html>` element has a `[lang]` attribute

If a page doesn't specify a `lang` attribute, a screen reader assumes that the page is in the default language that the user chose when setting up the screen reader. If the page isn't actually in the default language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute.

`<html>` element has a valid value for its `[lang]` attribute

Specifying a valid BCP 47 language helps screen readers announce text properly. Learn how to use the `lang` attribute.

Links have a discernible name

Link text (and alternate text for images, when used as links) that is discernible, unique, and focusable improves the navigation experience for screen reader users. Learn how to make links accessible.

Lists contain only `<li>` elements and script supporting elements (`<script>` and `<template>`).

Screen readers have a specific way of announcing lists. Ensuring proper list structure aids screen reader output. Learn more about proper list structure.

List items (`<li>`) are contained within `<ul>`, `<ol>` or `<menu>` parent elements

Screen readers require list items (`<li>`) to be contained within a parent `<ul>`, `<ol>` or `<menu>` to be announced properly. Learn more about proper list structure.

No element has a `[tabindex]` value greater than 0

A value greater than 0 implies an explicit navigation ordering. Although technically valid, this often creates frustrating experiences for users who rely on assistive technologies. Learn more about the `tabindex` attribute.

Touch targets have sufficient size and spacing.

Touch targets with sufficient size and spacing help users who may have difficulty targeting small controls to activate the targets. Learn more about touch targets.

Document has a main landmark.

One main landmark helps screen reader users navigate a web page. Learn more about landmarks.

Deprecated ARIA roles were not used

Deprecated ARIA roles may not be processed correctly by assistive technology. Learn more about deprecated ARIA roles.

NOT APPLICABLE (39) Hide

`[accesskey]` values are unique

Access keys let users quickly focus a part of the page. For proper navigation, each access key must be unique. Learn more about access keys. Unscored

`button`, `link`, and `menuitem` elements have accessible names

When an element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to make command elements more accessible. Unscored

Elements with `role="dialog"` or `role="alertdialog"` have accessible names.

ARIA dialog elements without accessible names may prevent screen readers users from discerning the purpose of these elements. Learn how to make ARIA dialog elements more accessible. Unscored

ARIA input fields have accessible names

When an input field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about input field labels. Unscored

ARIA `meter` elements have accessible names

When a meter element doesn't have an accessible name, screen readers announce it with a generic name, making it

unusable for users who rely on screen readers. Learn how to name `meter` elements. Unscored

- ARIA `progressbar` elements have accessible names

  When a `progressbar` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to label `progressbar` elements. Unscored

- Elements with an ARIA `[role]` that require children to contain a specific `[role]` have all required children.

  Some ARIA parent roles must contain specific child roles to perform their intended accessibility functions. Learn more about roles and required children elements. Unscored

- `[role]`s are contained by their required parent element

  Some ARIA child roles must be contained by specific parent roles to properly perform their intended accessibility functions. Learn more about ARIA roles and required parent element. Unscored

- Elements with the `role=text` attribute do not have focusable descendents.

  Adding `role=text` around a text node split by markup enables VoiceOver to treat it as one phrase, but the element's focusable descendents will not be announced. Learn more about the `role=text` attribute. Unscored

- ARIA toggle fields have accessible names

  When a toggle field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about toggle fields. Unscored

- ARIA `tooltip` elements have accessible names

  When a tooltip element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `tooltip` elements. Unscored

- ARIA `treeitem` elements have accessible names

  When a `treeitem` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about labeling `treeitem` elements. Unscored

- The page contains a heading, skip link, or landmark region

  Adding ways to bypass repetitive content lets keyboard users navigate the page more efficiently. Learn more about bypass blocks. Unscored

- `<dl>`'s contain only properly-ordered `<dt>` and `<dd>` groups, `<script>`, `<template>` or `<div>` elements.

When definition lists are not properly marked up, screen readers may produce confusing or inaccurate output. Learn how to structure definition lists correctly. Unscored

- Definition list items are wrapped in `<dl>` elements

  Definition list items (`<dt>` and `<dd>`) must be wrapped in a parent `<dl>` element to ensure that screen readers can properly announce them. Learn how to structure definition lists correctly. Unscored

- ARIA IDs are unique

  The value of an ARIA ID must be unique to prevent other instances from being overlooked by assistive technologies. Learn how to fix duplicate ARIA IDs. Unscored

- No form fields have multiple labels

  Form fields with multiple labels can be confusingly announced by assistive technologies like screen readers which use either the first, the last, or all of the labels. Learn how to use form labels. Unscored

- `<frame>` or `<iframe>` elements have a title

  Screen reader users rely on frame titles to describe the contents of frames. Learn more about frame titles. Unscored

- `<html>` element has an `[xml:lang]` attribute with the same base language as the `[lang]` attribute.

  If the webpage does not specify a consistent language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute. Unscored

- Image elements have `[alt]` attributes

  Informative elements should aim for short, descriptive alternate text. Decorative elements can be ignored with an empty alt attribute. Learn more about the `alt` attribute. Unscored

- Input buttons have discernible text.

  Adding discernable and accessible text to input buttons may help screen reader users understand the purpose of the input button. Learn more about input buttons. Unscored

- `<input type="image">` elements have `[alt]` text

  When an image is being used as an `<input>` button, providing alternative text can help screen reader users understand the purpose of the button. Learn about input image alt text. Unscored

- Links are distinguishable without relying on color.

Low-contrast text is difficult or impossible for many users to read. Link text that is discernible improves the experience for users with low vision. Learn how to make links distinguishable. Unscored

- The document does not use `<meta http-equiv="refresh">`

  Users do not expect a page to refresh automatically, and doing so will move focus back to the top of the page. This may create a frustrating or confusing experience. Learn more about the refresh meta tag. Unscored

- `<object>` elements have alternate text

  Screen readers cannot translate non-text content. Adding alternate text to `<object>` elements helps screen readers convey meaning to users. Learn more about alt text for `object` elements. Unscored

- Select elements have associated label elements.

  Form elements without effective labels can create frustrating experiences for screen reader users. Learn more about the `select` element. Unscored

- Skip links are focusable.

  Including a skip link can help users skip to the main content to save time. Learn more about skip links. Unscored

- Cells in a `<table>` element that use the `[headers]` attribute refer to table cells within the same table.

  Screen readers have features to make navigating tables easier. Ensuring `<td>` cells using the `[headers]` attribute only refer to other cells in the same table may improve the experience for screen reader users. Learn more about the `headers` attribute. Unscored

- `<th>` elements and elements with `[role="columnheader"/"rowheader"]` have data cells they describe.

  Screen readers have features to make navigating tables easier. Ensuring table headers always refer to some set of cells may improve the experience for screen reader users. Learn more about table headers. Unscored

- `[lang]` attributes have a valid value

  Specifying a valid BCP 47 language on elements helps ensure that text is pronounced correctly by a screen reader. Learn how to use the `lang` attribute. Unscored

- `<video>` elements contain a `<track>` element with `[kind="captions"]`

  When a video provides a caption it is easier for deaf and hearing impaired users to access its information. Learn more about video captions. Unscored

- `autocomplete` attributes are used correctly

The `autocomplete` attribute values must be valid and correctly applied for screen readers to function correctly. Learn more about valid autocomplete values. Unscored

- Elements with `role="none"` or `role="presentation"` do not have conflicts

There are certain cases where the semantic role of an element with `role="none"` or `role="presentation"` does not resolve to none or presentation. To ensure the element remains removed from the accessibility tree, you should not add any global ARIA attributes to the element or make it focusable. Learn more about presentation role conflict. Unscored

- SVG elements with an `img` role have an accessible text alternative

Ensures SVG elements with an `img`, `graphics-document` or `graphics-symbol` role have an accessible text alternative. Learn more about SVG alt text. Unscored

- Tables have different content in the summary attribute and `<caption>`.

The summary attribute should describe the table structure, while `<caption>` should have the onscreen title. Accurate table mark-up helps users of screen readers. Learn more about summary and caption. Unscored

- All heading elements contain content.

A heading with no content or inaccessible text prevent screen reader users from accessing information on the page's structure. Learn more about headings. Unscored

- Uses ARIA roles only on compatible elements

Many HTML elements can only be assigned certain ARIA roles. Using ARIA roles where they are not allowed can interfere with the accessibility of the web page. Learn more about ARIA roles. Unscored

- Image elements do not have `[alt]` attributes that are redundant text.

Informative elements should aim for short, descriptive alternative text. Alternative text that is exactly the same as the text adjacent to the link or image is potentially confusing for screen reader users, because the text will be read twice. Learn more about the `alt` attribute. Unscored

- Identical links have the same purpose.

Links with the same destination should have the same description, to help users understand the link's purpose and decide whether to follow it. Learn more about identical links. Unscored

Captured at Jul 6, 2026, 1:22 PM GMT-5 | Emulated Desktop with Lighthouse 13.3.0 | Single page session

Initial page load | Custom throttling | Using Chromium 150.0.0.0 with devtools

Generated by **Lighthouse** 13.3.0 | File an issue



ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

CERTIFICADO DE CUMPLIMIENTO DE LA GUÍA PARA NORMALIZACIÓN DE TRABAJOS DE FIN DE GRADO

Fecha de entrega:

INFORMACIÓN DEL AUTOR
Nombres- Apellidos: Kevin Alexander Urbano Lombeida
INFORMACIÓN INSTITUCIONAL
Facultad: Informática y Electrónica
Carrera: Software
Título para optar: Ingeniero de Software
Ing. Jorge Ariel Menéndez Verdecia **Director del Trabajo de Titulación**